

xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix Version 6 (v6). xv6 loosely follows the structure and style of v6, but is implemented for a modern RISC-V multiprocessor using ANSI C.

ACKNOWLEDGMENTS

xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer to Peer Communications; ISBN: 1-57398-013-7; 1st edition (June 14, 2000)). See also <https://pdos.csail.mit.edu/6.1810/>, which provides pointers to on-line resources for v6.

The following people have made contributions: Russ Cox (context switching, locking), Cliff Frey (MP), Xiao Yu (MP), Nikolai Zeldovich, and Austin Clements.

We are also grateful for the bug reports and patches contributed by Abhinavpatel00, Takahiro Aoyagi, Marcelo Arroyo, Hirbod Behnam, Silas Boyd-Wickizer, Anton Burtsev, carlclone, Ian Chen, clivezeng, Dan Cross, Cody Cutler, Mike CAT, Tej Chajed, Asami Doi, Wenyang Duan, echtwerner, eyalz800, Nelson Elhage, Saar Ettinger, Alice Ferrazzi, Nathaniel Filardo, flespark, Peter Froehlich, Yakir Goaron, Shivam Handa, Matt Harvey, Bryan Henry, jaichenhengjie, Jim Huang, MatÅ°Å; JÃ³kay, John Jolly, Alexander Kapshuk, Anders Kaseorg, kehao95, Wolfgang Keller, Jungwoo Kim, Jonathan Kimmitt, Eddie Kohler, Vadim Kolontsov, Austin Liew, Bruce Lowekamp, l0stman, Pavan Maddamsetti, Imbar Marinescu, Yandong Mao, Matan Shabtay, Hitoshi Mitake, Carmi Merimovich, mes900903, Mark Morrissey, mtasm, Joel Nider, Hayato Ohhashi, OptimisticSide, papparapa, phosphagos, Harry Porter, Greg Price, Zheng qhuo, Quancheng, RayAndrew, Jude Rich, segfault, Ayan Shafqat, Eldar Sheybek, Yongming Shen, Fumiya Shigemitsu, snoire, Taojie, Cam Tenny, tyfkda, Warren Toomey, Stephen Tu, Alissa Tung, Rafael Ubal, unicornx, Amane Uehara, Pablo Ventura, Luc Videau, Xi Wang, WaheedHafez, Keiichi Watanabe, Lucas Wolf, Nicolas Wolovick, wxdao, Grant Wu, x653, Andy Zhang, Jindong Zhang, Icenowy Zheng, ZhuyU1997, and Zou Chang Wei.

ERROR REPORTS

Please send errors and suggestions to Frans Kaashoek and Robert Morris (kaashoek,rtm@mit.edu). The main purpose of xv6 is as a teaching operating system for MIT's 6.1810, so we are more interested in simplifications and clarifications than new features.

BUILDING AND RUNNING XV6

You will need a RISC-V "newlib" tool chain from <https://github.com/riscv/riscv-gnu-toolchain>, and qemu compiled for riscv64-softhmmu. Once they are installed, and in your shell search path, you can run "make qemu".

The numbers to the left of the file names in the table are sheet numbers.
The source code has been printed in a double column format with fifty
lines per column, giving one hundred lines per sheet (or page).
Thus there is a convenient relationship between line numbers and sheet numbers.

# basic headers	45 kernel/fs.h
09 kernel/types.h	46 kernel/file.h
01 kernel/param.h	46 kernel/bio.c
02 kernel/memlayout.h	48 kernel/sleeplock.c
03 kernel/defs.h	49 kernel/log.c
05 kernel/riscv.h	52 kernel/fs.c
09 kernel/types.h	61 kernel/file.c
10 kernel/elf.h	63 kernel/sysfile.c
	70 kernel/exec.c
# entering xv6	
10 kernel/entry.S	# pipes
11 kernel/start.c	72 kernel/pipe.c
12 kernel/main.c	
	# string operations
# locks	73 kernel/string.c
13 kernel/spinlock.h	
13 kernel/spinlock.c	# low-level hardware
	75 kernel/plic.c
# processes	
15 kernel/vm.c	75 kernel/console.c
20 kernel/proc.h	78 kernel/uart.c
22 kernel/proc.c	80 kernel/virtio_disk.c
31 kernel/swtch.S	
31 kernel/kalloc.c	# user-level
	83 user/init.c
# system calls	84 user/sh.c
33 kernel/trampoline.S	
34 kernel/kernelvec.S	# link
35 kernel/trap.c	91 kernel/kernel.ld
38 kernel/syscall.h	
38 kernel/syscall.c	
41 kernel/sysproc.c	
# file system	
43 kernel/buf.h	
43 kernel/sleeplock.h	
44 kernel/fcntl.h	
44 kernel/stat.h	

The source listing is preceded by a cross-reference that lists every defined constant, struct, global variable, and function in xv6. Each entry gives, on the same line as the name, the line number (or, in a few cases, numbers) where the name is defined. Successive lines in an entry list the line numbers where the name is used. For example, this entry:

```
swtch 2658
      0374 2428 2466 2657 2658
```

indicates that swtch is defined on line 2658 and is mentioned on five lines on sheets 03, 24, and 26.

acquire 1371	backcmd 8496 8814
0412 1371 1375 2305 2323	8496 8521 8633 8814 8816
2542 2546 2593 2600 2621	8942 9055 9090
2629 2686 2743 2814 2828	BACKSPACE 7574
2840 2859 2877 2887 3041	7574 7586 7719 7726
3218 3232 3256 3736 4187	balloc 5317
4216 4711 4783 4803 4811	5317 5337 5690 5703 5711
4873 4885 4905 5104 5127	BBLOCK 4560
5146 5196 5217 5511 5544	4560 5324 5357
5610 5628 6133 6154 6168	begin_op 5102
7263 7284 7314 7648 7709	0367 2588 5102 5663 6183
7884 7958 8221 8303	6287 6510 6577 6697 6762
acquiresleep 4871	6780 6807 7038
0420 4718 4732 4871 5561 5617	bfree 5352
alloc3_desc 8203	5352 5735 5745 5748
8203 8230	bget 4707
allocpid 2301	4707 4736 4756
2301 2333	binit 4685
allocproc 2318	0313 1232 4685
2318 2456 2510	bmap 5683
alloc_desc 8155	5683 5721 5784 5819
8155 8206	BPB 4557
argaddr 3917	4557 4560 5323 5325 5358
0437 3917 3929 4134 6018	bpin 4801
6438 6457 6484 6832 6876	0317 4801 5208
6959 6991	bread 4752
argfd 6374	0314 4752 5027 5029 5054
6374 6423 6440 6459 6471	5070 5161 5162 5284 5306
6485 6905	5324 5357 5466 5491 5564
argint 3908	5655 5708 5741 5787 5822
0435 3908 4113 4157 4158	6040 6944 6970
4184 4205 4236 6379 6439	brelse 4776
6458 6693 6781 6782 6906 6907	0315 4776 4779 5034 5035
argraw 3885	5061 5078 5165 5166 5286
3885 3902 3910 3919	5309 5330 5335 5364 5472
argstr 3926	5475 5500 5572 5661 5717
0436 3926 6507 6574 6694	5747 5791 5795 5825 5829
6763 6783 6808 6833	6062 6946 6974
BACK 8461	BSIZE 4504
8461 8632 8820 9089	4309 4504 4526 4551 4557
5008 5030 5163 5307 5784	8702 8752 8755 8757 8758
5788 5789 5815 5819 5823	8759 8760 8763 8764 8766
5824 6044 6280 8219 8256	8768 8769 8770 8771 8772
buf 4300	8773 8774 8775 8776 8779
0301 0314 0315 0316 0317	8780 8782 8784 8785 8786
0318 0366 0485 3876 3879	8787 8788 8789 8800 8801
3881 3926 3930 4300 4307	8803 8805 8806 8807 8808
4308 4672 4676 4681 4687	8809 8810 8813 8814 8816
4694 4706 4709 4751 4754	8818 8819 8820 8821 8822

4766 4776 4801 4809 4957	8912 8913 8914 8915 8917
5027 5029 5054 5055 5061	8921 8924 8930 8931 8934
5070 5071 5077 5078 5161	8937 8939 8942 8946 8948
5162 5192 5270 5282 5304	8950 8953 8955 8958 8960
5320 5354 5462 5488 5555	8963 8964 8975 8978 8981
5655 5686 5730 5776 5811	8985 9000 9003 9008 9012
6040 6367 6942 6956 7605	9013 9016 9021 9022 9028
7618 7622 7625 7627 7660	9037 9038 9044 9045 9051
7717 7741 7882 7894 8016	9052 9061 9064 9066 9072
8049 8217 8325 8546 8554	9073 9078 9084 9090 9091 9094
8560 8562 8565 8566 8567	commit 5171
8571 8575 8651 8654 8655	5003 5145 5171
8656 8664 8676 8677 8684	CONSOLE 4640
bunpin 4809	4640 7765 7766 8369
0318 4809 5033	consoleinit 7757
bwrite 4766	0321 1218 7757
0316 4766 4769 5031 5077 5164	consoleintr 7707
bzero 5302	0322 7707 7971
5302 5331	consoleread 7641
C 7575	7641 7765
3047 7575 7662 7712 7715	consolewrite 7616
7722 7729 7743	7616 7766
clockintr 3733	consputc 7584
3733 3783	0323 7584 7719 7726 7738
cmd 8465	context 2051
8465 8477 8486 8487 8492	0302 0409 2051 2073 2149
8493 8498 8513 8514 8518	2355 2356 2357 2693 2734
8527 8530 8535 8585 8597	copyin 1921
8601 8609 8633 8636 8677	0471 1921 2923 3868 7295
8679 8680 8681 8688 8689	copyinstr 1955
8690 8694 8696 8697 8698	0472 1955 3879
copyout 1883	0402 2904 5789 7673
0470 1883 2635 2908 6071	elfhdr 1005
6212 6887 6888 6978 7109	1005 7032
7120 7326	ELF_MAGIC 1002
count_free_mem 3251	1002 7051
0362 3251 4253	ELF_PROG_LOAD 1036
cpu 2071	1036 7061
0392 1306 1365 1390 1407	end_op 5123
1439 1471 2071 2078 2208	0368 2590 5123 5667 6185
2277 2281 2290 2672	6292 6512 6519 6537 6546
cpuid 2269	6579 6613 6619 6703 6708
0382 1217 1244 2269 2280	6714 6721 6729 6751 6765
3735 7521 7535 7544	6769 6785 6789 6809 6815
create 6624	6820 7042 7078 7155
6624 6701 6764 6784	EXEC 8457
c_sstatus 0581	8457 8534 8759 9065
0581 0844	execcmd 8469 8753
devintr 3756	8469 8522 8535 8753 8755

3568 3620 3715 3756	9021 9027 9028 9056 9066
devsw 4633	FCR 7830
4633 4638 6115 6232 6234	7830 7870
6263 6265 7765 7766	FCR_FIFO_CLEAR 7832
dfstat 6007	7832 7870
6007 6065	FCR_FIFO_ENABLE 7831
dinode 4530	7831 7870
4530 4551 5463 5467 5489	fdalloc 6403
5492 5556 5565 5656	6403 6425 6725 6880
dirent 4567	fetchaddr 3861
4567 5856 5884 6555 6570	0439 3861 6841
dirlink 5881	fetchstr 3876
0339 5881 5896 6530 6658 6662	0438 3876 3930 6851
dirlookup 5853	file 4600
0340 5853 5859 5863 5888	0303 0329 0330 0331 0333
5975 6589 6634	0334 0335 0371 2150 2582
DIRSIZ 4563	4600 5271 5295 6111 6118
4563 4569 5848 5900 5936	6128 6131 6134 6151 6152
5937 5992 6504 6571 6627	6164 6166 6203 6222 6253
either_copyin 2919	6365 6374 6377 6403 6420
0403 2919 5824 7625	6434 6453 6469 6481 6689
either_copyout 2904	6872 6904 7208 7222 7568
8357 8478 8587 8588 8764	4572 6955
8772 8972	getcmd 8651
filealloc 6129	8651 8676
0329 6129 6725 7228	gettoken 8856
fileclose 6164	8856 8941 8945 8957 8970
0330 2583 6164 6170 6474	8971 9007 9011 9033
6727 6883 6884 6892 6893	growproc 2469
7254 7256	0385 2469 4162
filedup 6152	holding 1436
0331 2531 6152 6156 6427	0413 1374 1404 1436 2722
fileinit 6122	holdingsleep 4901
0332 1234 6122	0422 4768 4778 4901 5583
fileread 6222	ialloc 5459
0333 6222 6241 6442	0341 5459 5477 6643
filestat 6203	IBLOCK 4554
0334 6203 6487	4554 5466 5491 5564 5655
filewrite 6253	idup 5542
0335 6253 6302 6462	0342 2532 5542 5962
flags2perm 7013	IER 7827
7013 7071	7827 7854 7873
fork1 8723	IER_RX_ENABLE 7828
8511 8602 8612 8619 8634	7828 7873
8701 8723	IER_TX_ENABLE 7829
forkret 2753	7829 7873
2217 2356 2753	iget 5507
freeproc 2366	5452 5473 5507 5527 5659
2218 2340 2348 2366 2516 2641	5871 5960

freerange 3183	iinit 5442
3161 3179 3183	0343 1233 5442
freewalk 1803	ilock 5553
1803 1811 1814 1827	0344 5553 5559 5575 5664
free_chain 8184	5965 6209 6236 6268 6288
8184 8293	6516 6529 6542 6583 6591
free_desc 8168	6632 6636 6650 6711 6812
8168 8171 8173 8189 8209	6912 7045
fsinit 5291	initlock 1361
0338 2766 5291	0414 1361 2256 2257 2259
FSMAGIC 4523	3178 3573 4689 4864 5011
4523 5294	5446 6124 7236 7759 7875 8066
fs_dump 4572	initlog 5006
0365 5006 5009 5296	itrunc 5727
initsleeplock 4862	0355 5621 5727 6746
0423 4697 4862 5448	iunlock 5581
inode 4617	0346 5581 5584 5639 5665
0304 0339 0340 0341 0342	5972 6211 6239 6274 6291
0344 0345 0346 0347 0348	6525 6750 6818 6914
0350 0351 0352 0353 0354	iunlockput 5637
0355 2151 4606 4617 5438	0347 5637 5967 5976 5979
5448 5452 5458 5486 5506	6518 6531 6534 6545 6596
5509 5515 5541 5542 5553	6607 6611 6618 6635 6639
5581 5608 5637 5654 5658	6644 6671 6679 6680 6713
5683 5727 5759 5773 5808	6720 6728 6768 6788 6814
5852 5853 5881 5885 5954	7077 7154
5957 5989 6000 6505 6552	iupdate 5486
6569 6623 6626 6690 6760	0348 5486 5623 5753 5838
6776 6804 7009 7033 7165	6524 6544 6605 6610 6654
INPUT_BUF_SIZE 7604	6668 6678
7604 7605 7660 7717 7734	kalloc 3228
7741 7743	0359 1527 1575 1623 1706
install_trans 5019	1763 1851 2016 2237 2239
5019 5085 5176	2339 3228 6848 7230 8120
intr_get 0852	8121 8122 8124
0852 1472 2728 3712	KERNBASE 0243
intr_off 0842	0243 0244 1544
0842 2682 3661	kerneltrap 3703
intr_on 0835	3458 3487 3703 3711 3713 3719
0835 1478 2681 3617	kernelvec 3461
IPB 4551	3459 3461 3566 3580 3598
4551 4554 5467 5492 5565 5656	kernel_pagetable 1513
iput 5608	1513 1574 1590
0345 2589 5608 5640 5666	kexec 7027
5889 5983 6184 6535 6819	0326 2774 6855 7027
ireclaim 5651	kexit 2572
0356 5297 5651 5658	0383 2572 3609 3635 4114
isdiempty 6552	kfork 2503
6552 6559 6595	0384 2503 4127

ismapped 2029	kfree 3205
0473 2013 2029	0360 1733 1771 1817 1855
ISR 7833	2022 2369 3188 3205 3211
7833 7956	6858 6864 7252 7273
killed 2883	5074 5075 5086 5104 5106
0390 2137 2379 2651 2861	5107 5108 5111 5113 5114
2878 2883 2888 3045 3047	5127 5128 5129 5130 5131
3608 3634 4190 7286 7316 7653	5133 5138 5140 5146 5147
killforeground 3036	5148 5149 5150 5160 5161
0406 3036 7731	5162 5173 5177 5196 5197
kinit 3176	5199 5202 5203 5206 5207
0361 1224 3176	5209 5211 5217 5218 5219
kill 2854	5220 5221 5224
0389 2854 4206	LOGBLOCKS 0160
KSTACK 0252	0160 4986 5109 5197
0252 2240 2261	logheader 4984
kvminit 1572	4984 4996 5008 5009 5055 5071
0457 1225 1572	log_write 5192
kvminithart 1585	0366 5192 5200 5308 5329
0458 1226 1245 1585	5363 5471 5499 5714 5828
kvmmake 1523	LSR 7837
1523 1574	7837 7919 7933 7959
kvmmmap 1564	LSR_RX_READY 7838
0459 1531 1534 1537 1541	7838 7933
1544 1548 1552 1564 1567 2241	LSR_TX_IDLE 7839
kwait 2615	7839 7919 7959
0399 2615 4135	major 4612
LCR 7834	4532 4608 4612 4625 5494
7834 7857 7867	5567 6232 6234 6263 6265
LCR_BAUD_LATCH 7836	6624 6651 6719 6735 6778
7836 7857	6781 6784
LCR_EIGHT_BITS 7835	MAKE_SATP 0766
7835 7867	0766 1590 2782 3644
LIST 8460	mappages 1661
8460 8600 8807 9083	0460 1566 1661 1668 1671
listcmd 8490 8801	1674 1682 1769 1854 2020
8490 8523 8601 8801 8803	2404 2412 2419
8946 9057 9084	MAXARG 0158
loadseg 7165	0158 6828 7031 7103
7009 7074 7165 7174	MAXARGS 8463
log 4989 5000	8463 8471 8472 9040
4989 5000 5011 5012 5013	MAXFILE 4527
5023 5025 5027 5029 5054	4527 5815
5057 5058 5059 5070 5073	MAXOPBLOCKS 0159
0159 0160 0161 5108 6280	6205 6380 6406 6473 6805
MAXPATH 0163	6874 6978 7036 7081 7282
0163 6504 6507 6571 6574	7311 7653
6687 6694 6759 6763 6777	namecmp 5846

6783 6803 6808 6828 6833	0349 5846 5866 6586
MAXVA 0933	namei 5990
0248 0933 1615 1641 1890	0350 2459 5990 6511 6707
memcmp 7364	6808 7041
0426 7364	nameiparent 6001
memmove 7380	0351 5955 5970 5982 6001
0427 1853 1908 1936 2910	6527 6578 6629
2925 5030 5163 5285 5498	namex 5955
5571 5937 5939 6945 6973	5955 5993 6003
7380 7407	NBUF 0161
memset 7353	0161 4676 4694
0428 1528 1579 1625 1709	NCPU 0151
1768 2019 2355 3214 3239	0151 1110 2078 2208 3004
5307 5469 6600 6836 7353	NDEV 0155
8125 8126 8127 8654 8758	0155 6115 6232 6263 6719
8769 8785 8806 8819	NDIRECT 4525
min 5273	4525 4527 4536 4629 5688
5273 5788 5823	5697 5702 5706 5733 5740
minor 4613	5741 5748 5749
4533 4613 4626 5495 5568	NELEM 0492
6624 6652 6778 6782 6784	0492 2953 2986 4040 6838
MSTATUS_MPP_MASK 0513	6857 6863
0513 1118	nextpid 2214
MSTATUS_MPP_S 0515	2214 2256 2306 2307
0515 1119	NFILE 0153
mycpu 2278	0153 6118 6134
0392 1390 1439 1463 1464	NINDIRECT 4526
1465 1471 2278 2290 2672	4526 4527 5700 5743
2724 2733 2734 2735	NINODE 0154
myproc 2287	0154 5438 5447 5515
0393 2008 2287 2472 2507	NOFILE 0152
2574 2619 2720 2742 2757	0152 2150 2529 2580 6380 6408
2805 2839 2906 2921 3600	NPROC 0150
3656 3723 3863 3878 3887	0150 2210 2236 2258 2322
4037 4121 4159 4173 4190	2560 2626 2685 2838 2858
4240 4878 4906 5962 6071	2950 2983 3040
NULLDEV 4641	8914 8955 9017
4641 8375	parseline 8935
nu\terminate 9052	8912 8924 8935 8946 9008
8915 8930 9052 9073 9079	parsepipe 8951
9080 9085 9086 9091	8913 8939 8951 8958
O_APPEND 4405	parseredirs 8964
4405 6743	8964 9012 9031 9042
O_CREATE 4403	pde_t 0959 0959
4403 6700 8978 8981	0959 0959 1623 7009
O_RDONLY 4400	peek 8901
4400 6712 8592 8975	8901 8925 8940 8944 8956
O_RDWR 4402	8969 9005 9009 9024 9032
4402 6742 8368 8370 8668	PGROUNDDOWN 0909

O_TRUNC 4404	0909 1889 1926 1961 2012
4404 6745 8592 8978	PGROUNDUP 0908
O_WRONLY 4401	0908 1761 1789 1790 1791
4401 6741 6742 8978 8981	1826 3186 7087
PA2PTE 0918	PGSHIFT 0906
0918 1626 1683	0906 0926
panic 8716	PGSIZE 0905
0378 1375 1405 1473 1475	0248 0252 0265 0267 0905
1567 1577 1616 1668 1671	0908 0909 1528 1531 1534
1674 1682 1723 1814 1875	1541 1552 1579 1625 1667
2239 2577 2609 2723 2725	1670 1677 1686 1687 1709
2727 2729 2776 3211 3594	1722 1725 1762 1768 1769
3711 3713 3719 3902 4736	1790 1826 1844 1853 1854
4769 4779 5009 5130 5198	1862 1905 1912 1933 1940
5200 5295 5361 5527 5559	1965 1984 2019 2020 2241
5575 5584 5721 5859 5863	2357 2404 2412 2421 3187
5896 6156 6170 6241 6302	3209 3214 3239 3670 6851
6559 6594 6602 7174 8071	7067 7089 7093 7095 7171
8103 8110 8115 8117 8124	7175 7178 8125 8126 8127
8171 8173 8323 8512 8532	PHYSTOP 0244
8611 8716 8729 8928 8972	0244 1548 3179 3210
9006 9010 9036 9041	pid_lock 2215
parseblock 9001	2215 2256 2305 2308
9001 9006 9025	PIPE 8459
parsecmd 8918	8459 8608 8786 9077
8513 8702 8918	pipe 7212
parseexec 9017	0305 0372 0373 0374 4016
4605 6181 6230 6261 7212	1357 1507 2008 2072 2131
7224 7230 7236 7240 7244	2142 2205 2210 2212 2218
7261 7279 7308 8610 8611	2234 2236 2240 2254 2258
pipealloc 7222	2259 2261 2286 2291 2317
0371 6877 7222	2320 2322 2366 2387 2454
pipeclose 7261	2472 2506 2507 2556 2558
0372 6181 7261	2560 2574 2617 2619 2626
pipecmd 8484 8780	2671 2674 2685 2692 2697
8484 8524 8609 8780 8782	2720 2742 2757 2805 2836
8958 9058 9078	2838 2856 2858 2875 2883
piperead 7308	2906 2921 2946 2950 2978
0373 6230 7308	2983 3005 3038 3040 3555
PIPESIZE 7210	3600 3656 3855 3863 3878
7210 7214 7290 7297 7325	3887 4037 4106 4858 5267
pipewrite 7279	6113 6205 6362 6406 6805
0374 6261 7279	6874 7005 7036 7205 7282
PLIC 0233	7311 7572 7809
0233 0234 0235 0236 0237	procdump 2934
0238 1537 7514 7515	0404 2934 7713
plicinit 7511	procinit 2252
0478 1230 7511	0394 1227 2252
plicinithart 7519	proc_freepagetable 2443

0479 1231 1247 7519	0388 2372 2443 7142 7152
plc_claim 7533	proc_mapstacks 2232
0480 3764 7533	0386 1555 2232
plc_complete 7542	proc_pagetable 2387
0481 3778 7542	0387 2346 2387 7054
PLIC_SCLAIM 0238	proghdr 1024
0238 7536 7545	1024 7034
PLIC_SENABLE 0236	psprocdump 2966
0236 7525	0405 2966 4026 4225
PLIC_SPRIORITY 0237	PTE2PA 0920
0237 7528	0920 1621 1651 1732 1810 1849
pop_off 1469	PTE_FLAGS 0922
0417 1430 1469 1473 1475	0922 1850
2292 7924	PTE_R 0912
prepare_return 3654	0912 1531 1534 1537 1541
0447 2781 3641 3654	1545 1549 1552 1770 1808
proc 2131	2021 2241 2405 2413 2423
0306 0387 0390 0391 0393	pte_t 0900
0468 0900 1612 1619 1638	5002 5014 5082
1665 1720 1807 1839 1871	REDIR 8458
1886 2031	8458 8584 8770 9071
PTE_U 0915	redircmd 8475 8764
0915 1649 1770 1876 2021 2423	8475 8525 8585 8764 8766
PTE_V 0911	8975 8978 8981 9059 9072
0911 1620 1626 1647 1681	Reg 7815
1683 1729 1808 1813 1847 2035	7815 7817 7818
PTE_W 0913	release 1402
0913 1531 1534 1537 1541	0415 1402 1405 2308 2327
1549 1808 1902 2021 2241	2341 2349 2463 2517 2540
2413 2423 2479 7019 7090	2544 2548 2605 2637 2638
PTE_X 0914	2642 2643 2646 2652 2700
0914 1545 1552 1808 2405 7017	2746 2760 2815 2827 2844
push_off 1456	2866 2869 2879 2889 3046
0416 1373 1456 2289 7911	3050 3221 3236 3262 3739
PX 0927	4191 4196 4218 4717 4731
0927 1619 1629	4795 4805 4813 4879 4889
PXMASK 0925	4907 5114 5140 5150 5211
0925 0927	5224 5518 5534 5546 5619
PXSHIFT 0926	5632 6137 6141 6158 6172
0926 0927	6178 7272 7275 7287 7302
R 8020	7317 7334 7654 7685 7753
8020 8068 8069 8070 8075	7900 7964 8295 8332
8079 8083 8086 8094 8098	releasesleep 4883
8101 8106 8109 8113 8130	0421 4781 4883 5586 5626
8133 8134 8135 8136 8137	reparent 2556
8138 8141 8149 8285 8311	2556 2596
rc_sstatus 0589	RHR 7823
0589 1460	7823 7934
readi 5773	ROOTDEV 0156

0352 5773 5862 5895 6237	0156 2766 5960 6023 6040
6558 6559 7048 7059 7179	6962 6970
ReadReg 7817	ROOTINO 4503
7817 7919 7933 7934 7956 7959	4503 5960
readsb 5280 6940	RTC_BASE_ADDR 0228
5280 5293 6023 6940 6962	0228 1541
read_head 5052	run 3166
5052 5084	2942 2974 3166 3167 3172
recover_from_log 5082	3207 3216 3230 3253
runcmd 8518	6022 6023 6038 6040 6050
8514 8518 8532 8597 8603	6067 6940 6945 6954 6962
8605 8617 8624 8636 8702	6964 6965 6966 6967 6970
RUNNING 2128	sched 2717
2128 2691 2726 2727 2942	0396 2608 2717 2723 2725
2974 3003 3044	2727 2729 2745 2821
r_mcounteren 0817	scheduler 2669
0817 1158	0395 1250 2669
r_menvcfg 0735	setkilled 2875
0735 1155	0391 2875 3631
r_mhartid 0504	sfence_vma 0892
0504 1143	0892 1588 1593
r_mstatus 0519	SHARED_PAGE 0267
0519 1117	0267 2420 2431 2447
r_satp 0777	SIE_SEIE 0616
0777 3669	0616 1132
r_scause 0786	SIE_STIE 0617
0786 3605 3622 3624 3628	0617 1132
3708 3758	skipelem 5923
r_sepc 0660	5923 5964
0660 3603 3630 3706 3717	sleep 2803
r_sie 0619	0397 2657 2803 2940 2972
0619 1132	4194 4864 4875 5107 5111
r_sstatus 0559	5221 7292 7320 7657 7891
0559 0854 3593 3678 3707	8233 8289
r_stval 0802	sleeplock 4351
0802 3623 3630 3718	0308 0420 0421 0422 0423
r_time 0826	4305 4351 4621 4668 4859
0826 1161 3745	4862 4871 4883 4901 4955
r_tp 0869	5268 6110 6364 7207 7566
0869 2271 3672	8014 8355
safestrcpy 7435	spinlock 1301
0429 2536 7133 7435	0307 0397 0412 0413 0414
SATP_SV39 0764	0415 0446 1301 1355 1361
0764 0766	1371 1402 1436 1506 2132
sb 5276	2204 2215 2226 2803 3157
4554 4560 5006 5012 5276	3171 3554 3560 3854 4105
5280 5285 5293 5294 5296	4353 4667 4675 4857 4954
5323 5324 5325 5357 5465	4990 5266 5437 6109 6117
5466 5491 5564 5653 5655	6361 7004 7204 7213 7565

7601 7808 7843 8013 8057 8354	3829 4000 4024
SSTATUS_SIE 0555	sys_df 6014
0555 0837 0844 0855 1460 1461	0489 3962 4000 6014
SSTATUS_SPIE 0553	SYS_dup 3810
0553 3680	3810 3981 4018
SSTATUS_SPP 0552	sys_dup 6418
0552 3593 3679 3710	3943 3981 6418
start 1114	SYS_exec 3807
1068 1114 4991 5012 5027	3807 3978 4017
5054 5070 5161	sys_exec 6826
started 1207	3940 3978 6826
1207 1239 1241	SYS_exit 3802
stat 4454	3802 3973 4015
0309 0353 4454 5265 5759	sys_exit 4110
6112 6206 6360 8353	3935 3973 4110
stati 5759	SYS_fork 3801
0353 5759 6210	3801 3972 4015
strlen 7451	sys_fork 4125
0430 3881 7105 7109 7451	3934 3972 4125
8547 8548 8571 8696 8923	SYS_freemem 3830
strncmp 7411	3830 4001 4025
0431 5848 7411	sys_freemem 4251
strncpy 7421	3963 4001 4251
0432 5900 7421	SYS_fstat 3808
superblock 4512	3808 3979 4017
0310 0365 4512 5006 5276	sys_fstat 6479
5280 6022 6940 6954	3941 3979 6479
swtch 3108	SYS_getfsdata 3831
0409 2693 2734 3107 3108	3831 4002 4025
syscall 4034	sys_getfsdata 6951
0440 3619 3856 4012 4034	3964 4002 6951
SYS_chdir 3809	SYS_getpid 3811
3809 3980 4017	3811 3982 4018
sys_chdir 6801	sys_getpid 4119
3942 3980 6801	3944 3982 4119
SYS_close 3821	SYS_gettime 3833
3821 3992 4021	3833 4004 4026
sys_close 6466	sys_gettime 6986
3954 3992 6466	3966 4004 6986
SYS_df 3829	SYS_kill 3806
3806 3977 4016	3945 3983 4151
sys_kill 4201	SYS_sem_init 3826
3939 3977 4201	3826 3997 4023
SYS_link 3819	SYS_sem_post 3828
3819 3990 4021	3828 3999 4024
sys_link 6502	SYS_sem_wait 3827
3952 3990 6502	3827 3998 4024
SYS_lseek 3823	SYS_sync 3822

3823 3994 4022
 SYS_mkdir 3820
 3820 3991 4021
 sys_mkdir 6757
 3953 3991 6757
 SYS_mknod 3817
 3817 3988 4020
 sys_mknod 6774
 3950 3988 6774
 SYS_open 3815
 3815 3986 4019
 sys_open 6685
 3948 3986 6685
 SYS_pause 3813
 3813 3984 4019
 sys_pause 4179
 3946 3984 4179
 SYS_pipe 3804
 3804 3975 4016
 sys_pipe 6869
 3937 3975 6869
 SYS_psrprocdump 3832
 3832 4003 4026
 sys_psrprocdump 4223
 3965 4003 4223
 SYS_read 3805
 3805 3976 4016
 sys_read 6432
 3938 3976 6432
 SYS_sbrk 3812
 3812 3983 4018
 sys_sbrk 4151

0443 3561 3737 3738 4188
 4189 4194 4217
 tickslock 3560
 0446 3560 3573 3736 3739
 4187 4191 4194 4196 4216 4218
 timerinit 1152
 1107 1140 1152
 TRAMPOLINE 0248
 0248 0252 0265 1552 2404
 2414 2445 2783 3664
 trampoline 3318
 1517 1552 2220 2404 2783
 3316 3318 3563 3664 9119
 TRAPFRAME 0265
 0265 0267 2412 2446 2476
 3336 3410 4171
 trapframe 2089

3822 3993 4022
 sys_sync 5215
 3955 3993 5215
 SYS_trace 3825
 3825 3996 4023
 sys_trace 4231
 3958 3996 4231
 SYS_unlink 3818
 3818 3989 4020
 sys_unlink 6567
 3951 3989 6567
 SYS_uptime 3814
 3814 3985 4019
 sys_uptime 4212
 3947 3985 4212
 SYS_wait 3803
 3803 3974 4015
 sys_wait 4131
 3936 3974 4131
 SYS_write 3816
 3816 3987 4020
 sys_write 6451
 3949 3987 6451
 SYS_yield 3824
 3824 3995 4022
 sys_yield 4245
 3957 3995 4245
 s_sstatus 0573
 0573 0837
 THR 7825
 7825 7894 7921
 ticks 3561

4451 6637 6701 6745
 UART0 0220
 0220 1531 7815
 UART0_IRQ 0221
 0221 3766 7514 7525
 uartgetc 7930
 7930 7968
 uartinit 7851
 0451 7761 7851
 uartintr 7954
 0452 3767 7954
 uartputc_sync 7908
 0454 7588 7589 7590 7592 7908
 uartwrite 7882
 0453 7627 7882
 uint16 0955 0955
 0955 0955 8043

2089 2148 2339 2368 2369	uint32 0956 0956
2370 2412 2523 2526 2774	0956 0956 1025 1026 7514
2775 3603 3613 3669 3670	7515 7525 7528 7536 7545
3671 3672 3684 3890 3892	8020 8064 8113
3894 3896 3898 3900 4039	uint64 0957 0957
4043 4048 4052 7127 7139 7141	0957 0959 0333 0334 0335
trapinit 3571	0352 0354 0362 0373 0374
0444 1228 3571	0388 0399 0402 0403 0437
trapinithart 3578	0438 0439 0459 0460 0462
0445 1229 1246 3578	0463 0464 0465 0466 0467
tx_busy 7844	0468 0469 0470 0471 0472
7844 7888 7896 7961	0473 0474 0489 0503 0506
tx_chan 7845	0518 0521 0527 0536 0558
7845 7891 7962	0561 0567 0573 0581 0588
tx_lock 7843	0589 0601 0604 0610 0618
7843 7875 7884 7891 7900	0621 0627 0634 0637 0643
7958 7964	0654 0659 0662 0668 0671
T_DEVICE 4452	0677 0683 0686 0692 0703
4452 6637 6719 6733 6784	0708 0711 0717 0720 0727
T_DIR 4450	0734 0737 0744 0752 0758
4450 5858 5966 6517 6595	0766 0771 0776 0779 0785
6603 6656 6665 6712 6764 6813	0788 0801 0804 0811 0816
T_FILE 4451	0819 0825 0828 0854 0858
0861 0868 0871 0877 0882	4635 5214 5773 5808 5862
0885 0900 0901 0918 0927	5895 5902 6013 6016 6203
0957 0959 1011 1012 1013	6222 6253 6417 6431 6436
1027 1028 1029 1030 1031	6450 6455 6465 6478 6482
1032 1124 1210 1211 1460	6501 6558 6566 6601 6684
1544 1548 1552 1564 1613	6756 6773 6800 6825 6830
1635 1636 1639 1661 1664	6841 6868 6871 6950 6953
1717 1719 1732 1752 1753	6985 6989 7009 7031 7048
1756 1769 1783 1784 1810	7059 7069 7082 7088 7116
1823 1837 1840 1854 1869	7121 7165 7169 7179 7279
1883 1885 1921 1923 1955	7308 7616 7641 8086 8133
1957 2004 2005 2007 2016	8134 8135 8136 8137 8138
2029 2052 2053 2056 2057	8219 8250 8255 8265 8700
2058 2059 2060 2061 2062	8704 8705
2063 2064 2065 2066 2067	userinit 2452
2090 2091 2092 2093 2094	0398 1237 2452
2095 2096 2097 2098 2099	userret 3401
2100 2101 2102 2103 2104	2755 2783 3400 3401
2105 2106 2107 2108 2109	USERSTACK 0164
2110 2111 2112 2113 2114	0164 7089 7093 7095
2115 2116 2117 2118 2119	usertrap 3589
2120 2121 2122 2123 2124	3317 3589 3594 3628 3671
2125 2145 2146 2240 2241	uservec 3321
2356 2404 2412 2422 2443	3320 3321 3563 3664
2471 2615 2782 2783 2784	uvmalloc 1753
2904 2919 3013 3014 3186	0462 1753 2479 7070 7089

3209 3210 3250 3254 3580	uvmclear 1869
3588 3598 3644 3664 3671	0467 1869 1875 7093
3706 3707 3708 3758 3861	uvmcopy 1837
3865 3876 3884 3917 3928	0464 1837 2515
3934 3935 3936 3937 3938	uvmcreate 1703
3939 3940 3941 3942 3943	0461 1703 2392
3944 3945 3946 3947 3948	uvmdealloc 1784
3949 3950 3951 3952 3953	0463 1765 1772 1784 2483
3954 3955 3956 3957 3958	uvmfree 1823
3959 3960 3961 3962 3963	0465 1823 2406 2415 2424 2448
3964 3965 3966 3970 4109	uvmunmap 1717
4118 4124 4130 4133 4150	0466 1717 1723 1791 1826
4153 4178 4200 4211 4222	1862 2414 2445 2446 2447
4230 4244 4250 4459 4634	VIRTIO0 0224
0224 1534 8020	0811 1158
VIRTIO0_IRQ 0225	w_medeleg 0677
0225 3768 7515 7525	0677 1130
virtio_disk_init 8062	w_menvcfg 0744
0484 1235 8062	0744 1155
virtio_disk_intr 8301	w_mepc 0536
0486 3769 8301 8323	0536 1124
virtio_disk_rw 8217	w_mideleg 0692
0485 4758 4770 8217	0692 1131
vmfault 2005	w_mstatus 0527
0474 1895 1929 2005 3623	0527 1120
wait_lock 2226	w_pmpaddr0 0758
2226 2257 2542 2544 2593	0758 1136
2605 2621 2638 2643 2652 2657	w_pmpcfg0 0752
wakeup 2834	0752 1137
0400 2563 2599 2834 3738	w_satp 0771
4888 5138 5149 7266 7269	0771 1127 1590
7291 7301 7333 7747 7962	w_sepc 0654
8179 8327	0654 3684 3728
walk 1613	w_sie 0627
0468 1613 1616 1644 1679	0627 1132
1726 1845 1873 1900 2031	w_sstatus 0567
walkaddr 1636	0567 3681 3729
0469 1636 1893 1927 1962 7172	w_stimecmp 0727
writei 5808	0727 1161 3745
0354 5808 5902 6289 6601 6602	w_stvec 0703
WriteReg 7818	0703 3580 3598 3665
7818 7854 7857 7860 7863	w_tp 0877
7867 7870 7873 7894 7921	0877 1144
write_head 5068	yield 2740
5068 5087 5175 5178	0401 2740 3639 3724 4022 4247
write_log 5156	__attribute__ 1110
5156 5174	0377 0378 0395 1110 3558
w_mcounteren 0811	4569 8514


```
0950 typedef unsigned int uint;
0951 typedef unsigned short ushort;
0952 typedef unsigned char uchar;
0953
0954 typedef unsigned char uint8;
0955 typedef unsigned short uint16;
0956 typedef unsigned int uint32;
0957 typedef unsigned long uint64;
0958
0959 typedef uint64 pde_t;
0960
0961
0962
0963
0964
0965
0966
0967
0968
0969
0970
0971
0972
0973
0974
0975
0976
0977
0978
0979
0980
0981
0982
0983
0984
0985
0986
0987
0988
0989
0990
0991
0992
0993
0994
0995
0996
0997
0998
0999
```

```
0150 #define NPROC      64          // maximum number of processes
0151 #define NCPU        8          // maximum number of CPUs
0152 #define NOFILE      16          // open files per process
0153 #define NFILE       100         // open files per system
0154 #define NINODE       50         // maximum number of active i-nodes
0155 #define NDEV         10         // maximum major device number
0156 #define ROOTDEV      1          // device number of file system root
0157                               // disk
0158 #define MAXARG       32         // max exec arguments
0159 #define MAXOPBLOCKS  10         // max # of blocks any FS op writes
0160 #define LOGBLOCKS    (MAXOPBLOCKS * 3) // max data blocks in on-disk log
0161 #define NBUF         (MAXOPBLOCKS * 3) // size of disk block cache
0162 #define FSSIZE       8000       // size of file system in blocks
0163 #define MAXPATH      128        // maximum file path name
0164 #define USERSTACK    1          // user stack pages
0165
0166
0167
0168
0169
0170
0171
0172
0173
0174
0175
0176
0177
0178
0179
0180
0181
0182
0183
0184
0185
0186
0187
0188
0189
0190
0191
0192
0193
0194
0195
0196
0197
0198
0199
```

```

0200 // Physical memory layout
0201
0202 // qemu -machine virt is set up like this,
0203 // based on qemu's hw/riscv/virt.c:
0204 //
0205 // 00001000 -- boot ROM, provided by qemu
0206 // 02000000 -- CLINT
0207 // 0C000000 -- PLIC
0208 // 10000000 -- uart0
0209 // 10001000 -- virtio disk
0210 // 80000000 -- qemu's boot ROM loads the kernel here,
0211 //             then jumps here.
0212 // unused RAM after 80000000.
0213
0214 // the kernel uses physical memory thus:
0215 // 80000000 -- entry.S, then kernel text and data
0216 // end -- start of kernel page allocation area
0217 // PHYSTOP -- end RAM used by the kernel
0218
0219 // qemu puts UART registers here in physical memory.
0220 #define UART0      0x10000000L
0221 #define UART0_IRQ  10
0222
0223 // virtio mmio interface
0224 #define VIRTIO0     0x10001000
0225 #define VIRTIO0_IRQ 1
0226
0227 // Goldfish RTC definitions
0228 #define RTC_BASE_ADDR 0x00101000L
0229 #define RTC_TIME_LOW  0x00
0230 #define RTC_TIME_HIGH 0x04
0231
0232 // qemu puts platform-level interrupt controller (PLIC) here.
0233 #define PLIC          0x0c000000L
0234 #define PLIC_PRIORITY (PLIC + 0x0)
0235 #define PLIC_PENDING  (PLIC + 0x1000)
0236 #define PLIC_SENABLE(hart) (PLIC + 0x2080 + (hart) * 0x100)
0237 #define PLIC_SPRIORITY(hart) (PLIC + 0x201000 + (hart) * 0x2000)
0238 #define PLIC_SCLAIM(hart)  (PLIC + 0x201004 + (hart) * 0x2000)
0239
0240 // the kernel expects there to be RAM
0241 // for use by the kernel and user pages
0242 // from physical address 0x80000000 to PHYSTOP.
0243 #define KERNBASE 0x80000000L
0244 #define PHYSTOP (KERNBASE + 128 * 1024 * 1024)
0245
0246 // map the trampoline page to the highest address,
0247 // in both user and kernel space.
0248 #define TRAMPOLINE (MAXVA - PGSIZE)
0249

```

```
0250 // map kernel stacks beneath the trampoline,
0251 // each surrounded by invalid guard pages.
0252 #define KSTACK(p) (TRAMPOLINE - ((p) + 1) * 2 * PGSIZE)
0253
0254 // User memory layout.
0255 // Address zero first:
0256 //   text
0257 //   original data and bss
0258 //   fixed-size stack
0259 //   expandable heap
0260 //   ...
0261 //   SHARED_PAGE (available to all user processes)
0262 //   TRAPFRAME (p->trapframe, used by the trampoline)
0263 //   TRAMPOLINE (the same page as in the kernel)
0264
0265 #define TRAPFRAME (TRAMPOLINE - PGSIZE)
0266
0267 #define SHARED_PAGE (TRAPFRAME - PGSIZE)
0268
0269
0270
0271
0272
0273
0274
0275
0276
0277
0278
0279
0280
0281
0282
0283
0284
0285
0286
0287
0288
0289
0290
0291
0292
0293
0294
0295
0296
0297
0298
0299
```

```
0300 // clang-format off
0301 struct buf;
0302 struct context;
0303 struct file;
0304 struct inode;
0305 struct pipe;
0306 struct proc;
0307 struct spinlock;
0308 struct sleeplock;
0309 struct stat;
0310 struct superblock;
0311
0312 // bio.c
0313 void          binit(void);
0314 struct buf*   bread(uint, uint);
0315 void          brelse(struct buf*);
0316 void          bwrite(struct buf*);
0317 void          bpin(struct buf*);
0318 void          bunpin(struct buf*);
0319
0320 // console.c
0321 void          consoleinit(void);
0322 void          consoleintr(int);
0323 void          consputc(int);
0324
0325 // exec.c
0326 int           kexec(char*, char**);
0327
0328 // file.c
0329 struct file*  filealloc(void);
0330 void          fileclose(struct file*);
0331 struct file*  filedup(struct file*);
0332 void          fileinit(void);
0333 int           fileread(struct file*, uint64, int n);
0334 int           filestat(struct file*, uint64 addr);
0335 int           filewrite(struct file*, uint64, int n);
0336
0337 // fs.c
0338 void          fsinit(int);
0339 int           dirlink(struct inode*, char*, uint);
0340 struct inode* dirlookup(struct inode*, char*, uint*);
0341 struct inode* ialloc(uint, short);
0342 struct inode* idup(struct inode*);
0343 void          iinit();
0344 void          ilock(struct inode*);
0345 void          iput(struct inode*);
0346 void          iunlock(struct inode*);
0347 void          iunlockput(struct inode*);
0348 void          iupdate(struct inode*);
0349 int           namecmp(const char*, const char*);
```

```

0350 struct inode*   namei(char*);
0351 struct inode*   nameiparent(char*, char*);
0352 int              readi(struct inode*, int, uint64, uint, uint);
0353 void             stati(struct inode*, struct stat*);
0354 int              writei(struct inode*, int, uint64, uint, uint);
0355 void             itrunc(struct inode*);
0356 void             ireclaim(int);
0357
0358 // kalloc.c
0359 void*            kalloc(void);
0360 void             kfree(void *);
0361 void             kinit(void);
0362 uint64           count_free_mem(void);
0363
0364 // log.c
0365 void             initlog(int, struct superblock*);
0366 void             log_write(struct buf*);
0367 void             begin_op(void);
0368 void             end_op(void);
0369
0370 // pipe.c
0371 int              pipealloc(struct file**, struct file**);
0372 void             pipeclose(struct pipe*, int);
0373 int              piperead(struct pipe*, uint64, int);
0374 int              pipewrite(struct pipe*, uint64, int);
0375
0376 // printk.c
0377 int              printk(char*, ...) __attribute__((format(printf, 1, 2)));
0378 void             panic(char*) __attribute__((noreturn));
0379 void             printkinit(void);
0380
0381 // proc.c
0382 int              cpuid(void);
0383 void             kexit(int);
0384 int              kfork(void);
0385 int              growproc(int);
0386 void             proc_mapstacks(pagetable_t);
0387 pagetable_t      proc_pagetable(struct proc *);
0388 void             proc_freepagetable(pagetable_t, uint64);
0389 int              kkill(int);
0390 int              killed(struct proc*);
0391 void             setkilled(struct proc*);
0392 struct cpu*      mycpu(void);
0393 struct proc*     myproc();
0394 void             procinit(void);
0395 void             scheduler(void) __attribute__((noreturn));
0396 void             sched(void);
0397 void             sleep(void*, struct spinlock*);
0398 void             userinit(void);
0399 int              kwait(uint64);

```

```

0400 void            wakeup(void*);
0401 void            yield(void);
0402 int             either_copyout(int user_dst, uint64 dst, void *src, uint64 len);
0403 int             either_copyin(void *dst, int user_src, uint64 src, uint64 len);
0404 void            procdump(void);
0405 void            psprocdump(void);
0406 void            killforeground(void);
0407
0408 // swtch.S
0409 void            swtch(struct context*, struct context*);
0410
0411 // spinlock.c
0412 void            acquire(struct spinlock*);
0413 int             holding(struct spinlock*);
0414 void            initlock(struct spinlock*, char*);
0415 void            release(struct spinlock*);
0416 void            push_off(void);
0417 void            pop_off(void);
0418
0419 // sleeplock.c
0420 void            acquiresleep(struct sleeplock*);
0421 void            releasesleep(struct sleeplock*);
0422 int             holdingsleep(struct sleeplock*);
0423 void            initsleeplock(struct sleeplock*, char*);
0424
0425 // string.c
0426 int             memcmp(const void*, const void*, uint);
0427 void*           memmove(void*, const void*, uint);
0428 void*           memset(void*, int, uint);
0429 char*           safestrcpy(char*, const char*, int);
0430 int             strlen(const char*);
0431 int             strncmp(const char*, const char*, uint);
0432 char*           strncpy(char*, const char*, int);
0433
0434 // syscall.c
0435 void            argint(int, int*);
0436 int             argstr(int, char*, int);
0437 void            argaddr(int, uint64 *);
0438 int             fetchstr(uint64, char*, int);
0439 int             fetchaddr(uint64, uint64*);
0440 void            syscall();
0441
0442 // trap.c
0443 extern uint     ticks;
0444 void            trapinit(void);
0445 void            trapinithart(void);
0446 extern struct spinlock tickslock;
0447 void            prepare_return(void);
0448
0449

```

```

0450 // uart.c
0451 void          uartinit(void);
0452 void          uartintr(void);
0453 void          uartwrite(char [], int);
0454 void          uartputc_sync(int);
0455
0456 // vm.c
0457 void          kvminit(void);
0458 void          kvmminithart(void);
0459 void          kvmmap(pagetable_t, uint64, uint64, uint64, int);
0460 int           mappages(pagetable_t, uint64, uint64, uint64, int);
0461 pagetable_t  uvmcreate(void);
0462 uint64        uvmalloc(pagetable_t, uint64, uint64, int);
0463 uint64        uvmdealloc(pagetable_t, uint64, uint64);
0464 int           uvmcopy(pagetable_t, pagetable_t, uint64);
0465 void          uvmfree(pagetable_t, uint64);
0466 void          uvmunmap(pagetable_t, uint64, uint64, int);
0467 void          uvmclear(pagetable_t, uint64);
0468 pte_t *       walk(pagetable_t, uint64, int);
0469 uint64        walkaddr(pagetable_t, uint64);
0470 int           copyout(pagetable_t, uint64, char *, uint64);
0471 int           copyin(pagetable_t, char *, uint64, uint64);
0472 int           copyinstr(pagetable_t, char *, uint64, uint64);
0473 int           ismapped(pagetable_t, uint64);
0474 uint64        vmfault(pagetable_t, uint64, int);
0475 extern char *shared_page;
0476
0477 // plic.c
0478 void          plicinit(void);
0479 void          plicinithart(void);
0480 int           plic_claim(void);
0481 void          plic_complete(int);
0482
0483 // virtio_disk.c
0484 void          virtio_disk_init(void);
0485 void          virtio_disk_rw(struct buf *, int);
0486 void          virtio_disk_intr(void);
0487
0488 // sysdf.c
0489 uint64        sys_df(void);
0490
0491 // number of elements in fixed-size array
0492 #define NELEM(x) (sizeof(x) / sizeof((x)[0]))
0493
0494
0495
0496
0497
0498
0499

```

```
0500 #ifndef __ASSEMBLER__
0501
0502 // which hart (core) is this?
0503 static inline uint64
0504 r_mhartid()
0505 {
0506     uint64 x;
0507     asm volatile("csrr %0, mhartid" : "=r"(x));
0508     return x;
0509 }
0510
0511 // Machine Status Register, mstatus
0512
0513 #define MSTATUS_MPP_MASK (3L << 11) // previous mode.
0514 #define MSTATUS_MPP_M    (3L << 11)
0515 #define MSTATUS_MPP_S    (1L << 11)
0516 #define MSTATUS_MPP_U    (0L << 11)
0517
0518 static inline uint64
0519 r_mstatus()
0520 {
0521     uint64 x;
0522     asm volatile("csrr %0, mstatus" : "=r"(x));
0523     return x;
0524 }
0525
0526 static inline void
0527 w_mstatus(uint64 x)
0528 {
0529     asm volatile("csrw mstatus, %0" : : "r"(x));
0530 }
0531
0532 // machine exception program counter, holds the
0533 // instruction address to which a return from
0534 // exception will go.
0535 static inline void
0536 w_mepc(uint64 x)
0537 {
0538     asm volatile("csrw mepc, %0" : : "r"(x));
0539 }
0540
0541
0542
0543
0544
0545
0546
0547
0548
0549
```

```

0550 // Supervisor Status Register, sstatus
0551
0552 #define SSTATUS_SPP (1L << 8) // Previous mode, 1=Supervisor, 0=User
0553 #define SSTATUS_SPIE (1L << 5) // Supervisor Previous Interrupt Enable
0554 #define SSTATUS_UPIE (1L << 4) // User Previous Interrupt Enable
0555 #define SSTATUS_SIE (1L << 1) // Supervisor Interrupt Enable
0556 #define SSTATUS_UIE (1L << 0) // User Interrupt Enable
0557
0558 static inline uint64
0559 r_sstatus()
0560 {
0561     uint64 x;
0562     asm volatile("csrr %0, sstatus" : "=r"(x));
0563     return x;
0564 }
0565
0566 static inline void
0567 w_sstatus(uint64 x)
0568 {
0569     asm volatile("csw sstatus, %0" : : "r"(x));
0570 }
0571
0572 static inline void
0573 s_sstatus(uint64 x)
0574 {
0575     __asm__ __volatile__("csrs sstatus, %0" ::
0576                          "rK" (x) :
0577                          "memory");
0578 }
0579
0580 static inline void
0581 c_sstatus(uint64 x)
0582 {
0583     __asm__ __volatile__("csrc sstatus, %0" ::
0584                          "rK" (x) :
0585                          "memory");
0586 }
0587
0588 static inline uint64
0589 rc_sstatus(uint64 x)
0590 {
0591     __asm__ __volatile__("csrrc %0, sstatus, %1" :
0592                          "=r" (x) :
0593                          "rK" (x) :
0594                          "memory");
0595     return x;
0596 }
0597
0598
0599

```

```
0600 // Supervisor Interrupt Pending
0601 static inline uint64
0602 r_sip()
0603 {
0604     uint64 x;
0605     asm volatile("csrr %0, sip" : "=r"(x));
0606     return x;
0607 }
0608
0609 static inline void
0610 w_sip(uint64 x)
0611 {
0612     asm volatile("csrw sip, %0" : : "r"(x));
0613 }
0614
0615 // Supervisor Interrupt Enable
0616 #define SIE_SEIE (1L << 9) // external
0617 #define SIE_STIE (1L << 5) // timer
0618 static inline uint64
0619 r_sie()
0620 {
0621     uint64 x;
0622     asm volatile("csrr %0, sie" : "=r"(x));
0623     return x;
0624 }
0625
0626 static inline void
0627 w_sie(uint64 x)
0628 {
0629     asm volatile("csrw sie, %0" : : "r"(x));
0630 }
0631
0632 // Machine-mode Interrupt Enable
0633 #define MIE_STIE (1L << 5) // supervisor timer
0634 static inline uint64
0635 r_mie()
0636 {
0637     uint64 x;
0638     asm volatile("csrr %0, mie" : "=r"(x));
0639     return x;
0640 }
0641
0642 static inline void
0643 w_mie(uint64 x)
0644 {
0645     asm volatile("csrw mie, %0" : : "r"(x));
0646 }
0647
0648
0649
```

```
0650 // supervisor exception program counter, holds the
0651 // instruction address to which a return from
0652 // exception will go.
0653 static inline void
0654 w_sepc(uint64 x)
0655 {
0656     asm volatile("csrw sepc, %0" : : "r"(x));
0657 }
0658
0659 static inline uint64
0660 r_sepc()
0661 {
0662     uint64 x;
0663     asm volatile("csrr %0, sepc" : "=r"(x));
0664     return x;
0665 }
0666
0667 // Machine Exception Delegation
0668 static inline uint64
0669 r_medeleg()
0670 {
0671     uint64 x;
0672     asm volatile("csrr %0, medeleg" : "=r"(x));
0673     return x;
0674 }
0675
0676 static inline void
0677 w_medeleg(uint64 x)
0678 {
0679     asm volatile("csrw medeleg, %0" : : "r"(x));
0680 }
0681
0682 // Machine Interrupt Delegation
0683 static inline uint64
0684 r_mideleg()
0685 {
0686     uint64 x;
0687     asm volatile("csrr %0, mideleg" : "=r"(x));
0688     return x;
0689 }
0690
0691 static inline void
0692 w_mideleg(uint64 x)
0693 {
0694     asm volatile("csrw mideleg, %0" : : "r"(x));
0695 }
0696
0697
0698
0699
```

```
0700 // Supervisor Trap-Vector Base Address
0701 // low two bits are mode.
0702 static inline void
0703 w_stvec(uint64 x)
0704 {
0705     asm volatile("csrw stvec, %0" : : "r"(x));
0706 }
0707
0708 static inline uint64
0709 r_stvec()
0710 {
0711     uint64 x;
0712     asm volatile("csrr %0, stvec" : "=r"(x));
0713     return x;
0714 }
0715
0716 // Supervisor Timer Comparison Register
0717 static inline uint64
0718 r_stimecmp()
0719 {
0720     uint64 x;
0721     // asm volatile("csrr %0, stimecmp" : "=r" (x) );
0722     asm volatile("csrr %0, 0x14d" : "=r"(x));
0723     return x;
0724 }
0725
0726 static inline void
0727 w_stimecmp(uint64 x)
0728 {
0729     // asm volatile("csrw stimecmp, %0" : : "r" (x));
0730     asm volatile("csrw 0x14d, %0" : : "r"(x));
0731 }
0732
0733 // Machine Environment Configuration Register
0734 static inline uint64
0735 r_menvcfg()
0736 {
0737     uint64 x;
0738     // asm volatile("csrr %0, menvcfg" : "=r" (x) );
0739     asm volatile("csrr %0, 0x30a" : "=r"(x));
0740     return x;
0741 }
0742
0743 static inline void
0744 w_menvcfg(uint64 x)
0745 {
0746     // asm volatile("csrw menvcfg, %0" : : "r" (x));
0747     asm volatile("csrw 0x30a, %0" : : "r"(x));
0748 }
0749
```

```
0750 // Physical Memory Protection
0751 static inline void
0752 w_pmpcfg0(uint64 x)
0753 {
0754     asm volatile("csrw pmpcfg0, %0" : : "r"(x));
0755 }
0756
0757 static inline void
0758 w_pmpaddr0(uint64 x)
0759 {
0760     asm volatile("csrw pmpaddr0, %0" : : "r"(x));
0761 }
0762
0763 // use riscv's sv39 page table scheme.
0764 #define SATP_SV39 (8L << 60)
0765
0766 #define MAKE_SATP(pagetable) (SATP_SV39 | (((uint64)pagetable) >> 12))
0767
0768 // supervisor address translation and protection;
0769 // holds the address of the page table.
0770 static inline void
0771 w_satp(uint64 x)
0772 {
0773     asm volatile("csrw satp, %0" : : "r"(x));
0774 }
0775
0776 static inline uint64
0777 r_satp()
0778 {
0779     uint64 x;
0780     asm volatile("csrr %0, satp" : "=r"(x));
0781     return x;
0782 }
0783
0784 // Supervisor Trap Cause
0785 static inline uint64
0786 r_scause()
0787 {
0788     uint64 x;
0789     asm volatile("csrr %0, scause" : "=r"(x));
0790     return x;
0791 }
0792
0793
0794
0795
0796
0797
0798
0799
```

```
0800 // Supervisor Trap Value
0801 static inline uint64
0802 r_stval()
0803 {
0804     uint64 x;
0805     asm volatile("csrr %0, stval" : "=r"(x));
0806     return x;
0807 }
0808
0809 // Machine-mode Counter-Enable
0810 static inline void
0811 w_mcounteren(uint64 x)
0812 {
0813     asm volatile("csrw mcounteren, %0" : : "r"(x));
0814 }
0815
0816 static inline uint64
0817 r_mcounteren()
0818 {
0819     uint64 x;
0820     asm volatile("csrr %0, mcounteren" : "=r"(x));
0821     return x;
0822 }
0823
0824 // machine-mode cycle counter
0825 static inline uint64
0826 r_time()
0827 {
0828     uint64 x;
0829     asm volatile("csrr %0, time" : "=r"(x));
0830     return x;
0831 }
0832
0833 // enable device interrupts
0834 static inline void
0835 intr_on()
0836 {
0837     s_sstatus(SSTATUS_SIE);
0838 }
0839
0840 // disable device interrupts
0841 static inline void
0842 intr_off()
0843 {
0844     c_sstatus(SSTATUS_SIE);
0845 }
0846
0847
0848
0849
```

```

0850 // are device interrupts enabled?
0851 static inline int
0852 intr_get()
0853 {
0854     uint64 x = r_sstatus();
0855     return (x & SSTATUS_SIE) != 0;
0856 }
0857
0858 static inline uint64
0859 r_sp()
0860 {
0861     uint64 x;
0862     asm volatile("mv %0, sp" : "=r"(x));
0863     return x;
0864 }
0865
0866 // read and write tp, the thread pointer, which xv6 uses to hold
0867 // this core's hartid (core number), the index into cpus[].
0868 static inline uint64
0869 r_tp()
0870 {
0871     uint64 x;
0872     asm volatile("mv %0, tp" : "=r"(x));
0873     return x;
0874 }
0875
0876 static inline void
0877 w_tp(uint64 x)
0878 {
0879     asm volatile("mv tp, %0" : : "r"(x));
0880 }
0881
0882 static inline uint64
0883 r_ra()
0884 {
0885     uint64 x;
0886     asm volatile("mv %0, ra" : "=r"(x));
0887     return x;
0888 }
0889
0890 // flush the TLB.
0891 static inline void
0892 sfence_vma()
0893 {
0894     // the zero, zero means flush all TLB entries.
0895     asm volatile("sfence.vma zero, zero");
0896 }
0897
0898
0899

```

```

0900 typedef uint64 pte_t;
0901 typedef uint64 *pagetable_t; // 512 PTEs
0902
0903 #endif // __ASSEMBLER__
0904
0905 #define PGSIZE 4096 // bytes per page
0906 #define PGSHIFT 12 // bits of offset within a page
0907
0908 #define PGROUNDUP(sz) (((sz) + PGSIZE - 1) & ~(PGSIZE - 1))
0909 #define PGROUNDDOWN(a) (((a)) & ~(PGSIZE - 1))
0910
0911 #define PTE_V (1L << 0) // valid
0912 #define PTE_R (1L << 1)
0913 #define PTE_W (1L << 2)
0914 #define PTE_X (1L << 3)
0915 #define PTE_U (1L << 4) // user can access
0916
0917 // shift a physical address to the right place for a PTE.
0918 #define PA2PTE(pa) (((uint64)pa) >> 12) << 10)
0919
0920 #define PTE2PA(pte) (((pte) >> 10) << 12)
0921
0922 #define PTE_FLAGS(pte) ((pte) & 0x3FF)
0923
0924 // extract the three 9-bit page table indices from a virtual address.
0925 #define PXMASK 0x1FF // 9 bits
0926 #define PXSHIFT(level) (PGSHIFT + (9 * (level)))
0927 #define PX(level, va) (((uint64)(va)) >> PXSHIFT(level)) & PXMASK)
0928
0929 // one beyond the highest possible virtual address.
0930 // MAXVA is actually one bit less than the max allowed by
0931 // Sv39, to avoid having to sign-extend virtual addresses
0932 // that have the high bit set.
0933 #define MAXVA (1L << (9 + 9 + 9 + 12 - 1))
0934
0935
0936
0937
0938
0939
0940
0941
0942
0943
0944
0945
0946
0947
0948
0949

```

```
0950 typedef unsigned int uint;
0951 typedef unsigned short ushort;
0952 typedef unsigned char uchar;
0953
0954 typedef unsigned char uint8;
0955 typedef unsigned short uint16;
0956 typedef unsigned int uint32;
0957 typedef unsigned long uint64;
0958
0959 typedef uint64 pde_t;
0960
0961
0962
0963
0964
0965
0966
0967
0968
0969
0970
0971
0972
0973
0974
0975
0976
0977
0978
0979
0980
0981
0982
0983
0984
0985
0986
0987
0988
0989
0990
0991
0992
0993
0994
0995
0996
0997
0998
0999
```

```
1000 // Format of an ELF executable file
1001
1002 #define ELF_MAGIC 0x464C457FU // "\x7FELF" in little endian
1003
1004 // File header
1005 struct elfhdr {
1006     uint magic; // must equal ELF_MAGIC
1007     uchar elf[12];
1008     ushort type;
1009     ushort machine;
1010     uint version;
1011     uint64 entry;
1012     uint64 phoff;
1013     uint64 shoff;
1014     uint flags;
1015     ushort ehsize;
1016     ushort phentsize;
1017     ushort phnum;
1018     ushort shentsize;
1019     ushort shnum;
1020     ushort shstrndx;
1021 };
1022
1023 // Program section header
1024 struct proghdr {
1025     uint32 type;
1026     uint32 flags;
1027     uint64 off;
1028     uint64 vaddr;
1029     uint64 paddr;
1030     uint64 filesz;
1031     uint64 memsz;
1032     uint64 align;
1033 };
1034
1035 // Values for Proghdr type
1036 #define ELF_PROG_LOAD 1
1037
1038 // Flag bits for Proghdr flags
1039 #define ELF_PROG_FLAG_EXEC 1
1040 #define ELF_PROG_FLAG_WRITE 2
1041 #define ELF_PROG_FLAG_READ 4
1042
1043
1044
1045
1046
1047
1048
1049
```

```
1050      # qemu -kernel loads the kernel at 0x80000000
1051      # and causes each hart (i.e. CPU) to jump there.
1052      # kernel.ld causes the following code to
1053      # be placed at 0x80000000.
1054      .section .text
1055      .global _entry
1056      _entry:
1057          # set up a stack for C.
1058          # stack0 is declared in start.c,
1059          # with a 4096-byte stack per CPU.
1060          # sp = stack0 + ((hartid + 1) * 4096)
1061          la sp, stack0
1062          li a0, 1024*4
1063          csrr a1, mhartid
1064          addi a1, a1, 1
1065          mul a0, a0, a1
1066          add sp, sp, a0
1067          # jump to start() in start.c
1068          call start
1069      spin:
1070          j spin
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
```

```

1100 #include "types.h"
1101 #include "param.h"
1102 #include "memlayout.h"
1103 #include "riscv.h"
1104 #include "defs.h"
1105
1106 void main();
1107 void timerinit();
1108
1109 // entry.S needs one stack per CPU.
1110 __attribute__((aligned(16))) char stack0[4096 * NCPU];
1111
1112 // entry.S jumps here in machine mode on stack0.
1113 void
1114 start()
1115 {
1116     // set M Previous Privilege mode to Supervisor, for mret.
1117     unsigned long x = r_mstatus();
1118     x &= ~MSTATUS_MPP_MASK;
1119     x |= MSTATUS_MPP_S;
1120     w_mstatus(x);
1121
1122     // set M Exception Program Counter to main, for mret.
1123     // requires gcc -mmodel=medany
1124     w_mepc((uint64)main);
1125
1126     // disable paging for now.
1127     w_satp(0);
1128
1129     // delegate all interrupts and exceptions to supervisor mode.
1130     w_medeleg(0xffff);
1131     w_mideleg(0xffff);
1132     w_sie(r_sie() | SIE_SEIE | SIE_STIE);
1133
1134     // configure Physical Memory Protection to give supervisor mode
1135     // access to all of physical memory.
1136     w_pmpaddr0(0x3fffffffffffffffll);
1137     w_pmpcfg0(0xf);
1138
1139     // ask for clock interrupts.
1140     timerinit();
1141
1142     // keep each CPU's hartid in its tp register, for cpuid().
1143     int id = r_mhartid();
1144     w_tp(id);
1145
1146     // switch to supervisor mode and jump to main().
1147     asm volatile("mret");
1148 }
1149

```

```
1150 // ask each hart to generate timer interrupts.
1151 void
1152 timerinit()
1153 {
1154     // enable the sstc extension (i.e. stimecmp).
1155     w_menvcfg(r_menvcfg() | (1L << 63));
1156
1157     // allow supervisor to use stimecmp and time.
1158     w_mcounteren(r_mcounteren() | 2);
1159
1160     // ask for the very first timer interrupt.
1161     w_stimecmp(r_time() + 1000000);
1162 }
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
```

```

1200 #include "types.h"
1201 #include "param.h"
1202 #include "memlayout.h"
1203 #include "riscv.h"
1204 #include "defs.h"
1205 #include "syssem.h"
1206
1207 volatile static int started = 0;
1208
1209 void nullinit(void);
1210 int nullread(int, uint64, int);
1211 int nullwrite(int, uint64, int);
1212
1213 // start() jumps here in supervisor mode on all CPUs.
1214 void
1215 main()
1216 {
1217     if (cpuid() == 0) {
1218         consoleinit();
1219         nullinit();
1220         printkinit();
1221         printk("\n");
1222         printk("xv6 kernel is booting\n");
1223         printk("\n");
1224         kinit();           // physical page allocator
1225         kvminit();         // create kernel page table
1226         kvminithart();     // turn on paging
1227         procinit();        // process table
1228         trapinit();        // trap vectors
1229         trapinithart();    // install kernel trap vector
1230         plicinit();        // set up interrupt controller
1231         plicinithart();    // ask PLIC for device interrupts
1232         binit();           // buffer cache
1233         iinit();           // inode table
1234         fileinit();        // file table
1235         virtio_disk_init(); // emulated hard disk
1236         sem_table_init();  // semaphores
1237         userinit();        // first user process
1238         __atomic_thread_fence(__ATOMIC_SEQ_CST);
1239         started = 1;
1240     } else {
1241         while (started == 0)
1242             ;
1243         __atomic_thread_fence(__ATOMIC_SEQ_CST);
1244         printk("hart %d starting\n", cpuid());
1245         kvminithart();     // turn on paging
1246         trapinithart();    // install kernel trap vector
1247         plicinithart();    // ask PLIC for device interrupts
1248     }
1249

```

```
1250     scheduler();
1251 }
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
```

```
1300 // Mutual exclusion lock.
1301 struct spinlock {
1302     uint locked; // Is the lock held?
1303
1304     // For debugging:
1305     char *name; // Name of lock.
1306     struct cpu *cpu; // The cpu holding the lock.
1307 };
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
```

```
1350 // Mutual exclusion spin locks.
1351
1352 #include "types.h"
1353 #include "param.h"
1354 #include "memlayout.h"
1355 #include "spinlock.h"
1356 #include "riscv.h"
1357 #include "proc.h"
1358 #include "defs.h"
1359
1360 void
1361 initlock(struct spinlock *lk, char *name)
1362 {
1363     lk->name = name;
1364     lk->locked = 0;
1365     lk->cpu = 0;
1366 }
1367
1368 // Acquire the lock.
1369 // Loops (spins) until the lock is acquired.
1370 void
1371 acquire(struct spinlock *lk)
1372 {
1373     push_off(); // disable interrupts to avoid deadlock.
1374     if (holding(lk))
1375         panic("acquire");
1376
1377     // On RISC-V, __atomic_exchange_n turns into an atomic swap:
1378     //     a5 = 1
1379     //     s1 = &lk->locked
1380     //     amoswap.w.aq a5, a5, (s1)
1381     //
1382     // Passing __ATOMIC_ACQUIRE to __atomic_exchange_n tells
1383     // the C compiler and the processor to not move loads or stores
1384     // past this point, to ensure that the critical section's memory
1385     // references happen strictly after the lock is acquired.
1386     while (__atomic_exchange_n(&lk->locked, 1, __ATOMIC_ACQUIRE) != 0)
1387         ;
1388
1389     // Record info about lock acquisition for holding() and debugging.
1390     lk->cpu = mycpu();
1391 }
1392
1393
1394
1395
1396
1397
1398
1399
```

```

1400 // Release the lock.
1401 void
1402 release(struct spinlock *lk)
1403 {
1404     if (!holding(lk))
1405         panic("release");
1406
1407     lk->cpu = 0;
1408
1409     // Release the lock, equivalent to lk->locked = 0.
1410     //
1411     // This code doesn't use a C assignment, since the C standard
1412     // implies that an assignment might be implemented with
1413     // multiple store instructions.
1414     //
1415     // On RISC-V, __atomic_store_n turns into a single atomic store:
1416     //     s1 = &lk->locked
1417     //     sw zero,0(s1)
1418     //
1419     // The __ATOMIC_RELEASE argument to __atomic_store_n tells the
1420     // the C compiler and the CPU to not move loads or stores past
1421     // this point, to ensure that all the stores in the critical
1422     // section are visible to other CPUs before the lock is released,
1423     // and that loads in the critical section occur strictly before
1424     // the lock is released.
1425     //
1426     // On RISC-V, this generates a fence instruction before the store:
1427     //     fence rw,w
1428     __atomic_store_n(&lk->locked, 0, __ATOMIC_RELEASE);
1429
1430     pop_off();
1431 }
1432
1433 // Check whether this cpu is holding the lock.
1434 // Interrupts must be off.
1435 int
1436 holding(struct spinlock *lk)
1437 {
1438     int r;
1439     r = (lk->locked && lk->cpu == mycpu());
1440     return r;
1441 }
1442
1443
1444
1445
1446
1447
1448
1449

```

```

1450 // push_off/pop_off are like intr_off()/intr_on() except that they are
1451 // matched:
1452 // it takes two pop_off()s to undo two push_off()s. Also, if interrupts
1453 // are initially off, then push_off, pop_off leaves them off.
1454
1455 void
1456 push_off(void)
1457 {
1458     // disable interrupts to prevent an involuntary context
1459     // switch while using mycpu().
1460     uint64 flags = rc_sstatus(SSTATUS_SIE);
1461     int old = !(flags & SSTATUS_SIE);
1462
1463     if (mycpu()->noff == 0)
1464         mycpu()->intena = old;
1465     mycpu()->noff += 1;
1466 }
1467
1468 void
1469 pop_off(void)
1470 {
1471     struct cpu *c = mycpu();
1472     if (intr_get())
1473         panic("pop_off - interruptible");
1474     if (c->noff < 1)
1475         panic("pop_off");
1476     c->noff -= 1;
1477     if (c->noff == 0 && c->intena)
1478         intr_on();
1479 }
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499

```

```

1500 #include "param.h"
1501 #include "types.h"
1502 #include "memlayout.h"
1503 #include "elf.h"
1504 #include "riscv.h"
1505 #include "defs.h"
1506 #include "spinlock.h"
1507 #include "proc.h"
1508 #include "fs.h"
1509
1510 /*
1511  * the kernel's page table.
1512  */
1513 pagetable_t kernel_pagetable;
1514
1515 extern char etext[]; // kernel.ld sets this to end of kernel code.
1516
1517 extern char trampoline[]; // trampoline.S
1518
1519 char *shared_page;
1520
1521 // Make a direct-map page table for the kernel.
1522 pagetable_t
1523 kvmmake(void)
1524 {
1525     pagetable_t kpgtbl;
1526
1527     kpgtbl = (pagetable_t)kalloc();
1528     memset(kpgtbl, 0, PGSIZE);
1529
1530     // uart registers
1531     kvmmap(kpgtbl, UART0, UART0, PGSIZE, PTE_R | PTE_W);
1532
1533     // virtio mmio disk interface
1534     kvmmap(kpgtbl, VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);
1535
1536     // PLIC
1537     kvmmap(kpgtbl, PLIC, PLIC, 0x4000000, PTE_R | PTE_W);
1538
1539     // Map Goldfish RTC
1540     // RTC_BASE_ADDR must be page-aligned (0x101000 is page-aligned)
1541     kvmmap(kpgtbl, RTC_BASE_ADDR, RTC_BASE_ADDR, PGSIZE, PTE_R | PTE_W);
1542
1543     // map kernel text executable and read-only.
1544     kvmmap(kpgtbl, KERNBASE, KERNBASE, (uint64)etext - KERNBASE,
1545           PTE_R | PTE_X);
1546
1547     // map kernel data and the physical RAM we'll make use of.
1548     kvmmap(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP - (uint64)etext,
1549           PTE_R | PTE_W);

```

```
1550 // map the trampoline for trap entry/exit to
1551 // the highest virtual address in the kernel.
1552 kvmmap(kpgtbl, TRAMPOLINE, (uint64)trampoline, PGSIZE, PTE_R | PTE_X);
1553
1554 // allocate and map a kernel stack for each process.
1555 proc_mapstacks(kpgtbl);
1556
1557 return kpgtbl;
1558 }
1559
1560 // add a mapping to the kernel page table.
1561 // only used when booting.
1562 // does not flush TLB or enable paging.
1563 void
1564 kvmmap(pagetable_t kpgtbl, uint64 va, uint64 pa, uint64 sz, int perm)
1565 {
1566     if (mappages(kpgtbl, va, sz, pa, perm) != 0)
1567         panic("kvmmap");
1568 }
1569
1570 // Initialize the kernel_pagetable, shared by all CPUs.
1571 void
1572 kvminit(void)
1573 {
1574     kernel_pagetable = kvmmake();
1575     shared_page = kalloc();
1576     if(shared_page == 0)
1577         panic("shared_page");
1578
1579     memset(shared_page, 0, PGSIZE);
1580 }
1581
1582 // Switch the current CPU's h/w page table register to
1583 // the kernel's page table, and enable paging.
1584 void
1585 kminithart()
1586 {
1587     // wait for any previous writes to the page table memory to finish.
1588     sfence_vma();
1589
1590     w_satp(MAKE_SATP(kernel_pagetable));
1591
1592     // flush stale entries from the TLB.
1593     sfence_vma();
1594 }
1595
1596
1597
1598
1599
```

```

1600 // Return the address of the PTE in page table pagetable
1601 // that corresponds to virtual address va. If alloc!=0,
1602 // create any required page-table pages.
1603 //
1604 // The risc-v Sv39 scheme has three levels of page-table
1605 // pages. A page-table page contains 512 64-bit PTEs.
1606 // A 64-bit virtual address is split into five fields:
1607 //   39..63 -- must be zero.
1608 //   30..38 -- 9 bits of level-2 index.
1609 //   21..29 -- 9 bits of level-1 index.
1610 //   12..20 -- 9 bits of level-0 index.
1611 //   0..11  -- 12 bits of byte offset within the page.
1612 pte_t *
1613 walk(pagetable_t pagetable, uint64 va, int alloc)
1614 {
1615     if (va >= MAXVA)
1616         panic("walk");
1617
1618     for (int level = 2; level > 0; level--) {
1619         pte_t *pte = &pagetable[PX(level, va)];
1620         if (*pte & PTE_V) {
1621             pagetable = (pagetable_t)PTE2PA(*pte);
1622         } else {
1623             if (!alloc || (pagetable = (pde_t *)kalloc()) == 0)
1624                 return 0;
1625             memset(pagetable, 0, PGSIZE);
1626             *pte = PA2PTE(pagetable) | PTE_V;
1627         }
1628     }
1629     return &pagetable[PX(0, va)];
1630 }
1631
1632 // Look up a virtual address, return the physical address,
1633 // or 0 if not mapped.
1634 // Can only be used to look up user pages.
1635 uint64
1636 walkaddr(pagetable_t pagetable, uint64 va)
1637 {
1638     pte_t *pte;
1639     uint64 pa;
1640
1641     if (va >= MAXVA)
1642         return 0;
1643
1644     pte = walk(pagetable, va, 0);
1645     if (pte == 0)
1646         return 0;
1647     if ((*pte & PTE_V) == 0)
1648         return 0;
1649     if ((*pte & PTE_U) == 0)

```

```

1650     return 0;
1651     pa = PTE2PA(*pte);
1652     return pa;
1653 }
1654
1655 // Create PTEs for virtual addresses starting at va that refer to
1656 // physical addresses starting at pa.
1657 // va and size MUST be page-aligned.
1658 // Returns 0 on success, -1 if walk() couldn't
1659 // allocate a needed page-table page.
1660 int
1661 mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa,
1662          int perm)
1663 {
1664     uint64 a, last;
1665     pte_t *pte;
1666
1667     if ((va % PGSIZE) != 0)
1668         panic("mappages: va not aligned");
1669
1670     if ((size % PGSIZE) != 0)
1671         panic("mappages: size not aligned");
1672
1673     if (size == 0)
1674         panic("mappages: size");
1675
1676     a = va;
1677     last = va + size - PGSIZE;
1678     for (;;) {
1679         if ((pte = walk(pagetable, a, 1)) == 0)
1680             return -1;
1681         if (*pte & PTE_V)
1682             panic("mappages: remap");
1683         *pte = PA2PTE(pa) | perm | PTE_V;
1684         if (a == last)
1685             break;
1686         a += PGSIZE;
1687         pa += PGSIZE;
1688     }
1689     return 0;
1690 }
1691
1692
1693
1694
1695
1696
1697
1698
1699

```

```

1700 // create an empty user page table.
1701 // returns 0 if out of memory.
1702 pagetable_t
1703 uvmcreate()
1704 {
1705     pagetable_t pagetable;
1706     pagetable = (pagetable_t)kalloc();
1707     if (pagetable == 0)
1708         return 0;
1709     memset(pagetable, 0, PGSIZE);
1710     return pagetable;
1711 }
1712
1713 // Remove npages of mappings starting from va. va must be
1714 // page-aligned. It's OK if the mappings don't exist.
1715 // Optionally free the physical memory.
1716 void
1717 uvmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)
1718 {
1719     uint64 a;
1720     pte_t *pte;
1721
1722     if ((va % PGSIZE) != 0)
1723         panic("uvmunmap: not aligned");
1724
1725     for (a = va; a < va + npages * PGSIZE; a += PGSIZE) {
1726         if ((pte = walk(pagetable, a, 0)) == 0) // leaf page table entry
1727             // allocated?
1728             continue;
1729         if ((*pte & PTE_V) == 0) // has physical page been allocated?
1730             continue;
1731         if (do_free) {
1732             uint64 pa = PTE2PA(*pte);
1733             kfree((void *)pa);
1734         }
1735         *pte = 0;
1736     }
1737 }
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749

```

```

1750 // Allocate PTEs and physical memory to grow a process from oldsz to
1751 // newsz, which need not be page aligned. Returns new size or 0 on error.
1752 uint64
1753 uvmalloc(pagetable_t pagetable, uint64 oldsz, uint64 newsz, int xperm)
1754 {
1755     char *mem;
1756     uint64 a;
1757
1758     if (newsz < oldsz)
1759         return oldsz;
1760
1761     oldsz = PGROUNDUP(oldsz);
1762     for (a = oldsz; a < newsz; a += PGSIZE) {
1763         mem = kalloc();
1764         if (mem == 0) {
1765             uvmdealloc(pagetable, a, oldsz);
1766             return 0;
1767         }
1768         memset(mem, 0, PGSIZE);
1769         if (mappages(pagetable, a, PGSIZE, (uint64)mem,
1770             PTE_R | PTE_U | xperm) != 0) {
1771             kfree(mem);
1772             uvmdealloc(pagetable, a, oldsz);
1773             return 0;
1774         }
1775     }
1776     return newsz;
1777 }
1778
1779 // Deallocate user pages to bring the process size from oldsz to
1780 // newsz. oldsz and newsz need not be page-aligned, nor does newsz
1781 // need to be less than oldsz. oldsz can be larger than the actual
1782 // process size. Returns the new process size.
1783 uint64
1784 uvmdealloc(pagetable_t pagetable, uint64 oldsz, uint64 newsz)
1785 {
1786     if (newsz >= oldsz)
1787         return oldsz;
1788
1789     if (PGROUNDUP(newsz) < PGROUNDUP(oldsz)) {
1790         int npages = (PGROUNDUP(oldsz) - PGROUNDUP(newsz)) / PGSIZE;
1791         uvmunmap(pagetable, PGROUNDUP(newsz), npages, 1);
1792     }
1793
1794     return newsz;
1795 }
1796
1797
1798
1799

```

```

1800 // Recursively free page-table pages.
1801 // All leaf mappings must already have been removed.
1802 void
1803 freewalk(pagetable_t pagetable)
1804 {
1805     // there are 2^9 = 512 PTEs in a page table.
1806     for (int i = 0; i < 512; i++) {
1807         pte_t pte = pagetable[i];
1808         if ((pte & PTE_V) && (pte & (PTE_R | PTE_W | PTE_X)) == 0) {
1809             // this PTE points to a lower-level page table.
1810             uint64 child = PTE2PA(pte);
1811             freewalk((pagetable_t)child);
1812             pagetable[i] = 0;
1813         } else if (pte & PTE_V) {
1814             panic("freewalk: leaf");
1815         }
1816     }
1817     kfree((void *)pagetable);
1818 }
1819
1820 // Free user memory pages,
1821 // then free page-table pages.
1822 void
1823 uvmfree(pagetable_t pagetable, uint64 sz)
1824 {
1825     if (sz > 0)
1826         uvmunmap(pagetable, 0, PGROUNDUP(sz) / PGSIZE, 1);
1827     freewalk(pagetable);
1828 }
1829
1830 // Given a parent process's page table, copy
1831 // its memory into a child's page table.
1832 // Copies both the page table and the
1833 // physical memory.
1834 // returns 0 on success, -1 on failure.
1835 // frees any allocated pages on failure.
1836 int
1837 uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
1838 {
1839     pte_t *pte;
1840     uint64 pa, i;
1841     uint flags;
1842     char *mem;
1843
1844     for (i = 0; i < sz; i += PGSIZE) {
1845         if ((pte = walk(old, i, 0)) == 0)
1846             continue; // page table entry hasn't been allocated
1847         if ((*pte & PTE_V) == 0)
1848             continue; // physical page hasn't been allocated
1849         pa = PTE2PA(*pte);

```

```

1850     flags = PTE_FLAGS(*pte);
1851     if ((mem = kalloc()) == 0)
1852         goto err;
1853     memmove(mem, (char *)pa, PGSIZE);
1854     if (mappages(new, i, PGSIZE, (uint64)mem, flags) != 0) {
1855         kfree(mem);
1856         goto err;
1857     }
1858 }
1859 return 0;
1860
1861 err:
1862     uvmunmap(new, 0, i / PGSIZE, 1);
1863     return -1;
1864 }
1865
1866 // mark a PTE invalid for user access.
1867 // used by exec for the user stack guard page.
1868 void
1869 uvmclear(pagetable_t pagetable, uint64 va)
1870 {
1871     pte_t *pte;
1872
1873     pte = walk(pagetable, va, 0);
1874     if (pte == 0)
1875         panic("uvmclear");
1876     *pte &= ~PTE_U;
1877 }
1878
1879 // Copy from kernel to user.
1880 // Copy len bytes from src to virtual address dstva in a given page table.
1881 // Return 0 on success, -1 on error.
1882 int
1883 copyout(pagetable_t pagetable, uint64 dstva, char *src, uint64 len)
1884 {
1885     uint64 n, va0, pa0;
1886     pte_t *pte;
1887
1888     while (len > 0) {
1889         va0 = PGROUNDDOWN(dstva);
1890         if (va0 >= MAXVA)
1891             return -1;
1892
1893         pa0 = walkaddr(pagetable, va0);
1894         if (pa0 == 0) {
1895             if ((pa0 = vmfault(pagetable, va0, 0)) == 0) {
1896                 return -1;
1897             }
1898         }
1899

```

```

1900     pte = walk(pagetable, va0, 0);
1901     // forbid copyout over read-only user text pages.
1902     if ((*pte & PTE_W) == 0)
1903         return -1;
1904
1905     n = PGSIZE - (dstva - va0);
1906     if (n > len)
1907         n = len;
1908     memmove((void *)(pa0 + (dstva - va0)), src, n);
1909
1910     len -= n;
1911     src += n;
1912     dstva = va0 + PGSIZE;
1913 }
1914 return 0;
1915 }
1916
1917 // Copy from user to kernel.
1918 // Copy len bytes to dst from virtual address srcva in a given page table.
1919 // Return 0 on success, -1 on error.
1920 int
1921 copyin(pagetable_t pagetable, char *dst, uint64 srcva, uint64 len)
1922 {
1923     uint64 n, va0, pa0;
1924
1925     while (len > 0) {
1926         va0 = PGROUNDDOWN(srcva);
1927         pa0 = walkaddr(pagetable, va0);
1928         if (pa0 == 0) {
1929             if ((pa0 = vmfault(pagetable, va0, 1)) == 0) {
1930                 return -1;
1931             }
1932         }
1933         n = PGSIZE - (srcva - va0);
1934         if (n > len)
1935             n = len;
1936         memmove(dst, (void *)(pa0 + (srcva - va0)), n);
1937
1938         len -= n;
1939         dst += n;
1940         srcva = va0 + PGSIZE;
1941     }
1942     return 0;
1943 }
1944
1945
1946
1947
1948
1949

```

```

1950 // Copy a null-terminated string from user to kernel.
1951 // Copy bytes to dst from virtual address srcva in a given page table,
1952 // until a '\0', or max.
1953 // Return 0 on success, -1 on error.
1954 int
1955 copyinstr(pagetable_t pagetable, char *dst, uint64 srcva, uint64 max)
1956 {
1957     uint64 n, va0, pa0;
1958     int got_null = 0;
1959
1960     while (got_null == 0 && max > 0) {
1961         va0 = PGROUNDDOWN(srcva);
1962         pa0 = walkaddr(pagetable, va0);
1963         if (pa0 == 0)
1964             return -1;
1965         n = PGSIZE - (srcva - va0);
1966         if (n > max)
1967             n = max;
1968
1969         char *p = (char *)(pa0 + (srcva - va0));
1970         while (n > 0) {
1971             if (*p == '\0') {
1972                 *dst = '\0';
1973                 got_null = 1;
1974                 break;
1975             } else {
1976                 *dst = *p;
1977             }
1978             --n;
1979             --max;
1980             p++;
1981             dst++;
1982         }
1983
1984         srcva = va0 + PGSIZE;
1985     }
1986     if (got_null) {
1987         return 0;
1988     } else {
1989         return -1;
1990     }
1991 }
1992
1993
1994
1995
1996
1997
1998
1999

```

```

2000 // allocate and map user memory if process is referencing a page
2001 // that was lazily allocated in sys_sbrk().
2002 // returns 0 if va is invalid or already mapped, or if
2003 // out of physical memory, and physical address if successful.
2004 uint64
2005 vmfault(pagetable_t pagetable, uint64 va, int read)
2006 {
2007     uint64 mem;
2008     struct proc *p = myproc();
2009
2010     if (va >= p->sz)
2011         return 0;
2012     va = PGROUNDDOWN(va);
2013     if (ismapped(pagetable, va)) {
2014         return 0;
2015     }
2016     mem = (uint64)kalloc();
2017     if (mem == 0)
2018         return 0;
2019     memset((void *)mem, 0, PGSIZE);
2020     if (mappages(p->pagetable, va, PGSIZE, mem,
2021                 PTE_W | PTE_U | PTE_R) != 0) {
2022         kfree((void *)mem);
2023         return 0;
2024     }
2025     return mem;
2026 }
2027
2028 int
2029 ismapped(pagetable_t pagetable, uint64 va)
2030 {
2031     pte_t *pte = walk(pagetable, va, 0);
2032     if (pte == 0) {
2033         return 0;
2034     }
2035     if (*pte & PTE_V) {
2036         return 1;
2037     }
2038     return 0;
2039 }
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049

```

```

2050 // Saved registers for kernel context switches.
2051 struct context {
2052     uint64 ra;
2053     uint64 sp;
2054
2055     // callee-saved
2056     uint64 s0;
2057     uint64 s1;
2058     uint64 s2;
2059     uint64 s3;
2060     uint64 s4;
2061     uint64 s5;
2062     uint64 s6;
2063     uint64 s7;
2064     uint64 s8;
2065     uint64 s9;
2066     uint64 s10;
2067     uint64 s11;
2068 };
2069
2070 // Per-CPU state.
2071 struct cpu {
2072     struct proc *proc; // The process running on this cpu, or null.
2073     struct context context; // swtch() here to enter scheduler().
2074     int noff; // Depth of push_off() nesting.
2075     int intena; // Were interrupts enabled before push_off()?
2076 };
2077
2078 extern struct cpu cpus[NCPU];
2079
2080 // per-process data for the trap handling code in trampoline.S.
2081 // sits in a page by itself just under the trampoline page in the
2082 // user page table. not specially mapped in the kernel page table.
2083 // uservec in trampoline.S saves user registers in the trapframe,
2084 // then initializes registers from the trapframe's
2085 // kernel_sp, kernel_hartid, kernel_satp, and jumps to kernel_trap.
2086 // prepare_return() and userret in trampoline.S set up
2087 // the trapframe's kernel_*, restore user registers from the
2088 // trapframe, switch to the user page table, and enter user space.
2089 struct trapframe {
2090     /* 0 */ uint64 kernel_satp; // kernel page table
2091     /* 8 */ uint64 kernel_sp; // top of process's kernel stack
2092     /* 16 */ uint64 kernel_trap; // usertrap()
2093     /* 24 */ uint64 epc; // saved user program counter
2094     /* 32 */ uint64 kernel_hartid; // saved kernel tp
2095     /* 40 */ uint64 ra;
2096     /* 48 */ uint64 sp;
2097     /* 56 */ uint64 gp;
2098     /* 64 */ uint64 tp;
2099     /* 72 */ uint64 t0;

```

```

2100  /* 80 */ uint64 t1;
2101  /* 88 */ uint64 t2;
2102  /* 96 */ uint64 s0;
2103  /* 104 */ uint64 s1;
2104  /* 112 */ uint64 a0;
2105  /* 120 */ uint64 a1;
2106  /* 128 */ uint64 a2;
2107  /* 136 */ uint64 a3;
2108  /* 144 */ uint64 a4;
2109  /* 152 */ uint64 a5;
2110  /* 160 */ uint64 a6;
2111  /* 168 */ uint64 a7;
2112  /* 176 */ uint64 s2;
2113  /* 184 */ uint64 s3;
2114  /* 192 */ uint64 s4;
2115  /* 200 */ uint64 s5;
2116  /* 208 */ uint64 s6;
2117  /* 216 */ uint64 s7;
2118  /* 224 */ uint64 s8;
2119  /* 232 */ uint64 s9;
2120  /* 240 */ uint64 s10;
2121  /* 248 */ uint64 s11;
2122  /* 256 */ uint64 t3;
2123  /* 264 */ uint64 t4;
2124  /* 272 */ uint64 t5;
2125  /* 280 */ uint64 t6;
2126 };
2127
2128 enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
2129
2130 // Per-process state
2131 struct proc {
2132     struct spinlock lock;
2133
2134     // p->lock must be held when using these:
2135     enum procstate state; // Process state
2136     void *chan;           // If non-zero, sleeping on chan
2137     int killed;           // If non-zero, have been killed
2138     int xstate;           // Exit status to be returned to parent's wait
2139     int pid;              // ProcessID
2140
2141     // wait_lock must be held when using this:
2142     struct proc *parent; // Parent process
2143
2144     // these are private to the process, so p->lock need not be held.
2145     uint64 kstack;        // Virtual address of kernel stack
2146     uint64 sz;            // Size of process memory (bytes)
2147     pagetable_t pagetable; // User page table
2148     struct trapframe *trapframe; // data page for trampoline.S
2149     struct context context; // swtch() here to run process

```

```
2150 struct file *ofile[NOFILE]; // Open files
2151 struct inode *cwd;          // Current directory
2152 char name[16];              // Process name (debugging)
2153 int ctx_switches;           // Number of context switches
2154 int mask;                   // Bitmask tracking which syscalls to trace
2155 };
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
```

```
2200 #include "types.h"
2201 #include "param.h"
2202 #include "memlayout.h"
2203 #include "riscv.h"
2204 #include "spinlock.h"
2205 #include "proc.h"
2206 #include "defs.h"
2207
2208 struct cpu cpus[NCPU];
2209
2210 struct proc proc[NPROC];
2211
2212 struct proc *initproc;
2213
2214 int nextpid = 1;
2215 struct spinlock pid_lock;
2216
2217 extern void forkret(void);
2218 static void freeproc(struct proc *p);
2219
2220 extern char trampoline[]; // trampoline.S
2221
2222 // helps ensure that wakeups of wait()ing
2223 // parents are not lost. helps obey the
2224 // memory model when using p->parent.
2225 // must be acquired before any p->lock.
2226 struct spinlock wait_lock;
2227
2228 // Allocate a page for each process's kernel stack.
2229 // Map it high in memory, followed by an invalid
2230 // guard page.
2231 void
2232 proc_mapstacks(pagetable_t kpgtbl)
2233 {
2234     struct proc *p;
2235
2236     for (p = proc; p < &proc[NPROC]; p++) {
2237         char *pa = kalloc();
2238         if (pa == 0)
2239             panic("kalloc");
2240         uint64 va = KSTACK((int)(p - proc));
2241         kvmmap(kpgtbl, va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
2242     }
2243 }
2244
2245
2246
2247
2248
2249
```

```
2250 // initialize the proc table.
2251 void
2252 procinit(void)
2253 {
2254     struct proc *p;
2255
2256     initlock(&pid_lock, "nextpid");
2257     initlock(&wait_lock, "wait_lock");
2258     for (p = proc; p < &proc[NPROC]; p++) {
2259         initlock(&p->lock, "proc");
2260         p->state = UNUSED;
2261         p->kstack = KSTACK((int)(p - proc));
2262     }
2263 }
2264
2265 // Must be called with interrupts disabled,
2266 // to prevent race with process being moved
2267 // to a different CPU.
2268 int
2269 cpuid()
2270 {
2271     int id = r_tp();
2272     return id;
2273 }
2274
2275 // Return this CPU's cpu struct.
2276 // Interrupts must be disabled.
2277 struct cpu *
2278 mycpu(void)
2279 {
2280     int id = cpuid();
2281     struct cpu *c = &cpus[id];
2282     return c;
2283 }
2284
2285 // Return the current struct proc *, or zero if none.
2286 struct proc *
2287 myproc(void)
2288 {
2289     push_off();
2290     struct cpu *c = mycpu();
2291     struct proc *p = c->proc;
2292     pop_off();
2293     return p;
2294 }
2295
2296
2297
2298
2299
```

```

2300 int
2301 allocpid()
2302 {
2303     int pid;
2304
2305     acquire(&pid_lock);
2306     pid = nextpid;
2307     nextpid = nextpid + 1;
2308     release(&pid_lock);
2309
2310     return pid;
2311 }
2312
2313 // Look in the process table for an UNUSED proc.
2314 // If found, initialize state required to run in the kernel,
2315 // and return with p->lock held.
2316 // If there are no free procs, or a memory allocation fails, return 0.
2317 static struct proc *
2318 allocproc(void)
2319 {
2320     struct proc *p;
2321
2322     for (p = proc; p < &proc[NPROC]; p++) {
2323         acquire(&p->lock);
2324         if (p->state == UNUSED) {
2325             goto found;
2326         } else {
2327             release(&p->lock);
2328         }
2329     }
2330     return 0;
2331
2332 found:
2333     p->pid = allocpid();
2334     p->state = USED;
2335     p->ctx_switches = 0; // Initialize counter to 0 for the new process
2336     p->mask = 0; // trace mask
2337
2338     // Allocate a trapframe page.
2339     if ((p->trapframe = (struct trapframe *)kalloc()) == 0) {
2340         freeproc(p);
2341         release(&p->lock);
2342         return 0;
2343     }
2344
2345     // An empty user page table.
2346     p->pagetable = proc_pagetable(p);
2347     if (p->pagetable == 0) {
2348         freeproc(p);
2349         release(&p->lock);

```

```

2350     return 0;
2351 }
2352
2353 // Set up new context to start executing at forkret,
2354 // which returns to user space.
2355 memset(&p->context, 0, sizeof(p->context));
2356 p->context.ra = (uint64)forkret;
2357 p->context.sp = p->kstack + PGSIZE;
2358
2359 return p;
2360 }
2361
2362 // free a proc structure and the data hanging from it,
2363 // including user pages.
2364 // p->lock must be held.
2365 static void
2366 freeproc(struct proc *p)
2367 {
2368     if (p->trapframe)
2369         kfree((void *)p->trapframe);
2370     p->trapframe = 0;
2371     if (p->pagetable)
2372         proc_freepagetable(p->pagetable, p->sz);
2373     p->pagetable = 0;
2374     p->sz = 0;
2375     p->pid = 0;
2376     p->parent = 0;
2377     p->name[0] = 0;
2378     p->chan = 0;
2379     p->killed = 0;
2380     p->xstate = 0;
2381     p->state = UNUSED;
2382 }
2383
2384 // Create a user page table for a given process, with no user memory,
2385 // but with trampoline and trapframe pages.
2386 pagetable_t
2387 proc_pagetable(struct proc *p)
2388 {
2389     pagetable_t pagetable;
2390
2391     // An empty page table.
2392     pagetable = uvmcreate();
2393     if (pagetable == 0)
2394         return 0;
2395
2396
2397
2398
2399

```

```

2400 // map the trampoline code (for system call return)
2401 // at the highest user virtual address.
2402 // only the supervisor uses it, on the way
2403 // to/from user space, so not PTE_U.
2404 if (mappages(pagetable, TRAMPOLINE, PGSIZE, (uint64)trampoline,
2405             PTE_R | PTE_X) < 0) {
2406     uvmfree(pagetable, 0);
2407     return 0;
2408 }
2409
2410 // map the trapframe page just below the trampoline page, for
2411 // trampoline.S.
2412 if (mappages(pagetable, TRAPFRAME, PGSIZE, (uint64)(p->trapframe),
2413             PTE_R | PTE_W) < 0) {
2414     uvmunmap(pagetable, TRAMPOLINE, 1, 0);
2415     uvmfree(pagetable, 0);
2416     return 0;
2417 }
2418
2419 if(mappages(pagetable,
2420             SHARED_PAGE,
2421             PGSIZE,
2422             (uint64)shared_page,
2423             PTE_R | PTE_W | PTE_U) < 0){
2424     uvmfree(pagetable, 0);
2425     return 0;
2426 }
2427 /*
2428  if ( p->pid == 1 ) {
2429      printk("Mapping shared page pa=%p va=%p, PID=%d\n",
2430          shared_page,
2431          (void *)SHARED_PAGE,
2432          p->pid
2433      );
2434  }
2435 */
2436
2437 return pagetable;
2438 }
2439
2440 // Free a process's page table, and free the
2441 // physical memory it refers to.
2442 void
2443 proc_freepagetable(pagetable_t pagetable, uint64 sz)
2444 {
2445     uvmunmap(pagetable, TRAMPOLINE, 1, 0);
2446     uvmunmap(pagetable, TRAPFRAME, 1, 0);
2447     uvmunmap(pagetable, SHARED_PAGE, 1, 0);
2448     uvmfree(pagetable, sz);
2449 }

```

```
2450 // Set up first user process.
2451 void
2452 userinit(void)
2453 {
2454     struct proc *p;
2455
2456     p = allocproc();
2457     initproc = p;
2458
2459     p->cwd = namei("/");
2460
2461     p->state = RUNNABLE;
2462
2463     release(&p->lock);
2464 }
2465
2466 // Grow or shrink user memory by n bytes.
2467 // Return 0 on success, -1 on failure.
2468 int
2469 growproc(int n)
2470 {
2471     uint64 sz;
2472     struct proc *p = myproc();
2473
2474     sz = p->sz;
2475     if (n > 0) {
2476         if (sz + n > TRAPFRAME) {
2477             return -1;
2478         }
2479         if ((sz = uvmmalloc(p->pagetable, sz, sz + n, PTE_W)) == 0) {
2480             return -1;
2481         }
2482     } else if (n < 0) {
2483         sz = uvmddealloc(p->pagetable, sz, sz + n);
2484     }
2485     p->sz = sz;
2486     return 0;
2487 }
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
```

```

2500 // Create a new process, copying the parent.
2501 // Sets up child kernel stack to return as if from fork() system call.
2502 int
2503 kfork(void)
2504 {
2505     int i, pid;
2506     struct proc *np;
2507     struct proc *p = myproc();
2508
2509     // Allocate process.
2510     if ((np = allocproc()) == 0) {
2511         return -1;
2512     }
2513
2514     // Copy user memory from parent to child.
2515     if (uvmcopy(p->pagetable, np->pagetable, p->sz) < 0) {
2516         freeproc(np);
2517         release(&np->lock);
2518         return -1;
2519     }
2520     np->sz = p->sz;
2521
2522     // copy saved user registers.
2523     *(np->trapframe) = *(p->trapframe);
2524
2525     // Cause fork to return 0 in the child.
2526     np->trapframe->a0 = 0;
2527
2528     // increment reference counts on open file descriptors.
2529     for (i = 0; i < NOFILE; i++)
2530         if (p->ofile[i])
2531             np->ofile[i] = filedup(p->ofile[i]);
2532     np->cwd = idup(p->cwd);
2533
2534     np->mask = p->mask; //Ensure child processes inherit the mask from
2535                       //their parents
2536     safestrcpy(np->name, p->name, sizeof(p->name));
2537
2538     pid = np->pid;
2539
2540     release(&np->lock);
2541
2542     acquire(&wait_lock);
2543     np->parent = p;
2544     release(&wait_lock);
2545
2546     acquire(&np->lock);
2547     np->state = RUNNABLE;
2548     release(&np->lock);
2549

```

```

2550     return pid;
2551 }
2552
2553 // Pass p's abandoned children to init.
2554 // Caller must hold wait_lock.
2555 void
2556 reparent(struct proc *p)
2557 {
2558     struct proc *pp;
2559
2560     for (pp = proc; pp < &proc[NPROC]; pp++) {
2561         if (pp->parent == p) {
2562             pp->parent = initproc;
2563             wakeup(initproc);
2564         }
2565     }
2566 }
2567
2568 // Exit the current process. Does not return.
2569 // An exited process remains in the zombie state
2570 // until its parent calls wait().
2571 void
2572 kexit(int status)
2573 {
2574     struct proc *p = myproc();
2575
2576     if (p == initproc)
2577         panic("init exiting");
2578
2579     // Close all open files.
2580     for (int fd = 0; fd < NOFILE; fd++) {
2581         if (p->ofile[fd]) {
2582             struct file *f = p->ofile[fd];
2583             fileclose(f);
2584             p->ofile[fd] = 0;
2585         }
2586     }
2587
2588     begin_op();
2589     iput(p->cwd);
2590     end_op();
2591     p->cwd = 0;
2592
2593     acquire(&wait_lock);
2594
2595     // Give any children to init.
2596     reparent(p);
2597
2598     // Parent might be sleeping in wait().
2599     wakeup(p->parent);

```

```

2600  acquire(&p->lock);
2601
2602  p->xstate = status;
2603  p->state = ZOMBIE;
2604
2605  release(&wait_lock);
2606
2607  // Jump into the scheduler, never to return.
2608  sched();
2609  panic("zombie exit");
2610 }
2611
2612 // Wait for a child process to exit and return its pid.
2613 // Return -1 if this process has no children.
2614 int
2615 kwait(uint64 addr)
2616 {
2617     struct proc *pp;
2618     int havekids, pid;
2619     struct proc *p = myproc();
2620
2621     acquire(&wait_lock);
2622
2623     for (;;) {
2624         // Scan through table looking for exited children.
2625         havekids = 0;
2626         for (pp = proc; pp < &proc[NPROC]; pp++) {
2627             if (pp->parent == p) {
2628                 // make sure the child isn't still in exit() or swtch().
2629                 acquire(&pp->lock);
2630
2631                 havekids = 1;
2632                 if (pp->state == ZOMBIE) {
2633                     // Found one.
2634                     pid = pp->pid;
2635                     if (addr != 0 && copyout(p->pagetable, addr,
2636                         (char *)&pp->xstate, sizeof(pp->xstate)) < 0) {
2637                         release(&pp->lock);
2638                         release(&wait_lock);
2639                         return -1;
2640                     }
2641                     freeproc(pp);
2642                     release(&pp->lock);
2643                     release(&wait_lock);
2644                     return pid;
2645                 }
2646                 release(&pp->lock);
2647             }
2648         }
2649     }

```

```

2650 // No point waiting if we don't have any children.
2651 if (!havekids || killed(p)) {
2652     release(&wait_lock);
2653     return -1;
2654 }
2655
2656 // Wait for a child to exit.
2657 sleep(p, &wait_lock);
2658 }
2659 }
2660
2661 // Per-CPU process scheduler.
2662 // Each CPU calls scheduler() after setting itself up.
2663 // Scheduler never returns. It loops, doing:
2664 //  - choose a process to run.
2665 //  - switch to start running that process.
2666 //  - eventually that process transfers control
2667 //    via swtch back to the scheduler.
2668 void
2669 scheduler(void)
2670 {
2671     struct proc *p;
2672     struct cpu *c = mycpu();
2673
2674     c->proc = 0;
2675     for (;;) {
2676         // The most recent process to run may have had interrupts
2677         // turned off; enable them to avoid a deadlock if all
2678         // processes are waiting. Then turn them back off
2679         // to avoid a possible race between an interrupt
2680         // and wfi.
2681         intr_on();
2682         intr_off();
2683
2684         int found = 0;
2685         for (p = proc; p < &proc[NPROC]; p++) {
2686             acquire(&p->lock);
2687             if (p->state == RUNNABLE) {
2688                 // Switch to chosen process. It is the process's job
2689                 // to release its lock and then reacquire it
2690                 // before jumping back to us.
2691                 p->state = RUNNING;
2692                 c->proc = p;
2693                 swtch(&c->context, &p->context);
2694
2695                 // Process is done running for now.
2696                 // It should have changed its p->state before coming back.
2697                 c->proc = 0;
2698                 found = 1;
2699             }

```

```

2700     release(&p->lock);
2701 }
2702 if (found == 0) {
2703     // nothing to run; stop running on this core until an interrupt.
2704     asm volatile("wfi");
2705 }
2706 }
2707 }
2708
2709 // Switch to scheduler. Must hold only p->lock
2710 // and have changed proc->state. Saves and restores
2711 // intena because intena is a property of this
2712 // kernel thread, not this CPU. It should
2713 // be proc->intena and proc->noff, but that would
2714 // break in the few places where a lock is held but
2715 // there's no process.
2716 void
2717 sched(void)
2718 {
2719     int intena;
2720     struct proc *p = myproc();
2721
2722     if (!holding(&p->lock))
2723         panic("sched p->lock");
2724     if (mycpu()->noff != 1)
2725         panic("sched locks");
2726     if (p->state == RUNNING)
2727         panic("sched RUNNING");
2728     if (intr_get())
2729         panic("sched interruptible");
2730     // Increment the counter right here before swtch() executes
2731     p->ctx_switches++;
2732
2733     intena = mycpu()->intena;
2734     swtch(&p->context, &mycpu()->context);
2735     mycpu()->intena = intena;
2736 }
2737
2738 // Give up the CPU for one scheduling round.
2739 void
2740 yield(void)
2741 {
2742     struct proc *p = myproc();
2743     acquire(&p->lock);
2744     p->state = RUNNABLE;
2745     sched();
2746     release(&p->lock);
2747 }
2748
2749

```

```

2750 // A fork child's very first scheduling by scheduler()
2751 // will switch to forkret.
2752 void
2753 forkret(void)
2754 {
2755     extern char userret[];
2756     static int first = 1;
2757     struct proc *p = myproc();
2758
2759     // Still holding p->lock from scheduler.
2760     release(&p->lock);
2761
2762     if (first) {
2763         // File system initialization must be run in the context of a
2764         // regular process (e.g., because it calls sleep), and thus cannot
2765         // be run from main().
2766         fsinit(ROOTDEV);
2767
2768         first = 0;
2769         // ensure other cores see first=0.
2770         __atomic_thread_fence(__ATOMIC_SEQ_CST);
2771
2772         // We can invoke kexec() now that file system is initialized.
2773         // Put the return value (argc) of kexec into a0.
2774         p->trapframe->a0 = kexec("/init", (char *[]){"/init", 0});
2775         if (p->trapframe->a0 == -1) {
2776             panic("exec");
2777         }
2778     }
2779
2780     // return to user space, mimicing usertrap()'s return.
2781     prepare_return();
2782     uint64 satp = MAKE_SATP(p->pagetable);
2783     uint64 trampoline_userret = TRAMPOLINE + (userret - trampoline);
2784     ((void (*)(uint64))trampoline_userret)(satp);
2785 }
2786
2787
2788
2789
2790
2791
2792
2793
2794
2795
2796
2797
2798
2799

```

```

2800 // Sleep on channel chan, releasing condition lock lk.
2801 // Re-acquires lk when awakened.
2802 void
2803 sleep(void *chan, struct spinlock *lk)
2804 {
2805     struct proc *p = myproc();
2806
2807     // Must acquire p->lock in order to
2808     // change p->state and then call sched.
2809     // Once we hold p->lock, we can be
2810     // guaranteed that we won't miss any wakeup
2811     // (wakeup locks p->lock),
2812     // so it's okay to release lk.
2813
2814     acquire(&p->lock);
2815     release(lk);
2816
2817     // Go to sleep.
2818     p->chan = chan;
2819     p->state = SLEEPING;
2820
2821     sched();
2822
2823     // Tidy up.
2824     p->chan = 0;
2825
2826     // Reacquire original lock.
2827     release(&p->lock);
2828     acquire(lk);
2829 }
2830
2831 // Wake up all processes sleeping on channel chan.
2832 // Caller should hold the condition lock.
2833 void
2834 wakeup(void *chan)
2835 {
2836     struct proc *p;
2837
2838     for (p = proc; p < &proc[NPROC]; p++) {
2839         if (p != myproc()) {
2840             acquire(&p->lock);
2841             if (p->state == SLEEPING && p->chan == chan) {
2842                 p->state = RUNNABLE;
2843             }
2844             release(&p->lock);
2845         }
2846     }
2847 }
2848
2849

```

```
2850 // Kill the process with the given pid.
2851 // The victim won't exit until it tries to return
2852 // to user space (see usertrap() in trap.c).
2853 int
2854 kkill(int pid)
2855 {
2856     struct proc *p;
2857
2858     for (p = proc; p < &proc[NPROC]; p++) {
2859         acquire(&p->lock);
2860         if (p->pid == pid) {
2861             p->killed = 1;
2862             if (p->state == SLEEPING) {
2863                 // Wake process from sleep().
2864                 p->state = RUNNABLE;
2865             }
2866             release(&p->lock);
2867             return 0;
2868         }
2869         release(&p->lock);
2870     }
2871     return -1;
2872 }
2873
2874 void
2875 setkilled(struct proc *p)
2876 {
2877     acquire(&p->lock);
2878     p->killed = 1;
2879     release(&p->lock);
2880 }
2881
2882 int
2883 killed(struct proc *p)
2884 {
2885     int k;
2886
2887     acquire(&p->lock);
2888     k = p->killed;
2889     release(&p->lock);
2890     return k;
2891 }
2892
2893
2894
2895
2896
2897
2898
2899
```

```

2900 // Copy to either a user address, or kernel address,
2901 // depending on usr_dst.
2902 // Returns 0 on success, -1 on error.
2903 int
2904 either_copyout(int user_dst, uint64 dst, void *src, uint64 len)
2905 {
2906     struct proc *p = myproc();
2907     if (user_dst) {
2908         return copyout(p->pagetable, dst, src, len);
2909     } else {
2910         memmove((char *)dst, src, len);
2911         return 0;
2912     }
2913 }
2914
2915 // Copy from either a user address, or kernel address,
2916 // depending on usr_src.
2917 // Returns 0 on success, -1 on error.
2918 int
2919 either_copyin(void *dst, int user_src, uint64 src, uint64 len)
2920 {
2921     struct proc *p = myproc();
2922     if (user_src) {
2923         return copyin(p->pagetable, dst, src, len);
2924     } else {
2925         memmove(dst, (char *)src, len);
2926         return 0;
2927     }
2928 }
2929
2930 // Print a process listing to console.  For debugging.
2931 // Runs when user types ^P on console.
2932 // No lock to avoid wedging a stuck machine further.
2933 void
2934 procdump(void)
2935 {
2936     static char *states[] = {
2937         // clang-format off
2938         [UNUSED]    "unused",
2939         [USED]      "used",
2940         [SLEEPING]  "sleep ",
2941         [RUNNABLE]  "runble",
2942         [RUNNING]   "run   ",
2943         [ZOMBIE]    "zombie"
2944         // clang-format on
2945     };
2946     struct proc *p;
2947     char *state;
2948
2949     printk("\n");

```

```

2950     for (p = proc; p < &proc[NPROC]; p++) {
2951         if (p->state == UNUSED)
2952             continue;
2953         if (p->state >= 0 && p->state < NELEM(states) && states[p->state])
2954             state = states[p->state];
2955         else
2956             state = "???";
2957         printk("%d %s %s", p->pid, state, p->name);
2958         printk("\n");
2959     }
2960 }
2961
2962
2963 // Print a process listing to console, used by ps command.
2964 // No lock to avoid wedging a stuck machine further.
2965 void
2966 psprocdump(void)
2967 {
2968     static char *states[] = {
2969         // clang-format off
2970         [UNUSED]    "unused",
2971         [USED]       "used",
2972         [SLEEPING]  "sleep ",
2973         [RUNNABLE]   "runble",
2974         [RUNNING]    "run   ",
2975         [ZOMBIE]     "zombie"
2976         // clang-format on
2977     };
2978     struct proc *p;
2979     char *state;
2980
2981     // header line
2982     printk("PID\tPPID\tState\tCPU\tCS\tMem(KB)\tPages\tName\n");
2983     for (p = proc; p < &proc[NPROC]; p++) {
2984         if (p->state == UNUSED)
2985             continue;
2986         if (p->state >= 0 && p->state < NELEM(states) && states[p->state])
2987             state = states[p->state];
2988         else
2989             state = "???";
2990
2991         // Check if a parent process exists, otherwise assign 0
2992         int ppid = 0;
2993         if(p->parent != 0) {
2994             ppid = p->parent->pid;
2995         }
2996
2997
2998
2999

```

```

3000 // Find which CPU this process is currently attached to
3001 int cpu_id = -1; // -1 means it is not actively running on any CPU
3002 // core right now
3003 if(p->state == RUNNING) {
3004     for(int i = 0; i < NCPU; i++) {
3005         if(cpus[i].proc == p) {
3006             cpu_id = i;
3007             break;
3008         }
3009     }
3010 }
3011
3012 // Calculate values safely
3013 uint64 mem_kb = p->sz / 1024;
3014 uint64 page_count = p->sz / 4096; // 4096 is the standard PGSIZE in
3015 // xv6
3016
3017 // Print PID, state, the memory size (p->sz), page count and name
3018 if ( cpu_id >= 0 )
3019     printk("%d\t%d\t%s\t%d\t%d\t%ld\t%ld\t%s", p->pid, ppid, state,
3020           cpu_id, p->ctx_switches, mem_kb, page_count, p->name);
3021 else
3022     printk("%d\t%d\t%s\t-\t%d\t%ld\t%ld\t%s", p->pid, ppid, state,
3023           p->ctx_switches, mem_kb, page_count, p->name);
3024     printk("\n");
3025
3026 // Pointers to memory management structs can be outputted as
3027 // hex strings
3028 // printk(" [Arch] Pagetable: %p, Trapframe: %p\n",
3029 // p->pagetable, p->trapframe);
3030
3031 }
3032 }
3033
3034 // Helper function called by console driver
3035 void
3036 killforeground(void)
3037 {
3038     struct proc *p;
3039
3040     for(p = proc; p < &proc[NPROC]; p++) {
3041         acquire(&p->lock);
3042         // Find a running user process that isn't init (pid 1) or the
3043         // main shell (pid 2)
3044         if(p->state == RUNNING && p->pid > 2) {
3045             p->killed = 1; // Mark the process to be killed on its next trap
3046             release(&p->lock);
3047             printk("\n[Process %d killed by Ctrl+C]\n", p->pid);
3048             return;
3049         }

```

```
3050     release(&p->lock);
3051 }
3052 }
3053
3054
3055
3056
3057
3058
3059
3060
3061
3062
3063
3064
3065
3066
3067
3068
3069
3070
3071
3072
3073
3074
3075
3076
3077
3078
3079
3080
3081
3082
3083
3084
3085
3086
3087
3088
3089
3090
3091
3092
3093
3094
3095
3096
3097
3098
3099
```

```
3100 # Context switch
3101 #
3102 # void swtch(struct context *old, struct context *new);
3103 #
3104 # Save current registers in old. Load from new.
3105
3106
3107 .globl swtch
3108 swtch:
3109     sd ra, 0(a0)
3110     sd sp, 8(a0)
3111     sd s0, 16(a0)
3112     sd s1, 24(a0)
3113     sd s2, 32(a0)
3114     sd s3, 40(a0)
3115     sd s4, 48(a0)
3116     sd s5, 56(a0)
3117     sd s6, 64(a0)
3118     sd s7, 72(a0)
3119     sd s8, 80(a0)
3120     sd s9, 88(a0)
3121     sd s10, 96(a0)
3122     sd s11, 104(a0)
3123
3124     ld ra, 0(a1)
3125     ld sp, 8(a1)
3126     ld s0, 16(a1)
3127     ld s1, 24(a1)
3128     ld s2, 32(a1)
3129     ld s3, 40(a1)
3130     ld s4, 48(a1)
3131     ld s5, 56(a1)
3132     ld s6, 64(a1)
3133     ld s7, 72(a1)
3134     ld s8, 80(a1)
3135     ld s9, 88(a1)
3136     ld s10, 96(a1)
3137     ld s11, 104(a1)
3138
3139     ret
3140
3141
3142
3143
3144
3145
3146
3147
3148
3149
```

```
3150 // Physical memory allocator, for user processes,
3151 // kernel stacks, page-table pages,
3152 // and pipe buffers. Allocates whole 4096-byte pages.
3153
3154 #include "types.h"
3155 #include "param.h"
3156 #include "memlayout.h"
3157 #include "spinlock.h"
3158 #include "riscv.h"
3159 #include "defs.h"
3160
3161 void freerange(void *pa_start, void *pa_end);
3162
3163 extern char end[]; // first address after kernel.
3164                  // defined by kernel.ld.
3165
3166 struct run {
3167     struct run *next;
3168 };
3169
3170 struct {
3171     struct spinlock lock;
3172     struct run *freelist;
3173 } kmem;
3174
3175 void
3176 kinit()
3177 {
3178     initlock(&kmem.lock, "kmem");
3179     freerange(end, (void *)PHYSTOP);
3180 }
3181
3182 void
3183 freerange(void *pa_start, void *pa_end)
3184 {
3185     char *p;
3186     p = (char *)PGROUNDUP((uint64)pa_start);
3187     for (; p + PGSIZE <= (char *)pa_end; p += PGSIZE)
3188         kfree(p);
3189 }
3190
3191
3192
3193
3194
3195
3196
3197
3198
3199
```

```

3200 // Free the page of physical memory pointed at by pa,
3201 // which normally should have been returned by a
3202 // call to kalloc(). (The exception is when
3203 // initializing the allocator; see kinit above.)
3204 void
3205 kfree(void *pa)
3206 {
3207     struct run *r;
3208
3209     if (((uint64)pa % PGSIZE) != 0 || (char *)pa < end ||
3210         (uint64)pa >= PHYSTOP)
3211         panic("kfree");
3212
3213     // Fill with junk to catch dangling refs.
3214     memset(pa, 1, PGSIZE);
3215
3216     r = (struct run *)pa;
3217
3218     acquire(&kmem.lock);
3219     r->next = kmem.freelist;
3220     kmem.freelist = r;
3221     release(&kmem.lock);
3222 }
3223
3224 // Allocate one 4096-byte page of physical memory.
3225 // Returns a pointer that the kernel can use.
3226 // Returns 0 if the memory cannot be allocated.
3227 void *
3228 kalloc(void)
3229 {
3230     struct run *r;
3231
3232     acquire(&kmem.lock);
3233     r = kmem.freelist;
3234     if (r)
3235         kmem.freelist = r->next;
3236     release(&kmem.lock);
3237
3238     if (r)
3239         memset((char *)r, 5, PGSIZE); // fill with junk
3240     return (void *)r;
3241 }
3242
3243
3244
3245
3246
3247
3248
3249

```

```
3250 uint64
3251 count_free_mem(void)
3252 {
3253     struct run *r;
3254     uint64 pages = 0;
3255
3256     acquire(&kmem.lock);
3257     r = kmem.freelist;
3258     while(r){
3259         pages++;
3260         r = r->next;
3261     }
3262     release(&kmem.lock);
3263
3264     return pages * 4096; // Returns free memory in bytes
3265 }
3266
3267
3268
3269
3270
3271
3272
3273
3274
3275
3276
3277
3278
3279
3280
3281
3282
3283
3284
3285
3286
3287
3288
3289
3290
3291
3292
3293
3294
3295
3296
3297
3298
3299
```

```
3300      #
3301      # low-level code to handle traps from user space into
3302      # the kernel, and returns from kernel to user.
3303      #
3304      # the kernel maps the page holding this code
3305      # at the same virtual address (TRAMPOLINE)
3306      # in user and kernel space so that it continues
3307      # to work when it switches page tables.
3308      # kernel.ld causes this code to start at
3309      # a page boundary.
3310      #
3311
3312      #include "riscv.h"
3313      #include "memlayout.h"
3314
3315      .section trampsec
3316      .globl trampoline
3317      .globl usertrap
3318      trampoline:
3319      .align 4
3320      .globl uservec
3321      uservec:
3322      #
3323      # trap.c sets stvec to point here, so
3324      # traps from user space start here,
3325      # in supervisor mode, but with a
3326      # user page table.
3327      #
3328
3329      # save user a0 in sscratch so
3330      # a0 can be used to get at TRAPFRAME.
3331      csrw sscratch, a0
3332
3333      # each process has a separate p->trapframe memory area,
3334      # but it's mapped to the same virtual address
3335      # (TRAPFRAME) in every process's user page table.
3336      li a0, TRAPFRAME
3337
3338      # save the user registers in TRAPFRAME
3339      sd ra, 40(a0)
3340      sd sp, 48(a0)
3341      sd gp, 56(a0)
3342      sd tp, 64(a0)
3343      sd t0, 72(a0)
3344      sd t1, 80(a0)
3345      sd t2, 88(a0)
3346      sd s0, 96(a0)
3347      sd s1, 104(a0)
3348      sd a1, 120(a0)
3349      sd a2, 128(a0)
```

```

3350      sd a3, 136(a0)
3351      sd a4, 144(a0)
3352      sd a5, 152(a0)
3353      sd a6, 160(a0)
3354      sd a7, 168(a0)
3355      sd s2, 176(a0)
3356      sd s3, 184(a0)
3357      sd s4, 192(a0)
3358      sd s5, 200(a0)
3359      sd s6, 208(a0)
3360      sd s7, 216(a0)
3361      sd s8, 224(a0)
3362      sd s9, 232(a0)
3363      sd s10, 240(a0)
3364      sd s11, 248(a0)
3365      sd t3, 256(a0)
3366      sd t4, 264(a0)
3367      sd t5, 272(a0)
3368      sd t6, 280(a0)
3369
3370      # save the user a0 in p->trapframe->a0
3371      csrr t0, sscratch
3372      sd t0, 112(a0)
3373
3374      # initialize kernel stack pointer, from p->trapframe->kernel_sp
3375      ld sp, 8(a0)
3376
3377      # make tp hold the current hartid, from
3378      #      p->trapframe->kernel_hartid
3379      ld tp, 32(a0)
3380
3381      # load the address of usertrap(), from p->trapframe->kernel_trap
3382      ld t0, 16(a0)
3383
3384      # fetch the kernel page table address, from
3385      #      p->trapframe->kernel_satp.
3386      ld t1, 0(a0)
3387
3388      # wait for any previous memory operations to complete, so that
3389      # they use the user page table.
3390      sfence.vma zero, zero
3391
3392      # install the kernel page table.
3393      csrw satp, t1
3394
3395      # flush now-stale user entries from the TLB.
3396      sfence.vma zero, zero
3397
3398      # call usertrap()
3399      jalr t0

```

```
3400 .globl userret
3401 userret:
3402     # usertrap() returns here, with user satp in a0.
3403     # return from kernel to user.
3404
3405     # switch to the user page table.
3406     sfence.vma zero, zero
3407     csrw satp, a0
3408     sfence.vma zero, zero
3409
3410     li a0, TRAPFRAME
3411
3412     # restore all but a0 from TRAPFRAME
3413     ld ra, 40(a0)
3414     ld sp, 48(a0)
3415     ld gp, 56(a0)
3416     ld tp, 64(a0)
3417     ld t0, 72(a0)
3418     ld t1, 80(a0)
3419     ld t2, 88(a0)
3420     ld s0, 96(a0)
3421     ld s1, 104(a0)
3422     ld a1, 120(a0)
3423     ld a2, 128(a0)
3424     ld a3, 136(a0)
3425     ld a4, 144(a0)
3426     ld a5, 152(a0)
3427     ld a6, 160(a0)
3428     ld a7, 168(a0)
3429     ld s2, 176(a0)
3430     ld s3, 184(a0)
3431     ld s4, 192(a0)
3432     ld s5, 200(a0)
3433     ld s6, 208(a0)
3434     ld s7, 216(a0)
3435     ld s8, 224(a0)
3436     ld s9, 232(a0)
3437     ld s10, 240(a0)
3438     ld s11, 248(a0)
3439     ld t3, 256(a0)
3440     ld t4, 264(a0)
3441     ld t5, 272(a0)
3442     ld t6, 280(a0)
3443
3444     # restore user a0
3445     ld a0, 112(a0)
3446
3447     # return to user mode and user pc.
3448     # prepare_return() sets up sstatus and sepc.
3449     sret
```

```

3450      #
3451      # interrupts and exceptions while in supervisor
3452      # mode come here.
3453      #
3454      # the current stack is a kernel stack.
3455      # push registers, call kerneltrap().
3456      # when kerneltrap() returns, restore registers, return.
3457      #
3458      .globl kerneltrap
3459      .globl kernelvec
3460      .align 4
3461      kernelvec:
3462          # make room to save registers.
3463          addi sp, sp, -256
3464
3465          # save caller-saved registers.
3466          sd ra, 0(sp)
3467          # sd sp, 8(sp)
3468          sd gp, 16(sp)
3469          # sd tp, 24(sp)
3470          sd t0, 32(sp)
3471          sd t1, 40(sp)
3472          sd t2, 48(sp)
3473          sd a0, 72(sp)
3474          sd a1, 80(sp)
3475          sd a2, 88(sp)
3476          sd a3, 96(sp)
3477          sd a4, 104(sp)
3478          sd a5, 112(sp)
3479          sd a6, 120(sp)
3480          sd a7, 128(sp)
3481          sd t3, 216(sp)
3482          sd t4, 224(sp)
3483          sd t5, 232(sp)
3484          sd t6, 240(sp)
3485
3486          # call the C trap handler in trap.c
3487          call kerneltrap
3488
3489          # restore registers.
3490          ld ra, 0(sp)
3491          # ld sp, 8(sp)
3492          ld gp, 16(sp)
3493          # not tp (contains hartid), in case we moved CPUs
3494          ld t0, 32(sp)
3495          ld t1, 40(sp)
3496          ld t2, 48(sp)
3497          ld a0, 72(sp)
3498          ld a1, 80(sp)
3499          ld a2, 88(sp)

```

```
3500      ld a3, 96(sp)
3501      ld a4, 104(sp)
3502      ld a5, 112(sp)
3503      ld a6, 120(sp)
3504      ld a7, 128(sp)
3505      ld t3, 216(sp)
3506      ld t4, 224(sp)
3507      ld t5, 232(sp)
3508      ld t6, 240(sp)
3509
3510      addi sp, sp, 256
3511
3512      # return to whatever we were doing in the kernel.
3513      sret
3514
3515
3516
3517
3518
3519
3520
3521
3522
3523
3524
3525
3526
3527
3528
3529
3530
3531
3532
3533
3534
3535
3536
3537
3538
3539
3540
3541
3542
3543
3544
3545
3546
3547
3548
3549
```

```

3550 #include "types.h"
3551 #include "param.h"
3552 #include "memlayout.h"
3553 #include "riscv.h"
3554 #include "spinlock.h"
3555 #include "proc.h"
3556 #include "defs.h"
3557
3558 void exit(int) __attribute__((noreturn));
3559
3560 struct spinlock tickslock;
3561 uint ticks;
3562
3563 extern char trampoline[], uservec[];
3564
3565 // in kernelvec.S, calls kerneltrap().
3566 void kernelvec();
3567
3568 extern int devintr();
3569
3570 void
3571 trapinit(void)
3572 {
3573     initlock(&tickslock, "time");
3574 }
3575
3576 // set up to take exceptions and traps while in the kernel.
3577 void
3578 trapinithart(void)
3579 {
3580     w_stvec((uint64)kernelvec);
3581 }
3582
3583 //
3584 // handle an interrupt, exception, or system call from user space.
3585 // called from, and returns to, trampoline.S
3586 // return value is user satp for trampoline.S to switch to.
3587 //
3588 uint64
3589 usertrap(void)
3590 {
3591     int which_dev = 0;
3592
3593     if ((r_sstatus() & SSTATUS_SPP) != 0)
3594         panic("usertrap: not from user mode");
3595
3596     // send interrupts and exceptions to kerneltrap(),
3597     // since we're now in the kernel.
3598     w_stvec((uint64)kernelvec);
3599

```

```

3600 struct proc *p = myproc();
3601
3602 // save user program counter.
3603 p->trapframe->epc = r_sepc();
3604
3605 if (r_scause() == 8) {
3606     // system call
3607
3608     if (killed(p))
3609         kexit(-1);
3610
3611     // sepc points to the ecall instruction,
3612     // but we want to return to the next instruction.
3613     p->trapframe->epc += 4;
3614
3615     // an interrupt will change sepc, scause, and sstatus,
3616     // so enable only now that we're done with those registers.
3617     intr_on();
3618
3619     syscall();
3620 } else if ((which_dev = devintr()) != 0) {
3621     // ok
3622 } else if ((r_scause() == 15 || r_scause() == 13) &&
3623           vmfault(p->pagetable, r_stval(),
3624                 (r_scause() == 13) ? 1 : 0) !=
3625           0) {
3626     // page fault on lazily-allocated page
3627 } else {
3628     printk("usertrap(): unexpected scause 0x%lx pid=%d\n", r_scause(),
3629           p->pid);
3630     printk("          sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
3631     setkilled(p);
3632 }
3633
3634 if (killed(p))
3635     kexit(-1);
3636
3637 // give up the CPU if this is a timer interrupt.
3638 if (which_dev == 2)
3639     yield();
3640
3641 prepare_return();
3642
3643 // the user page table to switch to, for trampoline.S
3644 uint64 satp = MAKE_SATP(p->pagetable);
3645
3646 // return to trampoline.S; satp value in a0.
3647 return satp;
3648 }
3649

```

```

3650 //
3651 // set up trapframe and control registers for a return to user space
3652 //
3653 void
3654 prepare_return(void)
3655 {
3656     struct proc *p = myproc();
3657
3658     // we're about to switch the destination of traps from
3659     // kerneltrap() to usertrap(). because a trap from kernel
3660     // code to usertrap would be a disaster, turn off interrupts.
3661     intr_off();
3662
3663     // send syscalls, interrupts, and exceptions to uservec in trampoline.S
3664     uint64 trampoline_uservec = TRAMPOLINE + (uservec - trampoline);
3665     w_stvec(trampoline_uservec);
3666
3667     // set up trapframe values that uservec will need when
3668     // the process next traps into the kernel.
3669     p->trapframe->kernel_satp = r_satp(); // kernel page table
3670     p->trapframe->kernel_sp = p->kstack + PGSIZE; // process's kernel stack
3671     p->trapframe->kernel_trap = (uint64)usertrap;
3672     p->trapframe->kernel_hartid = r_tp(); // hartid for cpuid()
3673
3674     // set up the registers that trampoline.S's sret will use
3675     // to get to user space.
3676
3677     // set S Previous Privilege mode to User.
3678     unsigned long x = r_sstatus();
3679     x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode
3680     x |= SSTATUS_SPIE; // enable interrupts in user mode
3681     w_sstatus(x);
3682
3683     // set S Exception Program Counter to the saved user pc.
3684     w_sepc(p->trapframe->epc);
3685 }
3686
3687
3688
3689
3690
3691
3692
3693
3694
3695
3696
3697
3698
3699

```

```

3700 // interrupts and exceptions from kernel code go here via kernelfec,
3701 // on whatever the current kernel stack is.
3702 void
3703 kerneltrap()
3704 {
3705     int which_dev = 0;
3706     uint64 sepc = r_sepc();
3707     uint64 sstatus = r_sstatus();
3708     uint64 scause = r_scause();
3709
3710     if ((sstatus & SSTATUS_SPP) == 0)
3711         panic("kerneltrap: not from supervisor mode");
3712     if (intr_get() != 0)
3713         panic("kerneltrap: interrupts enabled");
3714
3715     if ((which_dev = devintr()) == 0) {
3716         // interrupt or trap from an unknown source
3717         printk("scause=0x%lx sepc=0x%lx stval=0x%lx\n", scause, r_sepc(),
3718             r_stval());
3719         panic("kerneltrap");
3720     }
3721
3722     // give up the CPU if this is a timer interrupt.
3723     if (which_dev == 2 && myproc() != 0)
3724         yield();
3725
3726     // the yield() may have caused some traps to occur,
3727     // so restore trap registers for use by kernelfec.S's sepc instruction.
3728     w_sepc(sepc);
3729     w_sstatus(sstatus);
3730 }
3731
3732 void
3733 clockintr()
3734 {
3735     if (cpuid() == 0) {
3736         acquire(&tickslock);
3737         ticks++;
3738         wakeup(&ticks);
3739         release(&tickslock);
3740     }
3741
3742     // ask for the next timer interrupt. this also clears
3743     // the interrupt request. 1000000 is about a tenth
3744     // of a second.
3745     w_stimecmp(r_time() + 1000000);
3746 }
3747
3748
3749

```

```

3750 // check if it's an external interrupt or software interrupt,
3751 // and handle it.
3752 // returns 2 if timer interrupt,
3753 // 1 if other device,
3754 // 0 if not recognized.
3755 int
3756 devintr()
3757 {
3758     uint64 scause = r_scause();
3759
3760     if (scause == 0x8000000000000009L) {
3761         // this is a supervisor external interrupt, via PLIC.
3762
3763         // irq indicates which device interrupted.
3764         int irq = plic_claim();
3765
3766         if (irq == UART0_IRQ) {
3767             uartintr();
3768         } else if (irq == VIRTIO0_IRQ) {
3769             virtio_disk_intr();
3770         } else if (irq) {
3771             printk("unexpected interrupt irq=%d\n", irq);
3772         }
3773
3774         // the PLIC allows each device to raise at most one
3775         // interrupt at a time; tell the PLIC the device is
3776         // now allowed to interrupt again.
3777         if (irq)
3778             plic_complete(irq);
3779
3780         return 1;
3781     } else if (scause == 0x8000000000000005L) {
3782         // timer interrupt.
3783         clockintr();
3784         return 2;
3785     } else {
3786         return 0;
3787     }
3788 }
3789
3790
3791
3792
3793
3794
3795
3796
3797
3798
3799

```

```
3800 // System call numbers
3801 #define SYS_fork    1
3802 #define SYS_exit    2
3803 #define SYS_wait    3
3804 #define SYS_pipe    4
3805 #define SYS_read    5
3806 #define SYS_kill    6
3807 #define SYS_exec    7
3808 #define SYS_fstat   8
3809 #define SYS_chdir   9
3810 #define SYS_dup     10
3811 #define SYS_getpid  11
3812 #define SYS_sbrk    12
3813 #define SYS_pause   13
3814 #define SYS_uptime  14
3815 #define SYS_open    15
3816 #define SYS_write   16
3817 #define SYS_mknod   17
3818 #define SYS_unlink  18
3819 #define SYS_link     19
3820 #define SYS_mkdir    20
3821 #define SYS_close    21
3822 #define SYS_sync     22
3823 #define SYS_lseek    23
3824 #define SYS_yield    24
3825 #define SYS_trace    25
3826 #define SYS_sem_init 26
3827 #define SYS_sem_wait 27
3828 #define SYS_sem_post 28
3829 #define SYS_df       29
3830 #define SYS_freemem  30
3831 #define SYS_getfsdata 31
3832 #define SYS_psprocdump 32
3833 #define SYS_gettime  33
3834
3835
3836
3837
3838
3839
3840
3841
3842
3843
3844
3845
3846
3847
3848
3849
```

```
3850 #include "types.h"
3851 #include "param.h"
3852 #include "memlayout.h"
3853 #include "riscv.h"
3854 #include "spinlock.h"
3855 #include "proc.h"
3856 #include "syscall.h"
3857 #include "defs.h"
3858
3859 // Fetch the uint64 at addr from the current process.
3860 int
3861 fetchaddr(uint64 addr, uint64 *ip)
3862 {
3863     struct proc *p = myproc();
3864     if (addr >= p->sz ||
3865         addr + sizeof(uint64) > p->sz) // both tests needed, in case
3866                                         // of overflow
3867         return -1;
3868     if (copyin(p->pagetable, (char *)ip, addr, sizeof(*ip)) != 0)
3869         return -1;
3870     return 0;
3871 }
3872
3873 // Fetch the nul-terminated string at addr from the current process.
3874 // Returns length of string, not including nul, or -1 for error.
3875 int
3876 fetchstr(uint64 addr, char *buf, int max)
3877 {
3878     struct proc *p = myproc();
3879     if (copyinstr(p->pagetable, buf, addr, max) < 0)
3880         return -1;
3881     return strlen(buf);
3882 }
3883
3884 static uint64
3885 argraw(int n)
3886 {
3887     struct proc *p = myproc();
3888     switch (n) {
3889     case 0:
3890         return p->trapframe->a0;
3891     case 1:
3892         return p->trapframe->a1;
3893     case 2:
3894         return p->trapframe->a2;
3895     case 3:
3896         return p->trapframe->a3;
3897     case 4:
3898         return p->trapframe->a4;
3899     case 5:
```

```

3900     return p->trapframe->a5;
3901 }
3902 panic("argraw");
3903 return -1;
3904 }
3905
3906 // Fetch the nth 32-bit system call argument.
3907 void
3908 argint(int n, int *ip)
3909 {
3910     *ip = argraw(n);
3911 }
3912
3913 // Retrieve an argument as a pointer.
3914 // Doesn't check for legality, since
3915 // copyin/copyout will do that.
3916 void
3917 argaddr(int n, uint64 *ip)
3918 {
3919     *ip = argraw(n);
3920 }
3921
3922 // Fetch the nth word-sized system call argument as a
3923 // null-terminated string. Copies into buf, at most max.
3924 // Returns string length if OK (not including nul), -1 if error.
3925 int
3926 argstr(int n, char *buf, int max)
3927 {
3928     uint64 addr;
3929     argaddr(n, &addr);
3930     return fetchstr(addr, buf, max);
3931 }
3932
3933 // Prototypes for the functions that handle system calls.
3934 extern uint64 sys_fork(void);
3935 extern uint64 sys_exit(void);
3936 extern uint64 sys_wait(void);
3937 extern uint64 sys_pipe(void);
3938 extern uint64 sys_read(void);
3939 extern uint64 sys_kill(void);
3940 extern uint64 sys_exec(void);
3941 extern uint64 sys_fstat(void);
3942 extern uint64 sys_chdir(void);
3943 extern uint64 sys_dup(void);
3944 extern uint64 sys_getpid(void);
3945 extern uint64 sys_sbrk(void);
3946 extern uint64 sys_pause(void);
3947 extern uint64 sys_uptime(void);
3948 extern uint64 sys_open(void);
3949 extern uint64 sys_write(void);

```

```

3950 extern uint64 sys_mknod(void);
3951 extern uint64 sys_unlink(void);
3952 extern uint64 sys_link(void);
3953 extern uint64 sys_mkdir(void);
3954 extern uint64 sys_close(void);
3955 extern uint64 sys_sync(void);
3956 extern uint64 sys_lseek(void);
3957 extern uint64 sys_yield(void);
3958 extern uint64 sys_trace(void);
3959 extern uint64 sys_sem_init(void);
3960 extern uint64 sys_sem_wait(void);
3961 extern uint64 sys_sem_post(void);
3962 extern uint64 sys_df(void);
3963 extern uint64 sys_freemem(void);
3964 extern uint64 sys_getfsdata(void);
3965 extern uint64 sys_psprocdump(void);
3966 extern uint64 sys_gettime(void);
3967
3968 // An array mapping syscall numbers from syscall.h
3969 // to the function that handles the system call.
3970 static uint64 (*syscalls[])(void) = {
3971     // clang-format off
3972     [SYS_fork]      sys_fork,
3973     [SYS_exit]      sys_exit,
3974     [SYS_wait]      sys_wait,
3975     [SYS_pipe]      sys_pipe,
3976     [SYS_read]      sys_read,
3977     [SYS_kill]      sys_kill,
3978     [SYS_exec]      sys_exec,
3979     [SYS_fstat]     sys_fstat,
3980     [SYS_chdir]     sys_chdir,
3981     [SYS_dup]       sys_dup,
3982     [SYS_getpid]    sys_getpid,
3983     [SYS_sbrk]      sys_sbrk,
3984     [SYS_pause]     sys_pause,
3985     [SYS_uptime]    sys_uptime,
3986     [SYS_open]      sys_open,
3987     [SYS_write]     sys_write,
3988     [SYS_mknod]     sys_mknod,
3989     [SYS_unlink]    sys_unlink,
3990     [SYS_link]      sys_link,
3991     [SYS_mkdir]     sys_mkdir,
3992     [SYS_close]     sys_close,
3993     [SYS_sync]      sys_sync,
3994     [SYS_lseek]     sys_lseek,
3995     [SYS_yield]     sys_yield,
3996     [SYS_trace]     sys_trace,
3997     [SYS_sem_init]  sys_sem_init,
3998     [SYS_sem_wait]  sys_sem_wait,
3999     [SYS_sem_post]  sys_sem_post,

```

```

4000 [SYS_df] sys_df,
4001 [SYS_freemem] sys_freemem,
4002 [SYS_getfsdata] sys_getfsdata,
4003 [SYS_psprocdump] sys_psprocdump,
4004 [SYS_gettime] sys_gettime,
4005
4006 // clang-format on
4007 };
4008
4009 // Array containing readable names mapping to numbers
4010 /*
4011  * grep -P "\s+[SYS_[0-9a-z_]+\]\s+sys_[0-9a-z_]+\s*$"
4012  *     kernel/syscall.c | sed 's/[[:space:]]\+sys_/ "/;s/,/,/'
4013  */
4014 static char *syscall_names[] = {
4015     [SYS_fork] "fork", [SYS_exit] "exit", [SYS_wait] "wait",
4016     [SYS_pipe] "pipe", [SYS_read] "read", [SYS_kill] "kill",
4017     [SYS_exec] "exec", [SYS_fstat] "fstat", [SYS_chdir] "chdir",
4018     [SYS_dup] "dup", [SYS_getpid] "getpid", [SYS_sbrk] "sbrk",
4019     [SYS_pause] "pause", [SYS_uptime] "uptime", [SYS_open] "open",
4020     [SYS_write] "write", [SYS_mknod] "mknod", [SYS_unlink] "unlink",
4021     [SYS_link] "link", [SYS_mkdir] "mkdir", [SYS_close] "close",
4022     [SYS_sync] "sync", [SYS_lseek] "lseek", [SYS_yield] "yield",
4023     [SYS_trace] "trace", [SYS_sem_init] "sem_init",
4024     [SYS_sem_wait] "sem_wait", [SYS_sem_post] "sem_post", [SYS_df] "df",
4025     [SYS_freemem] "freemem", [SYS_getfsdata] "getfsdata",
4026     [SYS_psprocdump] "psprocdump", [SYS_gettime] "gettime",
4027 };
4028
4029
4030 // https://github.com/chandan-shrivastava/tweaked-xv6/blob/main/
4031 // kernel/syscall.c
4032
4033 void
4034 syscall(void)
4035 {
4036     int num;
4037     struct proc *p = myproc();
4038
4039     num = p->trapframe->a7;
4040     if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
4041         // Use num to lookup the system call function for num, call it,
4042         // and store its return value in p->trapframe->a0
4043         p->trapframe->a0 = syscalls[num]();
4044
4045         // Check if the current syscall's bit is enabled in the mask
4046         if ((p->mask >> num) & 1) {
4047             printk("%d: %d %s -> %ld\n", p->pid, num, syscall_names[num],
4048                 p->trapframe->a0);
4049         }
4050     }

```

```
4050     } else {
4051         printk("%d %s: unknown sys call %d\n", p->pid, p->name, num);
4052         p->trapframe->a0 = -1;
4053     }
4054 }
4055
4056
4057
4058
4059
4060
4061
4062
4063
4064
4065
4066
4067
4068
4069
4070
4071
4072
4073
4074
4075
4076
4077
4078
4079
4080
4081
4082
4083
4084
4085
4086
4087
4088
4089
4090
4091
4092
4093
4094
4095
4096
4097
4098
4099
```

```
4100 #include "types.h"
4101 #include "riscv.h"
4102 #include "defs.h"
4103 #include "param.h"
4104 #include "memlayout.h"
4105 #include "spinlock.h"
4106 #include "proc.h"
4107 #include "vm.h"
4108
4109 uint64
4110 sys_exit(void)
4111 {
4112     int n;
4113     argint(0, &n);
4114     kexit(n);
4115     return 0; // not reached
4116 }
4117
4118 uint64
4119 sys_getpid(void)
4120 {
4121     return myproc()->pid;
4122 }
4123
4124 uint64
4125 sys_fork(void)
4126 {
4127     return kfork();
4128 }
4129
4130 uint64
4131 sys_wait(void)
4132 {
4133     uint64 p;
4134     argaddr(0, &p);
4135     return kwait(p);
4136 }
4137
4138
4139
4140
4141
4142
4143
4144
4145
4146
4147
4148
4149
```

```

4150 uint64
4151 sys_sbrk(void)
4152 {
4153     uint64 addr;
4154     int t;
4155     int n;
4156
4157     argint(0, &n);
4158     argint(1, &t);
4159     addr = myproc()->sz;
4160
4161     if (t == SBRK_EAGER || n < 0) {
4162         if (growproc(n) < 0) {
4163             return -1;
4164         }
4165     } else {
4166         // Lazily allocate memory for this process: increase its memory
4167         // size but don't allocate memory. If the processes uses the
4168         // memory, vmfault() will allocate it.
4169         if (addr + n < addr)
4170             return -1;
4171         if (addr + n > TRAPFRAME)
4172             return -1;
4173         myproc()->sz += n;
4174     }
4175     return addr;
4176 }
4177
4178 uint64
4179 sys_pause(void)
4180 {
4181     int n;
4182     uint ticks0;
4183
4184     argint(0, &n);
4185     if (n < 0)
4186         n = 0;
4187     acquire(&tickslock);
4188     ticks0 = ticks;
4189     while (ticks - ticks0 < n) {
4190         if (killed(myproc())) {
4191             release(&tickslock);
4192             return -1;
4193         }
4194         sleep(&ticks, &tickslock);
4195     }
4196     release(&tickslock);
4197     return 0;
4198 }
4199

```

```
4200 uint64
4201 sys_kill(void)
4202 {
4203     int pid;
4204
4205     argint(0, &pid);
4206     return kkill(pid);
4207 }
4208
4209 // return how many clock tick interrupts have occurred
4210 // since start.
4211 uint64
4212 sys_uptime(void)
4213 {
4214     uint xticks;
4215
4216     acquire(&tickslock);
4217     xticks = ticks;
4218     release(&tickslock);
4219     return xticks;
4220 }
4221
4222 uint64
4223 sys_psprocdump(void)
4224 {
4225     psprocdump();
4226     return 0;
4227 }
4228
4229
4230 uint64
4231 sys_trace(void)
4232 {
4233     int mask;
4234
4235     // Extract the integer argument from register a0
4236     argint(0, &mask);
4237     if( mask < 0 )
4238         return -1;
4239
4240     myproc()->mask = mask;
4241     return 0;
4242 }
4243
4244 uint64
4245 sys_yield(void)
4246 {
4247     yield();
4248     return 0;
4249 }
```

```
4250 uint64
4251 sys_freemem(void)
4252 {
4253     return count_free_mem();
4254 }
4255
4256
4257
4258
4259
4260
4261
4262
4263
4264
4265
4266
4267
4268
4269
4270
4271
4272
4273
4274
4275
4276
4277
4278
4279
4280
4281
4282
4283
4284
4285
4286
4287
4288
4289
4290
4291
4292
4293
4294
4295
4296
4297
4298
4299
```

```
4300 struct buf {
4301     int valid; // has data been read from disk?
4302     int disk;  // does disk "own" buf?
4303     uint dev;
4304     uint blockno;
4305     struct sleeplock lock;
4306     uint refcnt;
4307     struct buf *prev; // LRU cache list
4308     struct buf *next;
4309     uchar data[BSIZE];
4310 };
4311
4312
4313
4314
4315
4316
4317
4318
4319
4320
4321
4322
4323
4324
4325
4326
4327
4328
4329
4330
4331
4332
4333
4334
4335
4336
4337
4338
4339
4340
4341
4342
4343
4344
4345
4346
4347
4348
4349
```

```
4350 // Long-term locks for processes
4351 struct sleeplock {
4352     uint locked;           // Is the lock held?
4353     struct spinlock lk;    // spinlock protecting this sleep lock
4354
4355     // For debugging:
4356     char *name;            // Name of lock.
4357     int pid;               // Process holding lock
4358 };
4359
4360
4361
4362
4363
4364
4365
4366
4367
4368
4369
4370
4371
4372
4373
4374
4375
4376
4377
4378
4379
4380
4381
4382
4383
4384
4385
4386
4387
4388
4389
4390
4391
4392
4393
4394
4395
4396
4397
4398
4399
```

```
4400 #define O_RDONLY 0x000
4401 #define O_WRONLY 0x001
4402 #define O_RDWR 0x002
4403 #define O_CREATE 0x200
4404 #define O_TRUNC 0x400
4405 #define O_APPEND 0x800
4406
4407
4408
4409
4410
4411
4412
4413
4414
4415
4416
4417
4418
4419
4420 #define O_TRUNC 0x400
4421
4422
4423
4424
4425
4426
4427
4428
4429
4430
4431
4432
4433
4434
4435
4436
4437
4438
4439
4440
4441
4442
4443
4444
4445
4446
4447
4448
4449
```

```
4450 #define T_DIR      1 // Directory
4451 #define T_FILE      2 // File
4452 #define T_DEVICE    3 // Device
4453
4454 struct stat {
4455     int dev;        // File system's disk device
4456     uint ino;       // Inode number
4457     short type;     // Type of file
4458     short nlink;    // Number of links to file
4459     uint64 size;    // Size of file in bytes
4460 };
4461
4462
4463
4464
4465
4466
4467
4468
4469
4470
4471
4472
4473
4474
4475
4476
4477
4478
4479
4480
4481
4482
4483
4484
4485
4486
4487
4488
4489
4490
4491
4492
4493
4494
4495
4496
4497
4498
4499
```

```

4500 // On-disk file system format.
4501 // Both the kernel and user programs use this header file.
4502
4503 #define ROOTINO 1    // root i-number
4504 #define BSIZE 1024  // block size
4505
4506 // Disk layout:
4507 // [ boot block | super block | log | inode blocks |
4508 //                               free bit map | data blocks]
4509 //
4510 // mkfs computes the super block and builds an initial file system. The
4511 // super block describes the disk layout:
4512 struct superblock {
4513     uint magic;        // Must be FSMAGIC
4514     uint size;         // Size of file system image (blocks)
4515     uint nblocks;      // Number of data blocks
4516     uint ninodes;      // Number of inodes.
4517     uint nlog;         // Number of log blocks
4518     uint logstart;     // Block number of first log block
4519     uint inodestart;   // Block number of first inode block
4520     uint bmapstart;    // Block number of first free map block
4521 };
4522
4523 #define FSMAGIC 0x10203040
4524
4525 #define NDIRECT 12
4526 #define NINDIRECT (BSIZE / sizeof(uint))
4527 #define MAXFILE (NDIRECT + NINDIRECT)
4528
4529 // On-disk inode structure
4530 struct dinode {
4531     short type;        // File type
4532     short major;       // Major device number (T_DEVICE only)
4533     short minor;       // Minor device number (T_DEVICE only)
4534     short nlink;       // Number of links to inode in file system
4535     uint size;         // Size of file (bytes)
4536     uint addrs[NDIRECT + 1]; // Data block addresses
4537 };
4538
4539
4540
4541
4542
4543
4544
4545
4546
4547
4548
4549

```

```

4550 // Inodes per block.
4551 #define IPB (BSIZE / sizeof(struct dinode))
4552
4553 // Block containing inode i
4554 #define IBLOCK(i, sb) ((i) / IPB + sb.inodestart)
4555
4556 // Bitmap bits per block
4557 #define BPB (BSIZE * 8)
4558
4559 // Block of free map containing bit for block b
4560 #define BBLOCK(b, sb) ((b) / BPB + sb.bmapstart)
4561
4562 // Directory is a file containing a sequence of dirent structures.
4563 #define DIRSIZ 14
4564
4565 // The name field may have DIRSIZ characters and not end in a NUL
4566 // character.
4567 struct dirent {
4568     ushort inum;
4569     char name[DIRSIZ] __attribute__((nonstring));
4570 };
4571
4572 struct fs_dump {
4573     int size;           // Total filesystem blocks
4574     int nblocks;        // Number of data blocks
4575     int ninodes;        // Number of inodes
4576     int bmapstart;      // First block of bitmap
4577     uchar bitmap[1024]; // Adjust size based on your
4578                        // MAXOPBLOCKS / disk layout
4579 };
4580
4581
4582
4583
4584
4585
4586
4587
4588
4589
4590
4591
4592
4593
4594
4595
4596
4597
4598
4599

```

```

4600 struct file {
4601     enum { FD_NONE, FD_PIPE, FD_INODE, FD_DEVICE } type;
4602     int ref; // reference count
4603     char readable;
4604     char writable;
4605     struct pipe *pipe; // FD_PIPE
4606     struct inode *ip; // FD_INODE and FD_DEVICE
4607     uint off; // FD_INODE
4608     short major; // FD_DEVICE
4609     int append;
4610 };
4611
4612 #define major(dev) ((dev) >> 16 & 0xFFFF)
4613 #define minor(dev) ((dev) & 0xFFFF)
4614 #define mkdev(m, n) ((uint)((m) << 16 | (n)))
4615
4616 // in-memory copy of an inode
4617 struct inode {
4618     uint dev; // Device number
4619     uint inum; // Inode number
4620     int ref; // Reference count
4621     struct sleeplock lock; // protects everything below here
4622     int valid; // inode has been read from disk?
4623
4624     short type; // copy of disk inode
4625     short major;
4626     short minor;
4627     short nlink;
4628     uint size;
4629     uint addrs[NDIRECT + 1];
4630 };
4631
4632 // map major device number to device functions.
4633 struct devsw {
4634     int (*read)(int, uint64, int);
4635     int (*write)(int, uint64, int);
4636 };
4637
4638 extern struct devsw devsw[];
4639
4640 #define CONSOLE 1
4641 #define NULLDEV 2
4642
4643
4644
4645
4646
4647
4648
4649

```

```

4650 // Buffer cache.
4651 //
4652 // The buffer cache is a linked list of buf structures holding
4653 // cached copies of disk block contents. Caching disk blocks
4654 // in memory reduces the number of disk reads and also provides
4655 // a synchronization point for disk blocks used by multiple processes.
4656 //
4657 // Interface:
4658 // * To get a buffer for a particular disk block, call bread.
4659 // * After changing buffer data, call bwrite to write it to disk.
4660 // * When done with the buffer, call brelse.
4661 // * Do not use the buffer after calling brelse.
4662 // * Only one process at a time can use a buffer,
4663 //   so do not keep them longer than necessary.
4664
4665 #include "types.h"
4666 #include "param.h"
4667 #include "spinlock.h"
4668 #include "sleeplock.h"
4669 #include "riscv.h"
4670 #include "defs.h"
4671 #include "fs.h"
4672 #include "buf.h"
4673
4674 struct {
4675     struct spinlock lock;
4676     struct buf buf[NBUF];
4677
4678     // Linked list of all buffers, through prev/next.
4679     // Sorted by how recently the buffer was used.
4680     // head.next is most recent, head.prev is least.
4681     struct buf head;
4682 } bcache;
4683
4684 void
4685 binit(void)
4686 {
4687     struct buf *b;
4688
4689     initlock(&bcache.lock, "bcache");
4690
4691     // Create linked list of buffers
4692     bcache.head.prev = &bcache.head;
4693     bcache.head.next = &bcache.head;
4694     for (b = bcache.buf; b < bcache.buf + NBUF; b++) {
4695         b->next = bcache.head.next;
4696         b->prev = &bcache.head;
4697         initsleeplock(&b->lock, "buffer");
4698         bcache.head.next->prev = b;
4699         bcache.head.next = b;

```

```
4700 }
4701 }
4702
4703 // Look through buffer cache for block on device dev.
4704 // If not found, allocate a buffer.
4705 // In either case, return locked buffer.
4706 static struct buf *
4707 bget(uint dev, uint blockno)
4708 {
4709     struct buf *b;
4710
4711     acquire(&bcache.lock);
4712
4713     // Is the block already cached?
4714     for (b = bcache.head.next; b != &bcache.head; b = b->next) {
4715         if (b->dev == dev && b->blockno == blockno) {
4716             b->refcnt++;
4717             release(&bcache.lock);
4718             acquiresleep(&b->lock);
4719             return b;
4720         }
4721     }
4722
4723     // Not cached.
4724     // Recycle the least recently used (LRU) unused buffer.
4725     for (b = bcache.head.prev; b != &bcache.head; b = b->prev) {
4726         if (b->refcnt == 0) {
4727             b->dev = dev;
4728             b->blockno = blockno;
4729             b->valid = 0;
4730             b->refcnt = 1;
4731             release(&bcache.lock);
4732             acquiresleep(&b->lock);
4733             return b;
4734         }
4735     }
4736     panic("bget: no buffers");
4737 }
4738
4739
4740
4741
4742
4743
4744
4745
4746
4747
4748
4749
```

```

4750 // Return a locked buf with the contents of the indicated block.
4751 struct buf *
4752 bread(uint dev, uint blockno)
4753 {
4754     struct buf *b;
4755     b = bget(dev, blockno);
4756     if (!b->valid) {
4757         virtio_disk_rw(b, 0);
4758         b->valid = 1;
4759     }
4760     return b;
4761 }
4762 }
4763
4764 // Write b's contents to disk. Must be locked.
4765 void
4766 bwrite(struct buf *b)
4767 {
4768     if (!holdingsleep(&b->lock))
4769         panic("bwrite");
4770     virtio_disk_rw(b, 1);
4771 }
4772
4773 // Release a locked buffer.
4774 // Move to the head of the most-recently-used list.
4775 void
4776 brelse(struct buf *b)
4777 {
4778     if (!holdingsleep(&b->lock))
4779         panic("brelse");
4780
4781     releasesleep(&b->lock);
4782
4783     acquire(&bcache.lock);
4784     b->refcnt--;
4785     if (b->refcnt == 0) {
4786         // no one is waiting for it.
4787         b->next->prev = b->prev;
4788         b->prev->next = b->next;
4789         b->next = bcache.head.next;
4790         b->prev = &bcache.head;
4791         bcache.head.next->prev = b;
4792         bcache.head.next = b;
4793     }
4794
4795     release(&bcache.lock);
4796 }
4797
4798
4799

```

```
4800 void
4801 bpin(struct buf *b)
4802 {
4803     acquire(&bcache.lock);
4804     b->refcnt++;
4805     release(&bcache.lock);
4806 }
4807
4808 void
4809 bunpin(struct buf *b)
4810 {
4811     acquire(&bcache.lock);
4812     b->refcnt--;
4813     release(&bcache.lock);
4814 }
4815
4816
4817
4818
4819
4820
4821
4822
4823
4824
4825
4826
4827
4828
4829
4830
4831
4832
4833
4834
4835
4836
4837
4838
4839
4840
4841
4842
4843
4844
4845
4846
4847
4848
4849
```

```
4850 // Sleeping locks
4851
4852 #include "types.h"
4853 #include "riscv.h"
4854 #include "defs.h"
4855 #include "param.h"
4856 #include "memlayout.h"
4857 #include "spinlock.h"
4858 #include "proc.h"
4859 #include "sleeplock.h"
4860
4861 void
4862 initsleeplock(struct sleeplock *lk, char *name)
4863 {
4864     initlock(&lk->lk, "sleep lock");
4865     lk->name = name;
4866     lk->locked = 0;
4867     lk->pid = 0;
4868 }
4869
4870 void
4871 acquiresleep(struct sleeplock *lk)
4872 {
4873     acquire(&lk->lk);
4874     while (lk->locked) {
4875         sleep(lk, &lk->lk);
4876     }
4877     lk->locked = 1;
4878     lk->pid = myproc()->pid;
4879     release(&lk->lk);
4880 }
4881
4882 void
4883 releasesleep(struct sleeplock *lk)
4884 {
4885     acquire(&lk->lk);
4886     lk->locked = 0;
4887     lk->pid = 0;
4888     wakeup(lk);
4889     release(&lk->lk);
4890 }
4891
4892
4893
4894
4895
4896
4897
4898
4899
```

```
4900 int
4901 holdingsleep(struct sleeplock *lk)
4902 {
4903     int r;
4904
4905     acquire(&lk->lk);
4906     r = lk->locked && (lk->pid == myproc()->pid);
4907     release(&lk->lk);
4908     return r;
4909 }
4910
4911
4912
4913
4914
4915
4916
4917
4918
4919
4920
4921
4922
4923
4924
4925
4926
4927
4928
4929
4930
4931
4932
4933
4934
4935
4936
4937
4938
4939
4940
4941
4942
4943
4944
4945
4946
4947
4948
4949
```

```

4950 #include "types.h"
4951 #include "riscv.h"
4952 #include "defs.h"
4953 #include "param.h"
4954 #include "spinlock.h"
4955 #include "sleeplock.h"
4956 #include "fs.h"
4957 #include "buf.h"
4958
4959 // Simple logging that allows concurrent FS system calls.
4960 //
4961 // A log transaction contains the updates of multiple FS system
4962 // calls. The logging system only commits when there are
4963 // no FS system calls active. Thus there is never
4964 // any reasoning required about whether a commit might
4965 // write an uncommitted system call's updates to disk.
4966 //
4967 // A system call should call begin_op()/end_op() to mark
4968 // its start and end. Usually begin_op() just increments
4969 // the count of in-progress FS system calls and returns.
4970 // But if it thinks the log is close to running out, it
4971 // sleeps until the last outstanding end_op() commits.
4972 //
4973 // The log is a physical re-do log containing disk blocks.
4974 // The on-disk log format:
4975 //   header block, containing block #s for block A, B, C, ...
4976 //   block A
4977 //   block B
4978 //   block C
4979 //   ...
4980 // Log appends are synchronous.
4981
4982 // Contents of the header block, used for both the on-disk header block
4983 // and to keep track in memory of logged block# before commit.
4984 struct logheader {
4985     int n;
4986     int block[LOGBLOCKS];
4987 };
4988
4989 struct log {
4990     struct spinlock lock;
4991     int start;
4992     int outstanding; // how many FS sys calls are executing.
4993     int committing; // in commit(), please wait.
4994     int dev;
4995     int ncommit;
4996     struct logheader lh;
4997 };
4998
4999

```

```

5000 struct log log;
5001
5002 static void recover_from_log(void);
5003 static void commit();
5004
5005 void
5006 initlog(int dev, struct superblock *sb)
5007 {
5008     if (sizeof(struct logheader) >= BSIZE)
5009         panic("initlog: too big logheader");
5010
5011     initlock(&log.lock, "log");
5012     log.start = sb->logstart;
5013     log.dev = dev;
5014     recover_from_log();
5015 }
5016
5017 // Copy committed blocks from log to their home location
5018 static void
5019 install_trans(int recovering)
5020 {
5021     int tail;
5022
5023     for (tail = 0; tail < log.lh.n; tail++) {
5024         if (recovering) {
5025             printk("recovering tail %d dst %d\n", tail, log.lh.block[tail]);
5026         }
5027         struct buf *lbuf = bread(log.dev, log.start + tail + 1); // read log
5028                                                                    // block
5029         struct buf *dbuf = bread(log.dev, log.lh.block[tail]); // read dst
5030         memmove(dbuf->data, lbuf->data, BSIZE); // copy block to dst
5031         bwrite(dbuf); // write dst to disk
5032         if (recovering == 0)
5033             bunpin(dbuf);
5034         brelse(lbuf);
5035         brelse(dbuf);
5036     }
5037 }
5038
5039
5040
5041
5042
5043
5044
5045
5046
5047
5048
5049

```

```
5050 // Read the log header from disk into the in-memory log header
5051 static void
5052 read_head(void)
5053 {
5054     struct buf *buf = bread(log.dev, log.start);
5055     struct logheader *lh = (struct logheader *) (buf->data);
5056     int i;
5057     log.lh.n = lh->n;
5058     for (i = 0; i < log.lh.n; i++) {
5059         log.lh.block[i] = lh->block[i];
5060     }
5061     brelse(buf);
5062 }
5063
5064 // Write in-memory log header to disk.
5065 // This is the true point at which the
5066 // current transaction commits.
5067 static void
5068 write_head(void)
5069 {
5070     struct buf *buf = bread(log.dev, log.start);
5071     struct logheader *hb = (struct logheader *) (buf->data);
5072     int i;
5073     hb->n = log.lh.n;
5074     for (i = 0; i < log.lh.n; i++) {
5075         hb->block[i] = log.lh.block[i];
5076     }
5077     bwrite(buf);
5078     brelse(buf);
5079 }
5080
5081 static void
5082 recover_from_log(void)
5083 {
5084     read_head();
5085     install_trans(1); // if committed, copy from log to disk
5086     log.lh.n = 0;
5087     write_head(); // clear the log
5088 }
5089
5090
5091
5092
5093
5094
5095
5096
5097
5098
5099
```

```

5100 // called at the start of each FS system call.
5101 void
5102 begin_op(void)
5103 {
5104     acquire(&log.lock);
5105     while (1) {
5106         if (log.committing) {
5107             sleep(&log, &log.lock);
5108         } else if (log.lh.n + (log.outstanding + 1) * MAXOPBLOCKS >
5109             LOGBLOCKS) {
5110             // this op might exhaust log space; wait for commit.
5111             sleep(&log, &log.lock);
5112         } else {
5113             log.outstanding += 1;
5114             release(&log.lock);
5115             break;
5116         }
5117     }
5118 }
5119
5120 // called at the end of each FS system call.
5121 // commits if this was the last outstanding operation.
5122 void
5123 end_op(void)
5124 {
5125     int do_commit = 0;
5126
5127     acquire(&log.lock);
5128     log.outstanding -= 1;
5129     if (log.committing)
5130         panic("log.committing");
5131     if (log.outstanding == 0) {
5132         do_commit = 1;
5133         log.committing = 1;
5134     } else {
5135         // begin_op() may be waiting for log space,
5136         // and decrementing log.outstanding has decreased
5137         // the amount of reserved space.
5138         wakeup(&log);
5139     }
5140     release(&log.lock);
5141
5142     if (do_commit) {
5143         // call commit w/o holding locks, since not allowed
5144         // to sleep with locks.
5145         commit();
5146         acquire(&log.lock);
5147         log.committing = 0;
5148         log.ncommit += 1;
5149         wakeup(&log);

```

```

5150     release(&log.lock);
5151 }
5152 }
5153
5154 // Copy modified blocks from cache to log.
5155 static void
5156 write_log(void)
5157 {
5158     int tail;
5159
5160     for (tail = 0; tail < log.lh.n; tail++) {
5161         struct buf *to = bread(log.dev, log.start + tail + 1); // log block
5162         struct buf *from = bread(log.dev, log.lh.block[tail]); // cache block
5163         memmove(to->data, from->data, BSIZE);
5164         bwrite(to); // write the log
5165         brelse(from);
5166         brelse(to);
5167     }
5168 }
5169
5170 static void
5171 commit()
5172 {
5173     if (log.lh.n > 0) {
5174         write_log(); // Write modified blocks from cache to log
5175         write_head(); // Write header to disk -- the real commit
5176         install_trans(0); // Now install writes to home locations
5177         log.lh.n = 0;
5178         write_head(); // Erase the transaction from the log
5179     }
5180 }
5181
5182 // Caller has modified b->data and is done with the buffer.
5183 // Record the block number and pin in the cache by increasing refcnt.
5184 // commit()/write_log() will do the disk write.
5185 //
5186 // log_write() replaces bwrite(); a typical use is:
5187 //   bp = bread(...)
5188 //   modify bp->data[]
5189 //   log_write(bp)
5190 //   brelse(bp)
5191 void
5192 log_write(struct buf *b)
5193 {
5194     int i;
5195
5196     acquire(&log.lock);
5197     if (log.lh.n >= LOGBLOCKS)
5198         panic("too big a transaction");
5199     if (log.outstanding < 1)

```

```
5200     panic("log_write outside of trans");
5201
5202     for (i = 0; i < log.lh.n; i++) {
5203         if (log.lh.block[i] == b->blockno) // log absorption
5204             break;
5205     }
5206     log.lh.block[i] = b->blockno;
5207     if (i == log.lh.n) { // Add new block to log?
5208         bpin(b);
5209         log.lh.n++;
5210     }
5211     release(&log.lock);
5212 }
5213
5214 uint64
5215 sys_sync(void)
5216 {
5217     acquire(&log.lock);
5218     if (log.committing || log.outstanding > 0) {
5219         int n = log.ncommit + 1;
5220         while (log.ncommit < n) {
5221             sleep(&log, &log.lock);
5222         }
5223     }
5224     release(&log.lock);
5225     return 0;
5226 }
5227
5228
5229
5230
5231
5232
5233
5234
5235
5236
5237
5238
5239
5240
5241
5242
5243
5244
5245
5246
5247
5248
5249
```

```
5250 // File system implementation. Five layers:
5251 //   + Blocks: allocator for raw disk blocks.
5252 //   + Log: crash recovery for multi-step updates.
5253 //   + Files: inode allocator, reading, writing, metadata.
5254 //   + Directories: inode with special contents (list of other inodes!)
5255 //   + Names: paths like /usr/rtm/xv6/fs.c for convenient naming.
5256 //
5257 // This file contains the low-level file system manipulation
5258 // routines. The (higher-level) system call implementations
5259 // are in sysfile.c.
5260
5261 #include "types.h"
5262 #include "riscv.h"
5263 #include "defs.h"
5264 #include "param.h"
5265 #include "stat.h"
5266 #include "spinlock.h"
5267 #include "proc.h"
5268 #include "sleeplock.h"
5269 #include "fs.h"
5270 #include "buf.h"
5271 #include "file.h"
5272
5273 #define min(a, b) ((a) < (b) ? (a) : (b))
5274 // there should be one superblock per disk device, but we run with
5275 // only one device
5276 struct superblock sb;
5277
5278 // Read the super block.
5279 static void
5280 readsb(int dev, struct superblock *sb)
5281 {
5282     struct buf *bp;
5283
5284     bp = bread(dev, 1);
5285     memmove(sb, bp->data, sizeof(*sb));
5286     brelse(bp);
5287 }
5288
5289 // Init fs
5290 void
5291 fsinit(int dev)
5292 {
5293     readsb(dev, &sb);
5294     if (sb.magic != FSMAGIC)
5295         panic("invalid file system");
5296     initlog(dev, &sb);
5297     ireclaim(dev);
5298 }
5299
```

```

5300 // Zero a block.
5301 static void
5302 bzero(int dev, int bno)
5303 {
5304     struct buf *bp;
5305
5306     bp = bread(dev, bno);
5307     memset(bp->data, 0, BSIZE);
5308     log_write(bp);
5309     brelse(bp);
5310 }
5311
5312 // Blocks.
5313
5314 // Allocate a zeroed disk block.
5315 // returns 0 if out of disk space.
5316 static uint
5317 balloc(uint dev)
5318 {
5319     int b, bi, m;
5320     struct buf *bp;
5321
5322     bp = 0;
5323     for (b = 0; b < sb.size; b += BPB) {
5324         bp = bread(dev, BBLOCK(b, sb));
5325         for (bi = 0; bi < BPB && b + bi < sb.size; bi++) {
5326             m = 1 << (bi % 8);
5327             if ((bp->data[bi / 8] & m) == 0) { // Is block free?
5328                 bp->data[bi / 8] |= m;        // Mark block in use.
5329                 log_write(bp);
5330                 brelse(bp);
5331                 bzero(dev, b + bi);
5332                 return b + bi;
5333             }
5334         }
5335         brelse(bp);
5336     }
5337     printk("balloc: out of blocks\n");
5338     return 0;
5339 }
5340
5341
5342
5343
5344
5345
5346
5347
5348
5349

```

```

5350 // Free a disk block.
5351 static void
5352 bfree(int dev, uint b)
5353 {
5354     struct buf *bp;
5355     int bi, m;
5356
5357     bp = bread(dev, BBLOCK(b, sb));
5358     bi = b % BPB;
5359     m = 1 << (bi % 8);
5360     if ((bp->data[bi / 8] & m) == 0)
5361         panic("freeing free block");
5362     bp->data[bi / 8] &= ~m;
5363     log_write(bp);
5364     brelse(bp);
5365 }
5366
5367 // Inodes.
5368 //
5369 // An inode describes a single unnamed file.
5370 // The inode disk structure holds metadata: the file's type,
5371 // its size, the number of links referring to it, and the
5372 // list of blocks holding the file's content.
5373 //
5374 // The inodes are laid out sequentially on disk at block
5375 // sb.inodestart. Each inode has a number, indicating its
5376 // position on the disk.
5377 //
5378 // The kernel keeps a table of in-use inodes in memory
5379 // to provide a place for synchronizing access
5380 // to inodes used by multiple processes. The in-memory
5381 // inodes include book-keeping information that is
5382 // not stored on disk: ip->ref and ip->valid.
5383 //
5384 // An inode and its in-memory representation go through a
5385 // sequence of states before they can be used by the
5386 // rest of the file system code.
5387 //
5388 // * Allocation: an inode is allocated if its type (on disk)
5389 //   is non-zero. ialloc() allocates, and iput() frees if
5390 //   the reference and link counts have fallen to zero.
5391 //
5392 // * Referencing in table: an entry in the inode table
5393 //   is free if ip->ref is zero. Otherwise ip->ref tracks
5394 //   the number of in-memory pointers to the entry (open
5395 //   files and current directories). iget() finds or
5396 //   creates a table entry and increments its ref; iput()
5397 //   decrements ref.
5398 //
5399 // * Valid: the information (type, size, &c) in an inode

```

```

5400 //  table entry is only correct when ip->valid is 1.
5401 //  ilock() reads the inode from
5402 //  the disk and sets ip->valid, while iput() clears
5403 //  ip->valid if ip->ref has fallen to zero.
5404 //
5405 //  * Locked: file system code may only examine and modify
5406 //  the information in an inode and its content if it
5407 //  has first locked the inode.
5408 //
5409 //  Thus a typical sequence is:
5410 //  ip = iget(dev, inum)
5411 //  ilock(ip)
5412 //  ... examine and modify ip->xxx ...
5413 //  iunlock(ip)
5414 //  iput(ip)
5415 //
5416 //  ilock() is separate from iget() so that system calls can
5417 //  get a long-term reference to an inode (as for an open file)
5418 //  and only lock it for short periods (e.g., in read()).
5419 //  The separation also helps avoid deadlock and races during
5420 //  pathname lookup. iget() increments ip->ref so that the inode
5421 //  stays in the table and pointers to it remain valid.
5422 //
5423 //  Many internal file system functions expect the caller to
5424 //  have locked the inodes involved; this lets callers create
5425 //  multi-step atomic operations.
5426 //
5427 //  The itable.lock spin-lock protects the allocation of itable
5428 //  entries. Since ip->ref indicates whether an entry is free,
5429 //  and ip->dev and ip->inum indicate which i-node an entry
5430 //  holds, one must hold itable.lock while using any of those fields.
5431 //
5432 //  An ip->lock sleep-lock protects all ip-> fields other than ref,
5433 //  dev, and inum. One must hold ip->lock in order to
5434 //  read or write that inode's ip->valid, ip->size, ip->type, &c.
5435
5436 struct {
5437     struct spinlock lock;
5438     struct inode inode[NINODE];
5439 } itable;
5440
5441 void
5442 iinit()
5443 {
5444     int i = 0;
5445
5446     initlock(&itable.lock, "itable");
5447     for (i = 0; i < NINODE; i++) {
5448         initsleeplock(&itable.inode[i].lock, "inode");
5449     }

```

```

5450 }
5451
5452 static struct inode *iget(uint dev, uint inum);
5453
5454 // Allocate an inode on device dev.
5455 // Mark it as allocated by giving it type type.
5456 // Returns an unlocked but allocated and referenced inode,
5457 // or NULL if there is no free inode.
5458 struct inode *
5459 ialloc(uint dev, short type)
5460 {
5461     int inum;
5462     struct buf *bp;
5463     struct dinode *dip;
5464
5465     for (inum = 1; inum < sb.ninodes; inum++) {
5466         bp = bread(dev, IBLOCK(inum, sb));
5467         dip = (struct dinode *)bp->data + inum % IPB;
5468         if (dip->type == 0) { // a free inode
5469             memset(dip, 0, sizeof(*dip));
5470             dip->type = type;
5471             log_write(bp); // mark it allocated on the disk
5472             brelse(bp);
5473             return iget(dev, inum);
5474         }
5475         brelse(bp);
5476     }
5477     printk("ialloc: no inodes\n");
5478     return 0;
5479 }
5480
5481 // Copy a modified in-memory inode to disk.
5482 // Must be called after every change to an ip->xxx field
5483 // that lives on disk.
5484 // Caller must hold ip->lock.
5485 void
5486 iupdate(struct inode *ip)
5487 {
5488     struct buf *bp;
5489     struct dinode *dip;
5490
5491     bp = bread(ip->dev, IBLOCK(ip->inum, sb));
5492     dip = (struct dinode *)bp->data + ip->inum % IPB;
5493     dip->type = ip->type;
5494     dip->major = ip->major;
5495     dip->minor = ip->minor;
5496     dip->nlink = ip->nlink;
5497     dip->size = ip->size;
5498     memmove(dip->addrs, ip->addrs, sizeof(ip->addrs));
5499     log_write(bp);

```

```

5500     brelse(bp);
5501 }
5502
5503 // Find the inode with number inum on device dev
5504 // and return the in-memory copy. Does not lock
5505 // the inode and does not read it from disk.
5506 static struct inode *
5507 iget(uint dev, uint inum)
5508 {
5509     struct inode *ip, *empty;
5510
5511     acquire(&itable.lock);
5512
5513     // Is the inode already in the table?
5514     empty = 0;
5515     for (ip = &itable.inode[0]; ip < &itable.inode[NINODE]; ip++) {
5516         if (ip->ref > 0 && ip->dev == dev && ip->inum == inum) {
5517             ip->ref++;
5518             release(&itable.lock);
5519             return ip;
5520         }
5521         if (empty == 0 && ip->ref == 0) // Remember empty slot.
5522             empty = ip;
5523     }
5524
5525     // Recycle an inode entry.
5526     if (empty == 0)
5527         panic("iget: no inodes");
5528
5529     ip = empty;
5530     ip->dev = dev;
5531     ip->inum = inum;
5532     ip->ref = 1;
5533     ip->valid = 0;
5534     release(&itable.lock);
5535
5536     return ip;
5537 }
5538
5539 // Increment reference count for ip.
5540 // Returns ip to enable ip = idup(ip1) idiom.
5541 struct inode *
5542 idup(struct inode *ip)
5543 {
5544     acquire(&itable.lock);
5545     ip->ref++;
5546     release(&itable.lock);
5547     return ip;
5548 }
5549

```

```

5550 // Lock the given inode.
5551 // Reads the inode from disk if necessary.
5552 void
5553 ilock(struct inode *ip)
5554 {
5555     struct buf *bp;
5556     struct dinode *dip;
5557
5558     if (ip == 0 || ip->ref < 1)
5559         panic("ilock");
5560
5561     acquiresleep(&ip->lock);
5562
5563     if (ip->valid == 0) {
5564         bp = bread(ip->dev, IBLOCK(ip->inum, sb));
5565         dip = (struct dinode *)bp->data + ip->inum % IPB;
5566         ip->type = dip->type;
5567         ip->major = dip->major;
5568         ip->minor = dip->minor;
5569         ip->nlink = dip->nlink;
5570         ip->size = dip->size;
5571         memmove(ip->addrs, dip->addrs, sizeof(ip->addrs));
5572         brelse(bp);
5573         ip->valid = 1;
5574         if (ip->type == 0)
5575             panic("ilock: no type");
5576     }
5577 }
5578
5579 // Unlock the given inode.
5580 void
5581 iunlock(struct inode *ip)
5582 {
5583     if (ip == 0 || !holdingsleep(&ip->lock) || ip->ref < 1)
5584         panic("iunlock");
5585
5586     releasesleep(&ip->lock);
5587 }
5588
5589
5590
5591
5592
5593
5594
5595
5596
5597
5598
5599

```

```

5600 // Drop a reference to an in-memory inode.
5601 // If that was the last reference, the inode table entry can
5602 // be recycled.
5603 // If that was the last reference and the inode has no links
5604 // to it, free the inode (and its content) on disk.
5605 // All calls to iput() must be inside a transaction in
5606 // case it has to free the inode.
5607 void
5608 iput(struct inode *ip)
5609 {
5610     acquire(&itable.lock);
5611
5612     if (ip->ref == 1 && ip->valid && ip->nlink == 0) {
5613         // inode has no links and no other references: truncate and free.
5614
5615         // ip->ref == 1 means no other process can have ip locked,
5616         // so this acquiresleep() won't block (or deadlock).
5617         acquiresleep(&ip->lock);
5618
5619         release(&itable.lock);
5620
5621         itrunc(ip);
5622         ip->type = 0;
5623         iupdate(ip);
5624         ip->valid = 0;
5625
5626         releasesleep(&ip->lock);
5627
5628         acquire(&itable.lock);
5629     }
5630
5631     ip->ref--;
5632     release(&itable.lock);
5633 }
5634
5635 // Common idiom: unlock, then put.
5636 void
5637 iunlockput(struct inode *ip)
5638 {
5639     iunlock(ip);
5640     iput(ip);
5641 }
5642
5643
5644
5645
5646
5647
5648
5649

```

```

5650 void
5651 ireclaim(int dev)
5652 {
5653     for (int inum = 1; inum < sb.ninodes; inum++) {
5654         struct inode *ip = 0;
5655         struct buf *bp = bread(dev, IBLOCK(inum, sb));
5656         struct dinode *dip = (struct dinode *)bp->data + inum % IPB;
5657         if (dip->type != 0 && dip->nlink == 0) { // is an orphaned inode
5658             printk("ireclaim: orphaned inode %d\n", inum);
5659             ip = iget(dev, inum);
5660         }
5661         brelse(bp);
5662         if (ip) {
5663             begin_op();
5664             ilock(ip);
5665             iunlock(ip);
5666             iput(ip);
5667             end_op();
5668         }
5669     }
5670 }
5671
5672 // Inode content
5673 //
5674 // The content (data) associated with each inode is stored
5675 // in blocks on the disk. The first NDIRECT block numbers
5676 // are listed in ip->addrs[]. The next NINDIRECT blocks are
5677 // listed in block ip->addrs[NDIRECT].
5678
5679 // Return the disk block address of the nth block in inode ip.
5680 // If there is no such block, bmap allocates one.
5681 // returns 0 if out of disk space.
5682 static uint
5683 bmap(struct inode *ip, uint bn)
5684 {
5685     uint addr, *a;
5686     struct buf *bp;
5687
5688     if (bn < NDIRECT) {
5689         if ((addr = ip->addrs[bn]) == 0) {
5690             addr = balloc(ip->dev);
5691             if (addr == 0)
5692                 return 0;
5693             ip->addrs[bn] = addr;
5694         }
5695         return addr;
5696     }
5697     bn -= NDIRECT;
5698
5699

```

```

5700  if (bn < NINDIRECT) {
5701      // Load indirect block, allocating if necessary.
5702      if ((addr = ip->addrs[NDIRECT]) == 0) {
5703          addr = balloc(ip->dev);
5704          if (addr == 0)
5705              return 0;
5706          ip->addrs[NDIRECT] = addr;
5707      }
5708      bp = bread(ip->dev, addr);
5709      a = (uint *)bp->data;
5710      if ((addr = a[bn]) == 0) {
5711          addr = balloc(ip->dev);
5712          if (addr) {
5713              a[bn] = addr;
5714              log_write(bp);
5715          }
5716      }
5717      brelse(bp);
5718      return addr;
5719  }
5720
5721  panic("bmap: out of range");
5722 }
5723
5724 // Truncate inode (discard contents).
5725 // Caller must hold ip->lock.
5726 void
5727 itrunc(struct inode *ip)
5728 {
5729     int i, j;
5730     struct buf *bp;
5731     uint *a;
5732
5733     for (i = 0; i < NDIRECT; i++) {
5734         if (ip->addrs[i]) {
5735             bfree(ip->dev, ip->addrs[i]);
5736             ip->addrs[i] = 0;
5737         }
5738     }
5739
5740     if (ip->addrs[NDIRECT]) {
5741         bp = bread(ip->dev, ip->addrs[NDIRECT]);
5742         a = (uint *)bp->data;
5743         for (j = 0; j < NINDIRECT; j++) {
5744             if (a[j])
5745                 bfree(ip->dev, a[j]);
5746         }
5747         brelse(bp);
5748         bfree(ip->dev, ip->addrs[NDIRECT]);
5749         ip->addrs[NDIRECT] = 0;

```

```

5750 }
5751
5752 ip->size = 0;
5753 iupdate(ip);
5754 }
5755
5756 // Copy stat information from inode.
5757 // Caller must hold ip->lock.
5758 void
5759 stati(struct inode *ip, struct stat *st)
5760 {
5761     st->dev = ip->dev;
5762     st->ino = ip->inum;
5763     st->type = ip->type;
5764     st->nlink = ip->nlink;
5765     st->size = ip->size;
5766 }
5767
5768 // Read data from inode.
5769 // Caller must hold ip->lock.
5770 // If user_dst==1, then dst is a user virtual address;
5771 // otherwise, dst is a kernel address.
5772 int
5773 readi(struct inode *ip, int user_dst, uint64 dst, uint off, uint n)
5774 {
5775     uint tot, m;
5776     struct buf *bp;
5777
5778     if (off > ip->size || off + n < off)
5779         return 0;
5780     if (off + n > ip->size)
5781         n = ip->size - off;
5782
5783     for (tot = 0; tot < n; tot += m, off += m, dst += m) {
5784         uint addr = bmap(ip, off / BSIZE);
5785         if (addr == 0)
5786             break;
5787         bp = bread(ip->dev, addr);
5788         m = min(n - tot, BSIZE - off % BSIZE);
5789         if (either_copyout(user_dst, dst, bp->data + (off % BSIZE),
5790             m) == -1) {
5791             brelse(bp);
5792             tot = -1;
5793             break;
5794         }
5795         brelse(bp);
5796     }
5797     return tot;
5798 }
5799

```

```

5800 // Write data to inode.
5801 // Caller must hold ip->lock.
5802 // If user_src==1, then src is a user virtual address;
5803 // otherwise, src is a kernel address.
5804 // Returns the number of bytes successfully written.
5805 // If the return value is less than the requested n,
5806 // there was an error of some kind.
5807 int
5808 writei(struct inode *ip, int user_src, uint64 src, uint off, uint n)
5809 {
5810     uint tot, m;
5811     struct buf *bp;
5812
5813     if (off > ip->size || off + n < off)
5814         return -1;
5815     if (off + n > MAXFILE * BSIZE)
5816         return -1;
5817
5818     for (tot = 0; tot < n; tot += m, off += m, src += m) {
5819         uint addr = bmap(ip, off / BSIZE);
5820         if (addr == 0)
5821             break;
5822         bp = bread(ip->dev, addr);
5823         m = min(n - tot, BSIZE - off % BSIZE);
5824         if (either_copyin(bp->data + (off % BSIZE), user_src, src, m) == -1) {
5825             brelse(bp);
5826             break;
5827         }
5828         log_write(bp);
5829         brelse(bp);
5830     }
5831
5832     if (off > ip->size)
5833         ip->size = off;
5834
5835     // write the i-node back to disk even if the size didn't change
5836     // because the loop above might have called bmap() and added a new
5837     // block to ip->addrs[].
5838     iupdate(ip);
5839
5840     return tot;
5841 }
5842
5843 // Directories
5844
5845 int
5846 namecmp(const char *s, const char *t)
5847 {
5848     return strncmp(s, t, DIRSIZ);
5849 }

```

```

5850 // Look for a directory entry in a directory.
5851 // If found, set *poff to byte offset of entry.
5852 struct inode *
5853 dirlookup(struct inode *dp, char *name, uint *poff)
5854 {
5855     uint off, inum;
5856     struct dirent de;
5857
5858     if (dp->type != T_DIR)
5859         panic("dirlookup not DIR");
5860
5861     for (off = 0; off < dp->size; off += sizeof(de)) {
5862         if (readi(dp, 0, (uint64*)&de, off, sizeof(de)) != sizeof(de))
5863             panic("dirlookup read");
5864         if (de.inum == 0)
5865             continue;
5866         if (namecmp(name, de.name) == 0) {
5867             // entry matches path element
5868             if (poff)
5869                 *poff = off;
5870             inum = de.inum;
5871             return iget(dp->dev, inum);
5872         }
5873     }
5874
5875     return 0;
5876 }
5877
5878 // Write a new directory entry (name, inum) into the directory dp.
5879 // Returns 0 on success, -1 on failure (e.g. out of disk blocks).
5880 int
5881 dirlink(struct inode *dp, char *name, uint inum)
5882 {
5883     int off;
5884     struct dirent de;
5885     struct inode *ip;
5886
5887     // Check that name is not present.
5888     if ((ip = dirlookup(dp, name, 0)) != 0) {
5889         iput(ip);
5890         return -1;
5891     }
5892
5893     // Look for an empty dirent.
5894     for (off = 0; off < dp->size; off += sizeof(de)) {
5895         if (readi(dp, 0, (uint64*)&de, off, sizeof(de)) != sizeof(de))
5896             panic("dirlink read");
5897         if (de.inum == 0)
5898             break;
5899     }

```

```

5900     strncpy(de.name, name, DIRSIZ);
5901     de.inum = inum;
5902     if (writei(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
5903         return -1;
5904
5905     return 0;
5906 }
5907
5908 // Paths
5909
5910 // Copy the next path element from path into name.
5911 // Return a pointer to the element following the copied one.
5912 // The returned path has no leading slashes,
5913 // so the caller can check *path=='\0' to see if the name is the last one.
5914 // If no name to remove, return 0.
5915 //
5916 // Examples:
5917 //     skipelem("a/bb/c", name) = "bb/c", setting name = "a"
5918 //     skipelem("///a//bb", name) = "bb", setting name = "a"
5919 //     skipelem("a", name) = "", setting name = "a"
5920 //     skipelem("", name) = skipelem("///", name) = 0
5921 //
5922 static char *
5923 skipelem(char *path, char *name)
5924 {
5925     char *s;
5926     int len;
5927
5928     while (*path == '/')
5929         path++;
5930     if (*path == 0)
5931         return 0;
5932     s = path;
5933     while (*path != '/' && *path != 0)
5934         path++;
5935     len = path - s;
5936     if (len >= DIRSIZ)
5937         memmove(name, s, DIRSIZ);
5938     else {
5939         memmove(name, s, len);
5940         name[len] = 0;
5941     }
5942     while (*path == '/')
5943         path++;
5944     return path;
5945 }
5946
5947
5948
5949

```

```

5950 // Look up and return the inode for a path name.
5951 // If parent != 0, return the inode for the parent and copy the final
5952 // path element into name, which must have room for DIRSIZ bytes.
5953 // Must be called inside a transaction since it calls iput().
5954 static struct inode *
5955 namex(char *path, int nameparent, char *name)
5956 {
5957     struct inode *ip, *next;
5958
5959     if (*path == '/')
5960         ip = iget(ROOTDEV, ROOTINO);
5961     else
5962         ip = idup(myproc()->cwd);
5963
5964     while ((path = skipelem(path, name)) != 0) {
5965         ilock(ip);
5966         if (ip->type != T_DIR) {
5967             iunlockput(ip);
5968             return 0;
5969         }
5970         if (nameparent && *path == '\0') {
5971             // Stop one level early.
5972             iunlock(ip);
5973             return ip;
5974         }
5975         if ((next = dirlookup(ip, name, 0)) == 0) {
5976             iunlockput(ip);
5977             return 0;
5978         }
5979         iunlockput(ip);
5980         ip = next;
5981     }
5982     if (nameparent) {
5983         iput(ip);
5984         return 0;
5985     }
5986     return ip;
5987 }
5988
5989 struct inode *
5990 namei(char *path)
5991 {
5992     char name[DIRSIZ];
5993     return namex(path, 0, name);
5994 }
5995
5996
5997
5998
5999

```

```

6000 struct inode *
6001 nameiparent(char *path, char *name)
6002 {
6003     return namex(path, 1, name);
6004 }
6005
6006
6007 struct dfstat {
6008     uint total;
6009     uint used;
6010     uint free;
6011 };
6012
6013 uint64
6014 sys_df(void)
6015 {
6016     uint64 uaddr;
6017
6018     argaddr(0, &uaddr);
6019     if( uaddr < 0 )
6020         return -1;
6021
6022     struct superblock sb;
6023     readsb(ROOTDEV, &sb);
6024
6025     uint free = 0;
6026     uint used = 0;
6027
6028     int bits = 0;
6029
6030     // for(uint b = 0; b < sb.bmapstart + sb.nbitmap; b++) {
6031         // nothing
6032     // }
6033
6034     // nbitmap number of bitmap blocks
6035     // sb.size/8
6036
6037     // for(uint b = 0; b < sb.nbitmap; b++) {
6038     for(uint b = 0; b < sb.size/8; b++) {
6039
6040         struct buf *bp = bread(ROOTDEV, sb.bmapstart + b);
6041
6042         uchar *bitmap = (uchar *)bp->data;
6043
6044         for(int i = 0; i < BSIZE; i++) {
6045             uchar byte = bitmap[i];
6046
6047             for(int j = 0; j < 8; j++) {
6048
6049

```

```
6050         if(bits >= sb.size)
6051             break;
6052
6053         if(byte & (1 << j))
6054             used++;
6055         else
6056             free++;
6057
6058         bits++;
6059     }
6060 }
6061
6062     brelease(bp);
6063 }
6064
6065     struct dfstat st;
6066
6067     st.total = sb.size;
6068     st.used  = used;
6069     st.free  = free;
6070
6071     if(copyout(myproc()->pagetable,
6072               uaddr,
6073               (char *)&st,
6074               sizeof(st)) < 0)
6075         return -1;
6076
6077     return 0;
6078 }
6079
6080
6081
6082
6083
6084
6085
6086
6087
6088
6089
6090
6091
6092
6093
6094
6095
6096
6097
6098
6099
```

```
6100 //
6101 // Support functions for system calls that involve file descriptors.
6102 //
6103
6104 #include "types.h"
6105 #include "riscv.h"
6106 #include "defs.h"
6107 #include "param.h"
6108 #include "fs.h"
6109 #include "spinlock.h"
6110 #include "sleeplock.h"
6111 #include "file.h"
6112 #include "stat.h"
6113 #include "proc.h"
6114
6115 struct devsw devsw[NDEV];
6116 struct {
6117     struct spinlock lock;
6118     struct file file[NFILE];
6119 } ftable;
6120
6121 void
6122 fileinit(void)
6123 {
6124     initlock(&ftable.lock, "ftable");
6125 }
6126
6127 // Allocate a file structure.
6128 struct file *
6129 filealloc(void)
6130 {
6131     struct file *f;
6132
6133     acquire(&ftable.lock);
6134     for (f = ftable.file; f < ftable.file + NFILE; f++) {
6135         if (f->ref == 0) {
6136             f->ref = 1;
6137             release(&ftable.lock);
6138             return f;
6139         }
6140     }
6141     release(&ftable.lock);
6142     return 0;
6143 }
6144
6145
6146
6147
6148
6149
```

```
6150 // Increment ref count for file f.
6151 struct file *
6152 filedup(struct file *f)
6153 {
6154     acquire(&ftable.lock);
6155     if (f->ref < 1)
6156         panic("filedup");
6157     f->ref++;
6158     release(&ftable.lock);
6159     return f;
6160 }
6161
6162 // Close file f. (Decrement ref count, close when reaches 0.)
6163 void
6164 fileclose(struct file *f)
6165 {
6166     struct file ff;
6167
6168     acquire(&ftable.lock);
6169     if (f->ref < 1)
6170         panic("fileclose");
6171     if (--f->ref > 0) {
6172         release(&ftable.lock);
6173         return;
6174     }
6175     ff = *f;
6176     f->ref = 0;
6177     f->type = FD_NONE;
6178     release(&ftable.lock);
6179
6180     if (ff.type == FD_PIPE) {
6181         pipeclose(ff.pipe, ff.writable);
6182     } else if (ff.type == FD_INODE || ff.type == FD_DEVICE) {
6183         begin_op();
6184         iput(ff.ip);
6185         end_op();
6186     }
6187 }
6188
6189
6190
6191
6192
6193
6194
6195
6196
6197
6198
6199
```

```

6200 // Get metadata about file f.
6201 // addr is a user virtual address, pointing to a struct stat.
6202 int
6203 filestat(struct file *f, uint64 addr)
6204 {
6205     struct proc *p = myproc();
6206     struct stat st;
6207
6208     if (f->type == FD_INODE || f->type == FD_DEVICE) {
6209         ilock(f->ip);
6210         stati(f->ip, &st);
6211         iunlock(f->ip);
6212         if (copyout(p->pagetable, addr, (char *)&st, sizeof(st)) < 0)
6213             return -1;
6214         return 0;
6215     }
6216     return -1;
6217 }
6218
6219 // Read from file f.
6220 // addr is a user virtual address.
6221 int
6222 fileread(struct file *f, uint64 addr, int n)
6223 {
6224     int r = 0;
6225
6226     if (f->readable == 0)
6227         return -1;
6228
6229     if (f->type == FD_PIPE) {
6230         r = piperead(f->pipe, addr, n);
6231     } else if (f->type == FD_DEVICE) {
6232         if (f->major < 0 || f->major >= NDEV || !devsw[f->major].read)
6233             return -1;
6234         r = devsw[f->major].read(1, addr, n);
6235     } else if (f->type == FD_INODE) {
6236         ilock(f->ip);
6237         if ((r = readi(f->ip, 1, addr, f->off, n)) > 0)
6238             f->off += r;
6239         iunlock(f->ip);
6240     } else {
6241         panic("fileread");
6242     }
6243
6244     return r;
6245 }
6246
6247
6248
6249

```

```

6250 // Write to file f.
6251 // addr is a user virtual address.
6252 int
6253 filewrite(struct file *f, uint64 addr, int n)
6254 {
6255     int r, ret = 0;
6256
6257     if (f->writable == 0)
6258         return -1;
6259
6260     if (f->type == FD_PIPE) {
6261         ret = pipewrite(f->pipe, addr, n);
6262     } else if (f->type == FD_DEVICE) {
6263         if (f->major < 0 || f->major >= NDEV || !devsw[f->major].write)
6264             return -1;
6265         ret = devsw[f->major].write(1, addr, n);
6266     } else if (f->type == FD_INODE) {
6267
6268         ilock(f->ip);
6269         // If append mode is set, change the offset to the end of the
6270         // file before writing
6271         if(f->append) {
6272             f->off = f->ip->size;
6273         }
6274         iunlock(f->ip);
6275
6276         // write a few blocks at a time to avoid exceeding
6277         // the maximum log transaction size, including
6278         // i-node, indirect block, allocation blocks,
6279         // and 2 blocks of slop for non-aligned writes.
6280         int max = ((MAXOPBLOCKS - 1 - 1 - 2) / 2) * BSIZE;
6281         int i = 0;
6282         while (i < n) {
6283             int n1 = n - i;
6284             if (n1 > max)
6285                 n1 = max;
6286
6287             begin_op();
6288             ilock(f->ip);
6289             if ((r = writei(f->ip, 1, addr + i, f->off, n1)) > 0)
6290                 f->off += r;
6291             iunlock(f->ip);
6292             end_op();
6293
6294             if (r != n1) {
6295                 // error from writei
6296                 break;
6297             }
6298             i += r;
6299         }

```

```
6300     ret = (i == n ? n : -1);
6301 } else {
6302     panic("filewrite");
6303 }
6304
6305 return ret;
6306 }
6307
6308
6309
6310
6311
6312
6313
6314
6315
6316
6317
6318
6319
6320
6321
6322
6323
6324
6325
6326
6327
6328
6329
6330
6331
6332
6333
6334
6335
6336
6337
6338
6339
6340
6341
6342
6343
6344
6345
6346
6347
6348
6349
```

```
6350 //
6351 // File-system system calls.
6352 // Mostly argument checking, since we don't trust
6353 // user code, and calls into file.c and fs.c.
6354 //
6355
6356 #include "types.h"
6357 #include "riscv.h"
6358 #include "defs.h"
6359 #include "param.h"
6360 #include "stat.h"
6361 #include "spinlock.h"
6362 #include "proc.h"
6363 #include "fs.h"
6364 #include "sleeplock.h"
6365 #include "file.h"
6366 #include "fcntl.h"
6367 #include "buf.h"
6368 #include "rtc.h"
6369
6370
6371 // Fetch the nth word-sized system call argument as a file descriptor
6372 // and return both the descriptor and the corresponding struct file.
6373 static int
6374 argfd(int n, int *pfd, struct file **pf)
6375 {
6376     int fd;
6377     struct file *f;
6378
6379     argint(n, &fd);
6380     if (fd < 0 || fd >= NOFILE || (f = myproc()->ofile[fd]) == 0)
6381         return -1;
6382     if (pfd)
6383         *pfd = fd;
6384     if (pf)
6385         *pf = f;
6386     return 0;
6387 }
6388
6389
6390
6391
6392
6393
6394
6395
6396
6397
6398
6399
```

```
6400 // Allocate a file descriptor for the given file.
6401 // Takes over file reference from caller on success.
6402 static int
6403 fdalloc(struct file *f)
6404 {
6405     int fd;
6406     struct proc *p = myproc();
6407
6408     for (fd = 0; fd < NOFILE; fd++) {
6409         if (p->ofile[fd] == 0) {
6410             p->ofile[fd] = f;
6411             return fd;
6412         }
6413     }
6414     return -1;
6415 }
6416
6417 uint64
6418 sys_dup(void)
6419 {
6420     struct file *f;
6421     int fd;
6422
6423     if (argfd(0, 0, &f) < 0)
6424         return -1;
6425     if ((fd = fdalloc(f)) < 0)
6426         return -1;
6427     filedup(f);
6428     return fd;
6429 }
6430
6431 uint64
6432 sys_read(void)
6433 {
6434     struct file *f;
6435     int n;
6436     uint64 p;
6437
6438     argaddr(1, &p);
6439     argint(2, &n);
6440     if (argfd(0, 0, &f) < 0)
6441         return -1;
6442     return fileread(f, p, n);
6443 }
6444
6445
6446
6447
6448
6449
```

```
6450 uint64
6451 sys_write(void)
6452 {
6453     struct file *f;
6454     int n;
6455     uint64 p;
6456
6457     argaddr(1, &p);
6458     argint(2, &n);
6459     if (argfd(0, 0, &f) < 0)
6460         return -1;
6461
6462     return filewrite(f, p, n);
6463 }
6464
6465 uint64
6466 sys_close(void)
6467 {
6468     int fd;
6469     struct file *f;
6470
6471     if (argfd(0, &fd, &f) < 0)
6472         return -1;
6473     myproc()->ofile[fd] = 0;
6474     fileclose(f);
6475     return 0;
6476 }
6477
6478 uint64
6479 sys_fstat(void)
6480 {
6481     struct file *f;
6482     uint64 st; // user pointer to struct stat
6483
6484     argaddr(1, &st);
6485     if (argfd(0, 0, &f) < 0)
6486         return -1;
6487     return filestat(f, st);
6488 }
6489
6490
6491
6492
6493
6494
6495
6496
6497
6498
6499
```

```

6500 // Create the path new as a link to the same inode as old.
6501 uint64
6502 sys_link(void)
6503 {
6504     char name[DIRSIZ], new[MAXPATH], old[MAXPATH];
6505     struct inode *dp, *ip;
6506
6507     if (argstr(0, old, MAXPATH) < 0 || argstr(1, new, MAXPATH) < 0)
6508         return -1;
6509
6510     begin_op();
6511     if ((ip = namei(old)) == 0) {
6512         end_op();
6513         return -1;
6514     }
6515
6516     ilock(ip);
6517     if (ip->type == T_DIR) {
6518         iunlockput(ip);
6519         end_op();
6520         return -1;
6521     }
6522
6523     ip->nlink++;
6524     iupdate(ip);
6525     iunlock(ip);
6526
6527     if ((dp = nameiparent(new, name)) == 0)
6528         goto bad;
6529     ilock(dp);
6530     if (dp->dev != ip->dev || dirlink(dp, name, ip->inum) < 0) {
6531         iunlockput(dp);
6532         goto bad;
6533     }
6534     iunlockput(dp);
6535     iput(ip);
6536
6537     end_op();
6538
6539     return 0;
6540
6541 bad:
6542     ilock(ip);
6543     ip->nlink--;
6544     iupdate(ip);
6545     iunlockput(ip);
6546     end_op();
6547     return -1;
6548 }
6549

```

```

6550 // Is the directory dp empty except for "." and ".." ?
6551 static int
6552 isdirempty(struct inode *dp)
6553 {
6554     int off;
6555     struct dirent de;
6556
6557     for (off = 2 * sizeof(de); off < dp->size; off += sizeof(de)) {
6558         if (readi(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
6559             panic("isdirempty: readi");
6560         if (de.inum != 0)
6561             return 0;
6562     }
6563     return 1;
6564 }
6565
6566 uint64
6567 sys_unlink(void)
6568 {
6569     struct inode *ip, *dp;
6570     struct dirent de;
6571     char name[DIRSIZ], path[MAXPATH];
6572     uint off;
6573
6574     if (argstr(0, path, MAXPATH) < 0)
6575         return -1;
6576
6577     begin_op();
6578     if ((dp = nameiparent(path, name)) == 0) {
6579         end_op();
6580         return -1;
6581     }
6582
6583     ilock(dp);
6584
6585     // Cannot unlink "." or "..".
6586     if (namecmp(name, ".") == 0 || namecmp(name, "..") == 0)
6587         goto bad;
6588
6589     if ((ip = dirlookup(dp, name, &off)) == 0)
6590         goto bad;
6591     ilock(ip);
6592
6593     if (ip->nlink < 1)
6594         panic("unlink: nlink < 1");
6595     if (ip->type == T_DIR && !isdirempty(ip)) {
6596         iunlockput(ip);
6597         goto bad;
6598     }
6599

```

```

6600  memset(&de, 0, sizeof(de));
6601  if (writei(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
6602      panic("unlink: writei");
6603  if (ip->type == T_DIR) {
6604      dp->nlink--;
6605      iupdate(dp);
6606  }
6607  iunlockput(dp);
6608
6609  ip->nlink--;
6610  iupdate(ip);
6611  iunlockput(ip);
6612
6613  end_op();
6614
6615  return 0;
6616
6617 bad:
6618  iunlockput(dp);
6619  end_op();
6620  return -1;
6621 }
6622
6623 static struct inode *
6624 create(char *path, short type, short major, short minor)
6625 {
6626     struct inode *ip, *dp;
6627     char name[DIRSIZ];
6628
6629     if ((dp = nameiparent(path, name)) == 0)
6630         return 0;
6631
6632     ilock(dp);
6633
6634     if ((ip = dirlookup(dp, name, 0)) != 0) {
6635         iunlockput(dp);
6636         ilock(ip);
6637         if (type == T_FILE && (ip->type == T_FILE || ip->type == T_DEVICE))
6638             return ip;
6639         iunlockput(ip);
6640         return 0;
6641     }
6642
6643     if ((ip = ialloc(dp->dev, type)) == 0) {
6644         iunlockput(dp);
6645         return 0;
6646     }
6647
6648
6649

```

```

6650  ilock(ip);
6651  ip->major = major;
6652  ip->minor = minor;
6653  ip->nlink = 1;
6654  iupdate(ip);
6655
6656  if (type == T_DIR) { // Create . and .. entries.
6657      // No ip->nlink++ for ".": avoid cyclic ref count.
6658      if (dirlink(ip, ".", ip->inum) < 0 || dirlink(ip, "..", dp->inum) < 0)
6659          goto fail;
6660  }
6661
6662  if (dirlink(dp, name, ip->inum) < 0)
6663      goto fail;
6664
6665  if (type == T_DIR) {
6666      // now that success is guaranteed:
6667      dp->nlink++; // for ".."
6668      iupdate(dp);
6669  }
6670
6671  iunlockput(dp);
6672
6673  return ip;
6674
6675 fail:
6676  // something went wrong. de-allocate ip.
6677  ip->nlink = 0;
6678  iupdate(ip);
6679  iunlockput(ip);
6680  iunlockput(dp);
6681  return 0;
6682 }
6683
6684 uint64
6685 sys_open(void)
6686 {
6687     char path[MAXPATH];
6688     int fd, omode;
6689     struct file *f;
6690     struct inode *ip;
6691     int n;
6692
6693     argint(1, &omode);
6694     if ((n = argstr(0, path, MAXPATH)) < 0)
6695         return -1;
6696
6697     begin_op();
6698
6699

```

```

6700  if (omode & O_CREATE) {
6701      ip = create(path, T_FILE, 0, 0);
6702      if (ip == 0) {
6703          end_op();
6704          return -1;
6705      }
6706  } else {
6707      if ((ip = namei(path)) == 0) {
6708          end_op();
6709          return -1;
6710      }
6711      ilock(ip);
6712      if (ip->type == T_DIR && omode != O_RDONLY) {
6713          iunlockput(ip);
6714          end_op();
6715          return -1;
6716      }
6717  }
6718
6719  if (ip->type == T_DEVICE && (ip->major < 0 || ip->major >= NDEV)) {
6720      iunlockput(ip);
6721      end_op();
6722      return -1;
6723  }
6724
6725  if ((f = filealloc()) == 0 || (fd = fdalloc(f)) < 0) {
6726      if (f)
6727          fileclose(f);
6728      iunlockput(ip);
6729      end_op();
6730      return -1;
6731  }
6732
6733  if (ip->type == T_DEVICE) {
6734      f->type = FD_DEVICE;
6735      f->major = ip->major;
6736  } else {
6737      f->type = FD_INODE;
6738      f->off = 0;
6739  }
6740  f->ip = ip;
6741  f->readable = !(omode & O_WRONLY);
6742  f->writable = (omode & O_WRONLY) || (omode & O_RDWR);
6743  f->append = (omode & O_APPEND);
6744
6745  if ((omode & O_TRUNC) && ip->type == T_FILE) {
6746      itrunc(ip);
6747  }
6748
6749

```

```

6750     iunlock(ip);
6751     end_op();
6752
6753     return fd;
6754 }
6755
6756 uint64
6757 sys_mkdir(void)
6758 {
6759     char path[MAXPATH];
6760     struct inode *ip;
6761
6762     begin_op();
6763     if (argstr(0, path, MAXPATH) < 0 ||
6764         (ip = create(path, T_DIR, 0, 0)) == 0) {
6765         end_op();
6766         return -1;
6767     }
6768     iunlockput(ip);
6769     end_op();
6770     return 0;
6771 }
6772
6773 uint64
6774 sys_mknod(void)
6775 {
6776     struct inode *ip;
6777     char path[MAXPATH];
6778     int major, minor;
6779
6780     begin_op();
6781     argint(1, &major);
6782     argint(2, &minor);
6783     if ((argstr(0, path, MAXPATH)) < 0 ||
6784         (ip = create(path, T_DEVICE, major, minor)) == 0) {
6785         end_op();
6786         return -1;
6787     }
6788     iunlockput(ip);
6789     end_op();
6790     return 0;
6791 }
6792
6793
6794
6795
6796
6797
6798
6799

```

```

6800 uint64
6801 sys_chdir(void)
6802 {
6803     char path[MAXPATH];
6804     struct inode *ip;
6805     struct proc *p = myproc();
6806
6807     begin_op();
6808     if (argstr(0, path, MAXPATH) < 0 || (ip = namei(path)) == 0) {
6809         end_op();
6810         return -1;
6811     }
6812     ilock(ip);
6813     if (ip->type != T_DIR) {
6814         iunlockput(ip);
6815         end_op();
6816         return -1;
6817     }
6818     iunlock(ip);
6819     iput(p->cwd);
6820     end_op();
6821     p->cwd = ip;
6822     return 0;
6823 }
6824
6825 uint64
6826 sys_exec(void)
6827 {
6828     char path[MAXPATH], *argv[MAXARG];
6829     int i;
6830     uint64 uargv, uarg;
6831
6832     argaddr(1, &uargv);
6833     if (argstr(0, path, MAXPATH) < 0) {
6834         return -1;
6835     }
6836     memset(argv, 0, sizeof(argv));
6837     for (i = 0;; i++) {
6838         if (i >= NELEM(argv)) {
6839             goto bad;
6840         }
6841         if (fetchaddr(uargv + sizeof(uint64) * i, (uint64 *)&uarg) < 0) {
6842             goto bad;
6843         }
6844         if (uarg == 0) {
6845             argv[i] = 0;
6846             break;
6847         }
6848         argv[i] = kalloc();
6849         if (argv[i] == 0)

```

```

6850     goto bad;
6851     if (fetchstr(uarg, argv[i], PGSIZE) < 0)
6852         goto bad;
6853 }
6854
6855 int ret = kexec(path, argv);
6856
6857 for (i = 0; i < NELEM(argv) && argv[i] != 0; i++)
6858     kfree(argv[i]);
6859
6860 return ret;
6861
6862 bad:
6863 for (i = 0; i < NELEM(argv) && argv[i] != 0; i++)
6864     kfree(argv[i]);
6865 return -1;
6866 }
6867
6868 uint64
6869 sys_pipe(void)
6870 {
6871     uint64 fdarray; // user pointer to array of two integers
6872     struct file *rf, *wf;
6873     int fd0, fd1;
6874     struct proc *p = myproc();
6875
6876     argaddr(0, &fdarray);
6877     if (pipealloc(&rf, &wf) < 0)
6878         return -1;
6879     fd0 = -1;
6880     if ((fd0 = fdalloc(rf)) < 0 || (fd1 = fdalloc(wf)) < 0) {
6881         if (fd0 >= 0)
6882             p->ofile[fd0] = 0;
6883         fileclose(rf);
6884         fileclose(wf);
6885         return -1;
6886     }
6887     if (copyout(p->pagetable, fdarray, (char *)&fd0,
6888         sizeof(fd0)) < 0 || copyout(p->pagetable, fdarray + sizeof(fd0),
6889         (char *)&fd1, sizeof(fd1)) < 0) {
6890         p->ofile[fd0] = 0;
6891         p->ofile[fd1] = 0;
6892         fileclose(rf);
6893         fileclose(wf);
6894         return -1;
6895     }
6896     return 0;
6897 }
6898
6899

```

```

6900 int sys_lseek(void) {
6901     int fd;
6902     int offset;
6903     int whence;
6904     struct file *f;
6905     argfd(0, &fd, &f);
6906     argint(1, &offset);
6907     argint(2, &whence);
6908
6909     if( fd < 0 || offset < 0 || whence < 0)
6910         return -1;
6911
6912     ilock(f->ip);
6913     int fsize = f->ip->size;
6914     iunlock(f->ip);
6915
6916     // Calculate new offset based on whence
6917     int new_offset = 0;
6918     if (whence == 0) { // SEEK_SET
6919         new_offset = offset;
6920     } else if (whence == 1) { // SEEK_CUR
6921         new_offset = f->off + offset;
6922     } else if (whence == 2) { // SEEK_END
6923         new_offset = fsize + offset;
6924     } else {
6925         return -1; // Invalid whence
6926     }
6927
6928     // Safety checks
6929     if (new_offset < 0) return -1;
6930     // we can't extend the file's size with this function
6931     if (new_offset > fsize) return -1;
6932
6933     // Update the actual offset
6934     f->off = new_offset;
6935     return f->off;
6936 }
6937
6938 // Read the super block.
6939 static void
6940 readsb(int dev, struct superblock *sb)
6941 {
6942     struct buf *bp;
6943
6944     bp = bread(dev, 1);
6945     memmove(sb, bp->data, sizeof(*sb));
6946     brelse(bp);
6947 }
6948
6949

```

```

6950 uint64
6951 sys_getfsdata(void)
6952 {
6953     uint64 user_addr;
6954     struct superblock sb;
6955     struct fs_dump dump;
6956     struct buf *bp;
6957
6958     // Retrieve the user-space pointer passed as the first argument
6959     argaddr(0, &user_addr);
6960
6961     // Read the superblock (ROOTDEV is defined as 1 in kernel)
6962     readsb(ROOTDEV, &sb);
6963
6964     dump.size = sb.size;
6965     dump.nblocks = sb.nblocks;
6966     dump.ninodes = sb.ninodes;
6967     dump.bmapstart = sb.bmapstart;
6968
6969     // Read the first bitmap block into the buffer
6970     bp = bread(ROOTDEV, sb.bmapstart);
6971
6972     // Copy bitmap data safely (up to the size of our buffer)
6973     memmove(dump.bitmap, bp->data, sizeof(dump.bitmap));
6974     brelse(bp);
6975
6976     // Safely copy the struct from kernel space to the user space virtual
6977     // memory address
6978     if(copyout(myproc()->pagetable, user_addr, (char *)&dump,
6979               sizeof(dump)) < 0)
6980         return -1; // Return error if copy fails
6981
6982     return 0; // Success
6983 }
6984
6985 uint64
6986 sys_gettime(void)
6987 {
6988
6989     uint64 mode;
6990     // Retrieve the user-space pointer passed as the first argument
6991     argaddr(0, &mode);
6992     // Returns unix timestamp safely over to user space
6993     return goldfish_rtc_read(mode);
6994 }
6995
6996
6997
6998
6999

```

```

7000 #include "types.h"
7001 #include "param.h"
7002 #include "memlayout.h"
7003 #include "riscv.h"
7004 #include "spinlock.h"
7005 #include "proc.h"
7006 #include "defs.h"
7007 #include "elf.h"
7008
7009 static int loadseg(pde_t *, uint64, struct inode *, uint, uint);
7010
7011 // map ELF permissions to PTE permission bits.
7012 int
7013 flags2perm(int flags)
7014 {
7015     int perm = 0;
7016     if (flags & 0x1)
7017         perm = PTE_X;
7018     if (flags & 0x2)
7019         perm |= PTE_W;
7020     return perm;
7021 }
7022
7023 //
7024 // the implementation of the exec() system call
7025 //
7026 int
7027 kexec(char *path, char **argv)
7028 {
7029     char *s, *last;
7030     int i, off;
7031     uint64 argc, sz = 0, sp, ustack[MAXARG], stackbase;
7032     struct elfhdr elf;
7033     struct inode *ip;
7034     struct proghdr ph;
7035     pagetable_t pagetable = 0, oldpagetable;
7036     struct proc *p = myproc();
7037
7038     begin_op();
7039
7040     // Open the executable file.
7041     if ((ip = namei(path)) == 0) {
7042         end_op();
7043         return -1;
7044     }
7045     ilock(ip);
7046
7047     // Read the ELF header.
7048     if (readi(ip, 0, (uint64*)&elf, 0, sizeof(elf)) != sizeof(elf))
7049         goto bad;

```

```

7050 // Is this really an ELF file?
7051 if (elf.magic != ELF_MAGIC)
7052     goto bad;
7053
7054 if ((pagetable = proc_pagetable(p)) == 0)
7055     goto bad;
7056
7057 // Load program into memory.
7058 for (i = 0, off = elf.phoff; i < elf.phnum; i++, off += sizeof(ph)) {
7059     if (readi(ip, 0, (uint64*)&ph, off, sizeof(ph)) != sizeof(ph))
7060         goto bad;
7061     if (ph.type != ELF_PROG_LOAD)
7062         continue;
7063     if (ph.memsz < ph.filesz)
7064         goto bad;
7065     if (ph.vaddr + ph.memsz < ph.vaddr)
7066         goto bad;
7067     if (ph.vaddr % PGSIZE != 0)
7068         goto bad;
7069     uint64 sz1;
7070     if ((sz1 = uvmmalloc(pagetable, sz, ph.vaddr + ph.memsz,
7071                          flags2perm(ph.flags))) == 0)
7072         goto bad;
7073     sz = sz1;
7074     if (loadseg(pagetable, ph.vaddr, ip, ph.off, ph.filesz) < 0)
7075         goto bad;
7076 }
7077 iunlockput(ip);
7078 end_op();
7079 ip = 0;
7080
7081 p = myproc();
7082 uint64 oldsz = p->sz;
7083
7084 // Allocate some pages at the next page boundary.
7085 // Make the first inaccessible as a stack guard.
7086 // Use the rest as the user stack.
7087 sz = PGROUNDUP(sz);
7088 uint64 sz1;
7089 if ((sz1 = uvmmalloc(pagetable, sz, sz + (USERSTACK + 1) * PGSIZE,
7090                      PTE_W)) == 0)
7091     goto bad;
7092 sz = sz1;
7093 uvmclear(pagetable, sz - (USERSTACK + 1) * PGSIZE);
7094 sp = sz;
7095 stackbase = sp - USERSTACK * PGSIZE;
7096
7097
7098
7099

```

```

7100 // Copy argument strings into new stack, remember their
7101 // addresses in ustack[].
7102 for (argc = 0; argv[argc]; argc++) {
7103     if (argc >= MAXARG)
7104         goto bad;
7105     sp -= strlen(argv[argc]) + 1;
7106     sp -= sp % 16; // riscv sp must be 16-byte aligned
7107     if (sp < stackbase)
7108         goto bad;
7109     if (copyout(pagetable, sp, argv[argc], strlen(argv[argc]) + 1) < 0)
7110         goto bad;
7111     ustack[argc] = sp;
7112 }
7113 ustack[argc] = 0;
7114
7115 // push a copy of ustack[], the array of argv[] pointers.
7116 sp -= (argc + 1) * sizeof(uint64);
7117 sp -= sp % 16;
7118 if (sp < stackbase)
7119     goto bad;
7120 if (copyout(pagetable, sp, (char *)ustack, (argc + 1) *
7121           sizeof(uint64)) < 0)
7122     goto bad;
7123
7124 // a0 and a1 contain arguments to user main(argc, argv)
7125 // argc is returned via the system call return
7126 // value, which goes in a0.
7127 p->trapframe->a1 = sp;
7128
7129 // Save program name for debugging.
7130 for (last = s = path; *s; s++)
7131     if (*s == '/')
7132         last = s + 1;
7133 safestrcpy(p->name, last, sizeof(p->name));
7134
7135 // Commit to the user image.
7136 oldpagetable = p->pagetable;
7137 p->pagetable = pagetable;
7138 p->sz = sz;
7139 p->trapframe->epc = elf.entry; // initial program counter =
7140                               //          ulib.c:start()
7141 p->trapframe->sp = sp;          // initial stack pointer
7142 proc_freepagetable(oldpagetable, oldsz);
7143
7144 return argc; // this ends up in a0, the first argument to
7145             //          main(argc, argv)
7146
7147
7148
7149

```

```
7150 bad:
7151     if (pagetable)
7152         proc_freepagetable(pagetable, sz);
7153     if (ip) {
7154         iunlockput(ip);
7155         end_op();
7156     }
7157     return -1;
7158 }
7159
7160 // Load an ELF program segment into pagetable at virtual address va.
7161 // va must be page-aligned
7162 // and the pages from va to va+sz must already be mapped.
7163 // Returns 0 on success, -1 on failure.
7164 static int
7165 loadseg(pagetable_t pagetable, uint64 va, struct inode *ip, uint offset,
7166         uint sz)
7167 {
7168     uint i, n;
7169     uint64 pa;
7170
7171     for (i = 0; i < sz; i += PGSIZE) {
7172         pa = walkaddr(pagetable, va + i);
7173         if (pa == 0)
7174             panic("loadseg: address should exist");
7175         if (sz - i < PGSIZE)
7176             n = sz - i;
7177         else
7178             n = PGSIZE;
7179         if (readi(ip, 0, (uint64)pa, offset + i, n) != n)
7180             return -1;
7181     }
7182
7183     return 0;
7184 }
7185
7186
7187
7188
7189
7190
7191
7192
7193
7194
7195
7196
7197
7198
7199
```

```
7200 #include "types.h"
7201 #include "riscv.h"
7202 #include "defs.h"
7203 #include "param.h"
7204 #include "spinlock.h"
7205 #include "proc.h"
7206 #include "fs.h"
7207 #include "sleeplock.h"
7208 #include "file.h"
7209
7210 #define PIPESIZE 512
7211
7212 struct pipe {
7213     struct spinlock lock;
7214     char data[PIPESIZE];
7215     uint nread;    // number of bytes read
7216     uint nwrite;   // number of bytes written
7217     int readopen;  // read fd is still open
7218     int writeopen; // write fd is still open
7219 };
7220
7221 int
7222 pipealloc(struct file **f0, struct file **f1)
7223 {
7224     struct pipe *pi;
7225
7226     pi = 0;
7227     *f0 = *f1 = 0;
7228     if ((*f0 = filealloc()) == 0 || (*f1 = filealloc()) == 0)
7229         goto bad;
7230     if ((pi = (struct pipe *)kalloc()) == 0)
7231         goto bad;
7232     pi->readopen = 1;
7233     pi->writeopen = 1;
7234     pi->nwrite = 0;
7235     pi->nread = 0;
7236     initlock(&pi->lock, "pipe");
7237     (*f0)->type = FD_PIPE;
7238     (*f0)->readable = 1;
7239     (*f0)->writable = 0;
7240     (*f0)->pipe = pi;
7241     (*f1)->type = FD_PIPE;
7242     (*f1)->readable = 0;
7243     (*f1)->writable = 1;
7244     (*f1)->pipe = pi;
7245     return 0;
7246
7247
7248
7249
```

```

7250 bad:
7251     if (pi)
7252         kfree((char *)pi);
7253     if (*f0)
7254         fileclose(*f0);
7255     if (*f1)
7256         fileclose(*f1);
7257     return -1;
7258 }
7259
7260 void
7261 pipeclose(struct pipe *pi, int writable)
7262 {
7263     acquire(&pi->lock);
7264     if (writable) {
7265         pi->writeopen = 0;
7266         wakeup(&pi->nread);
7267     } else {
7268         pi->readopen = 0;
7269         wakeup(&pi->nwrite);
7270     }
7271     if (pi->readopen == 0 && pi->writeopen == 0) {
7272         release(&pi->lock);
7273         kfree((char *)pi);
7274     } else
7275         release(&pi->lock);
7276 }
7277
7278 int
7279 pipewrite(struct pipe *pi, uint64 addr, int n)
7280 {
7281     int i = 0;
7282     struct proc *pr = myproc();
7283
7284     acquire(&pi->lock);
7285     while (i < n) {
7286         if (pi->readopen == 0 || killed(pr)) {
7287             release(&pi->lock);
7288             return -1;
7289         }
7290         if (pi->nwrite == pi->nread + PIPESIZE) {
7291             wakeup(&pi->nread);
7292             sleep(&pi->nwrite, &pi->lock);
7293         } else {
7294             char ch;
7295             if (copyin(pr->pagetable, &ch, addr + i, 1) == -1)
7296                 break;
7297             pi->data[pi->nwrite++ % PIPESIZE] = ch;
7298             i++;
7299         }

```

```
7300     }
7301     wakeup(&pi->nread);
7302     release(&pi->lock);
7303
7304     return i;
7305 }
7306
7307 int
7308 piperead(struct pipe *pi, uint64 addr, int n)
7309 {
7310     int i;
7311     struct proc *pr = myproc();
7312     char ch;
7313
7314     acquire(&pi->lock);
7315     while (pi->nread == pi->nwrite && pi->writeopen) {
7316         if (killed(pr)) {
7317             release(&pi->lock);
7318             return -1;
7319         }
7320         sleep(&pi->nread, &pi->lock);
7321     }
7322     for (i = 0; i < n; i++) {
7323         if (pi->nread == pi->nwrite)
7324             break;
7325         ch = pi->data[pi->nread % PIPESIZE];
7326         if (copyout(pr->pagetable, addr + i, &ch, 1) == -1) {
7327             if (i == 0)
7328                 i = -1;
7329             break;
7330         }
7331         pi->nread++;
7332     }
7333     wakeup(&pi->nwrite);
7334     release(&pi->lock);
7335     return i;
7336 }
7337
7338
7339
7340
7341
7342
7343
7344
7345
7346
7347
7348
7349
```

```
7350 #include "types.h"
7351
7352 void *
7353 memset(void *dst, int c, uint n)
7354 {
7355     char *cdst = (char *)dst;
7356     int i;
7357     for (i = 0; i < n; i++) {
7358         cdst[i] = c;
7359     }
7360     return dst;
7361 }
7362
7363 int
7364 memcmp(const void *v1, const void *v2, uint n)
7365 {
7366     const uchar *s1, *s2;
7367
7368     s1 = v1;
7369     s2 = v2;
7370     while (n-- > 0) {
7371         if (*s1 != *s2)
7372             return *s1 - *s2;
7373         s1++, s2++;
7374     }
7375
7376     return 0;
7377 }
7378
7379 void *
7380 memmove(void *dst, const void *src, uint n)
7381 {
7382     const char *s;
7383     char *d;
7384
7385     if (n == 0)
7386         return dst;
7387
7388     s = src;
7389     d = dst;
7390     if (s < d && s + n > d) {
7391         s += n;
7392         d += n;
7393         while (n-- > 0)
7394             *--d = *--s;
7395     } else
7396         while (n-- > 0)
7397             *d++ = *s++;
7398
7399 }
```

```

7400     return dst;
7401 }
7402
7403 // memcpy exists to placate GCC.  Use memmove.
7404 void *
7405 memcpy(void *dst, const void *src, uint n)
7406 {
7407     return memmove(dst, src, n);
7408 }
7409
7410 int
7411 strcmp(const char *p, const char *q, uint n)
7412 {
7413     while (n > 0 && *p && *p == *q)
7414         n--, p++, q++;
7415     if (n == 0)
7416         return 0;
7417     return (uchar)*p - (uchar)*q;
7418 }
7419
7420 char *
7421 strncpy(char *s, const char *t, int n)
7422 {
7423     char *os;
7424
7425     os = s;
7426     while (n-- > 0 && (*s++ = *t++) != 0)
7427         ;
7428     while (n-- > 0)
7429         *s++ = 0;
7430     return os;
7431 }
7432
7433 // Like strncpy but guaranteed to NUL-terminate.
7434 char *
7435 safestrcpy(char *s, const char *t, int n)
7436 {
7437     char *os;
7438
7439     os = s;
7440     if (n <= 0)
7441         return os;
7442     while (--n > 0 && (*s++ = *t++) != 0)
7443         ;
7444     *s = 0;
7445     return os;
7446 }
7447
7448
7449

```

```
7450 int
7451 strlen(const char *s)
7452 {
7453     int n;
7454
7455     for (n = 0; s[n]; n++)
7456         ;
7457     return n;
7458 }
7459
7460
7461
7462
7463
7464
7465
7466
7467
7468
7469
7470
7471
7472
7473
7474
7475
7476
7477
7478
7479
7480
7481
7482
7483
7484
7485
7486
7487
7488
7489
7490
7491
7492
7493
7494
7495
7496
7497
7498
7499
```

```
7500 #include "types.h"
7501 #include "param.h"
7502 #include "memlayout.h"
7503 #include "riscv.h"
7504 #include "defs.h"
7505
7506 //
7507 // the riscv Platform Level Interrupt Controller (PLIC).
7508 //
7509
7510 void
7511 plicinit(void)
7512 {
7513     // set desired IRQ priorities non-zero (otherwise disabled).
7514     *(uint32 *) (PLIC + UART0_IRQ * 4) = 1;
7515     *(uint32 *) (PLIC + VIRTIO0_IRQ * 4) = 1;
7516 }
7517
7518 void
7519 plicinithart(void)
7520 {
7521     int hart = cpuid();
7522
7523     // set enable bits for this hart's S-mode
7524     // for the uart and virtio disk.
7525     *(uint32 *) PLIC_SENABLE(hart) = (1 << UART0_IRQ) | (1 << VIRTIO0_IRQ);
7526
7527     // set this hart's S-mode priority threshold to 0.
7528     *(uint32 *) PLIC_SPRIORITY(hart) = 0;
7529 }
7530
7531 // ask the PLIC what interrupt we should serve.
7532 int
7533 plic_claim(void)
7534 {
7535     int hart = cpuid();
7536     int irq = *(uint32 *) PLIC_SCLAIM(hart);
7537     return irq;
7538 }
7539
7540 // tell the PLIC we've served this IRQ.
7541 void
7542 plic_complete(int irq)
7543 {
7544     int hart = cpuid();
7545     *(uint32 *) PLIC_SCLAIM(hart) = irq;
7546 }
7547
7548
7549
```

```

7550 //
7551 // Console input and output, to the uart.
7552 // Reads are line at a time.
7553 // Implements special input characters:
7554 //  newline -- end of line
7555 //  control-h -- backspace
7556 //  control-u -- kill line
7557 //  control-d -- end of file
7558 //  control-p -- print process list
7559 //
7560
7561 #include <stdarg.h>
7562
7563 #include "types.h"
7564 #include "param.h"
7565 #include "spinlock.h"
7566 #include "sleeplock.h"
7567 #include "fs.h"
7568 #include "file.h"
7569 #include "memlayout.h"
7570 #include "riscv.h"
7571 #include "defs.h"
7572 #include "proc.h"
7573
7574 #define BACKSPACE 0x100          // erase the last output character
7575 #define C(x)      ((x) - '@') // Control-x
7576
7577 //
7578 // send one character to the uart, but don't use
7579 // interrupts or sleep(). safe to be called from
7580 // interrupts, e.g. by printk and to echo input
7581 // characters.
7582 //
7583 void
7584 consputc(int c)
7585 {
7586     if (c == BACKSPACE) {
7587         // if the user typed backspace, overwrite with a space.
7588         uartputc_sync('\b');
7589         uartputc_sync(' ');
7590         uartputc_sync('\b');
7591     } else {
7592         uartputc_sync(c);
7593     }
7594 }
7595
7596
7597
7598
7599

```

```

7600 struct {
7601     struct spinlock lock;
7602
7603     // input circular buffer
7604 #define INPUT_BUF_SIZE 128
7605     char buf[INPUT_BUF_SIZE];
7606     uint r; // Read index
7607     uint w; // Write index
7608     uint e; // Edit index
7609 } cons;
7610
7611 //
7612 // user write() system calls to the console go here.
7613 // uses sleep() and UART interrupts.
7614 //
7615 int
7616 consolewrite(int user_src, uint64 src, int n)
7617 {
7618     char buf[32]; // move batches from user space to uart.
7619     int i = 0;
7620
7621     while (i < n) {
7622         int nn = sizeof(buf);
7623         if (nn > n - i)
7624             nn = n - i;
7625         if (either_copyin(buf, user_src, src + i, nn) == -1)
7626             break;
7627         uartwrite(buf, nn);
7628         i += nn;
7629     }
7630
7631     return i;
7632 }
7633
7634 //
7635 // user read()s from the console go here.
7636 // copy (up to) a whole input line to dst.
7637 // user_dst indicates whether dst is a user
7638 // or kernel address.
7639 //
7640 int
7641 consoleread(int user_dst, uint64 dst, int n)
7642 {
7643     uint target;
7644     int c;
7645     char cbuf;
7646
7647     target = n;
7648     acquire(&cons.lock);
7649     while (n > 0) {

```

```

7650 // wait until interrupt handler has put some
7651 // input into cons.buffer.
7652 while (cons.r == cons.w) {
7653     if (killed(myproc())) {
7654         release(&cons.lock);
7655         return -1;
7656     }
7657     sleep(&cons.r, &cons.lock);
7658 }
7659
7660 c = cons.buf[cons.r++ % INPUT_BUF_SIZE];
7661
7662 if (c == C('D')) { // end-of-file
7663     if (n < target) {
7664         // Save ^D for next time, to make sure
7665         // caller gets a 0-byte result.
7666         cons.r--;
7667     }
7668     break;
7669 }
7670
7671 // copy the input byte to the user-space buffer.
7672 cbuf = c;
7673 if (either_copyout(user_dst, dst, &cbuf, 1) == -1)
7674     break;
7675
7676 dst++;
7677 --n;
7678
7679 if (c == '\n') {
7680     // a whole line has arrived, return to
7681     // the user-level read().
7682     break;
7683 }
7684 }
7685 release(&cons.lock);
7686
7687 return target - n;
7688 }
7689
7690
7691
7692
7693
7694
7695
7696
7697
7698
7699

```

```

7700 //
7701 // the console input interrupt handler.
7702 // uartintr() calls this for each input character.
7703 // do erase/kill processing, append to cons.buf,
7704 // wake up consoleread() if a whole line has arrived.
7705 //
7706 void
7707 consoleintr(int c)
7708 {
7709     acquire(&cons.lock);
7710
7711     switch (c) {
7712     case C('P'): // Print process list.
7713         procdump();
7714         break;
7715     case C('U'): // Kill line.
7716         while (cons.e != cons.w &&
7717             cons.buf[(cons.e - 1) % INPUT_BUF_SIZE] != '\n') {
7718             cons.e--;
7719             consputc(BACKSPACE);
7720         }
7721         break;
7722     case C('H'): // Backspace
7723     case '\x7f': // Delete key
7724         if (cons.e != cons.w) {
7725             cons.e--;
7726             consputc(BACKSPACE);
7727         }
7728         break;
7729     case C('C'): // Added this case for Ctrl+C
7730         // Call a function to kill the foreground process
7731         killforeground();
7732         break;
7733     default:
7734         if (c != 0 && cons.e - cons.r < INPUT_BUF_SIZE) {
7735             c = (c == '\r') ? '\n' : c;
7736
7737             // echo back to the user.
7738             consputc(c);
7739
7740             // store for consumption by consoleread().
7741             cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
7742
7743             if (c == '\n' || c == C('D') || cons.e - cons.r == INPUT_BUF_SIZE) {
7744                 // wake up consoleread() if a whole line (or end-of-file)
7745                 // has arrived.
7746                 cons.w = cons.e;
7747                 wakeup(&cons.r);
7748             }
7749         }

```

```
7750     break;
7751 }
7752
7753     release(&cons.lock);
7754 }
7755
7756 void
7757 consoleinit(void)
7758 {
7759     initlock(&cons.lock, "cons");
7760
7761     uartinit();
7762
7763     // connect read and write system calls
7764     // to consoleread and consolewrite.
7765     devsw[CONSOLE].read = consoleread;
7766     devsw[CONSOLE].write = consolewrite;
7767 }
7768
7769
7770
7771
7772
7773
7774
7775
7776
7777
7778
7779
7780
7781
7782
7783
7784
7785
7786
7787
7788
7789
7790
7791
7792
7793
7794
7795
7796
7797
7798
7799
```

```

7800 //
7801 // low-level driver for 16550a UART.
7802 //
7803
7804 #include "types.h"
7805 #include "param.h"
7806 #include "memlayout.h"
7807 #include "riscv.h"
7808 #include "spinlock.h"
7809 #include "proc.h"
7810 #include "defs.h"
7811
7812 // the UART control registers are memory-mapped
7813 // at address UART0. this macro returns the
7814 // address of one of the registers.
7815 #define Reg(reg) ((volatile unsigned char *) (UART0 + (reg)))
7816
7817 #define ReadReg(reg) (*(Reg(reg)))
7818 #define WriteReg(reg, v) (*(Reg(reg)) = (v))
7819
7820 // the UART control registers.
7821 // some have different meanings for read vs write.
7822 // see http://byterunner.com/16550.html
7823 #define RHR 0 // receive holding register (for
7824 // input bytes)
7825 #define THR 0 // transmit holding register (for
7826 // output bytes)
7827 #define IER 1 // interrupt enable register
7828 #define IER_RX_ENABLE (1 << 0) // receiver interrupts
7829 #define IER_TX_ENABLE (1 << 1) // transmit interrupts
7830 #define FCR 2 // FIFO control register
7831 #define FCR_FIFO_ENABLE (1 << 0)
7832 #define FCR_FIFO_CLEAR (3 << 1) // clear the content of the two FIFOs
7833 #define ISR 2 // interrupt status register
7834 #define LCR 3 // line control register
7835 #define LCR_EIGHT_BITS (3 << 0)
7836 #define LCR_BAUD_LATCH (1 << 7) // special mode to set baud rate
7837 #define LSR 5 // line status register
7838 #define LSR_RX_READY (1 << 0) // input is waiting to be read from RHR
7839 #define LSR_TX_IDLE (1 << 5) // THR can accept another character
7840 // to send
7841
7842 // for sending threads to synchronize with uart "ready" interrupts.
7843 static struct spinlock tx_lock;
7844 static int tx_busy; // is the UART busy sending?
7845 static int tx_chan; // &tx_chan is the "wait channel"
7846
7847 extern volatile int panicking; // from printk.c
7848 extern volatile int panicked; // from printk.c
7849

```

```
7850 void
7851 uartinit(void)
7852 {
7853     // disable interrupts.
7854     WriteReg(IER, 0x00);
7855
7856     // special mode to set baud rate.
7857     WriteReg(LCR, LCR_BAUD_LATCH);
7858
7859     // LSB for baud rate of 38.4K.
7860     WriteReg(0, 0x03);
7861
7862     // MSB for baud rate of 38.4K.
7863     WriteReg(1, 0x00);
7864
7865     // leave set-baud mode,
7866     // and set word length to 8 bits, no parity.
7867     WriteReg(LCR, LCR_EIGHT_BITS);
7868
7869     // reset and enable FIFOs.
7870     WriteReg(FCR, FCR_FIFO_ENABLE | FCR_FIFO_CLEAR);
7871
7872     // enable transmit and receive interrupts.
7873     WriteReg(IER, IER_TX_ENABLE | IER_RX_ENABLE);
7874
7875     initlock(&tx_lock, "uart");
7876 }
7877
7878 // transmit buf[] to the uart. it blocks if the
7879 // uart is busy, so it cannot be called from
7880 // interrupts, only from write() system calls.
7881 void
7882 uartwrite(char buf[], int n)
7883 {
7884     acquire(&tx_lock);
7885
7886     int i = 0;
7887     while (i < n) {
7888         while (tx_busy != 0) {
7889             // wait for a UART transmit-complete interrupt
7890             // to set tx_busy to 0.
7891             sleep(&tx_chan, &tx_lock);
7892         }
7893
7894         WriteReg(THR, buf[i]);
7895         i += 1;
7896         tx_busy = 1;
7897     }
7898
7899 }
```

```
7900     release(&tx_lock);
7901 }
7902
7903 // write a byte to the uart without using
7904 // interrupts, for use by kernel printf() and
7905 // to echo characters. it spins waiting for the uart's
7906 // output register to be empty.
7907 void
7908 uartputc_sync(int c)
7909 {
7910     if (panicking == 0)
7911         push_off();
7912
7913     if (panicked) {
7914         for (;;)
7915             ;
7916     }
7917
7918     // wait for UART to set Transmit Holding Empty in LSR.
7919     while ((ReadReg(LSR) & LSR_TX_IDLE) == 0)
7920         ;
7921     WriteReg(THR, c);
7922
7923     if (panicking == 0)
7924         pop_off();
7925 }
7926
7927 // try to read one input character from the UART.
7928 // return -1 if none is waiting.
7929 static int
7930 uartgetc(void)
7931 {
7932     // is input ready?
7933     if (ReadReg(LSR) & LSR_RX_READY) {
7934         return ReadReg(RHR);
7935     } else {
7936         return -1;
7937     }
7938 }
7939
7940
7941
7942
7943
7944
7945
7946
7947
7948
7949
```

```
7950 // handle a uart interrupt, raised because input has
7951 // arrived, or the uart is ready for more output, or
7952 // both. called from devintr().
7953 void
7954 uartintr(void)
7955 {
7956     ReadReg(ISR); // acknowledge the interrupt
7957
7958     acquire(&tx_lock);
7959     if (ReadReg(LSR) & LSR_TX_IDLE) {
7960         // UART finished transmitting; wake up sending thread.
7961         tx_busy = 0;
7962         wakeup(&tx_chan);
7963     }
7964     release(&tx_lock);
7965
7966     // read and process incoming characters, if any.
7967     while (1) {
7968         int c = uartgetc();
7969         if (c == -1)
7970             break;
7971         consoleintr(c);
7972     }
7973 }
7974
7975
7976
7977
7978
7979
7980
7981
7982
7983
7984
7985
7986
7987
7988
7989
7990
7991
7992
7993
7994
7995
7996
7997
7998
7999
```

```
8000 //
8001 // driver for qemu's virtio disk device.
8002 // uses qemu's mmio interface to virtio.
8003 //
8004 // qemu ... -drive file=fs.img,if=none,format=raw,id=x0 -device
8005 //          virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
8006 //
8007
8008 #include "types.h"
8009 #include "riscv.h"
8010 #include "defs.h"
8011 #include "param.h"
8012 #include "memlayout.h"
8013 #include "spinlock.h"
8014 #include "sleeplock.h"
8015 #include "fs.h"
8016 #include "buf.h"
8017 #include "virtio.h"
8018
8019 // the address of virtio mmio register r.
8020 #define R(r) ((volatile uint32 *) (VIRTIO0 + (r)))
8021
8022 static struct disk {
8023     // a set (not a ring) of DMA descriptors, with which the
8024     // driver tells the device where to read and write individual
8025     // disk operations. there are NUM descriptors.
8026     // most commands consist of a "chain" (a linked list) of a couple of
8027     // these descriptors.
8028     struct virtq_desc *desc;
8029
8030     // a ring in which the driver writes descriptor numbers
8031     // that the driver would like the device to process. it only
8032     // includes the head descriptor of each chain. the ring has
8033     // NUM elements.
8034     struct virtq_avail *avail;
8035
8036     // a ring in which the device writes descriptor numbers that
8037     // the device has finished processing (just the head of each chain).
8038     // there are NUM used ring entries.
8039     struct virtq_used *used;
8040
8041     // our own book-keeping.
8042     char free[NUM]; // is a descriptor free?
8043     uint16 used_idx; // we've looked this far in used[2..NUM].
8044
8045     // track info about in-flight operations,
8046     // for use when completion interrupt arrives.
8047     // indexed by first descriptor index of chain.
8048     struct {
8049         struct buf *b;
```

```

8050     char status;
8051 } info[NUM];
8052
8053 // disk command headers.
8054 // one-for-one with descriptors, for convenience.
8055 struct virtio_blk_req ops[NUM];
8056
8057 struct spinlock vdisk_lock;
8058
8059 } disk;
8060
8061 void
8062 virtio_disk_init(void)
8063 {
8064     uint32 status = 0;
8065
8066     initlock(&disk.vdisk_lock, "virtio_disk");
8067
8068     if (*R(VIRTIO_MMIO_MAGIC_VALUE) != 0x74726976 ||
8069         *R(VIRTIO_MMIO_VERSION) != 2 || *R(VIRTIO_MMIO_DEVICE_ID) != 2 ||
8070         *R(VIRTIO_MMIO_VENDOR_ID) != 0x554d4551) {
8071         panic("could not find virtio disk");
8072     }
8073
8074     // reset device
8075     *R(VIRTIO_MMIO_STATUS) = status;
8076
8077     // set ACKNOWLEDGE status bit
8078     status |= VIRTIO_CONFIG_S_ACKNOWLEDGE;
8079     *R(VIRTIO_MMIO_STATUS) = status;
8080
8081     // set DRIVER status bit
8082     status |= VIRTIO_CONFIG_S_DRIVER;
8083     *R(VIRTIO_MMIO_STATUS) = status;
8084
8085     // negotiate features
8086     uint64 features = *R(VIRTIO_MMIO_DEVICE_FEATURES);
8087     features &= ~(1 << VIRTIO_BLK_F_RO);
8088     features &= ~(1 << VIRTIO_BLK_F_SCSI);
8089     features &= ~(1 << VIRTIO_BLK_F_CONFIG_WCE);
8090     features &= ~(1 << VIRTIO_BLK_F_MQ);
8091     features &= ~(1 << VIRTIO_F_ANY_LAYOUT);
8092     features &= ~(1 << VIRTIO_RING_F_EVENT_IDX);
8093     features &= ~(1 << VIRTIO_RING_F_INDIRECT_DESC);
8094     *R(VIRTIO_MMIO_DRIVER_FEATURES) = features;
8095
8096     // tell device that feature negotiation is complete.
8097     status |= VIRTIO_CONFIG_S_FEATURES_OK;
8098     *R(VIRTIO_MMIO_STATUS) = status;
8099

```

```

8100 // re-read status to ensure FEATURES_OK is set.
8101 status = *R(VIRTIO_MMIO_STATUS);
8102 if (!(status & VIRTIO_CONFIG_S_FEATURES_OK))
8103     panic("virtio disk FEATURES_OK unset");
8104
8105 // initialize queue 0.
8106 *R(VIRTIO_MMIO_QUEUE_SEL) = 0;
8107
8108 // ensure queue 0 is not in use.
8109 if (*R(VIRTIO_MMIO_QUEUE_READY))
8110     panic("virtio disk should not be ready");
8111
8112 // check maximum queue size.
8113 uint32 max = *R(VIRTIO_MMIO_QUEUE_NUM_MAX);
8114 if (max == 0)
8115     panic("virtio disk has no queue 0");
8116 if (max < NUM)
8117     panic("virtio disk max queue too short");
8118
8119 // allocate and zero queue memory.
8120 disk.desc = kalloc();
8121 disk.avail = kalloc();
8122 disk.used = kalloc();
8123 if (!disk.desc || !disk.avail || !disk.used)
8124     panic("virtio disk kalloc");
8125 memset(disk.desc, 0, PGSIZE);
8126 memset(disk.avail, 0, PGSIZE);
8127 memset(disk.used, 0, PGSIZE);
8128
8129 // set queue size.
8130 *R(VIRTIO_MMIO_QUEUE_NUM) = NUM;
8131
8132 // write physical addresses.
8133 *R(VIRTIO_MMIO_QUEUE_DESC_LOW) = (uint64)disk.desc;
8134 *R(VIRTIO_MMIO_QUEUE_DESC_HIGH) = (uint64)disk.desc >> 32;
8135 *R(VIRTIO_MMIO_DRIVER_DESC_LOW) = (uint64)disk.avail;
8136 *R(VIRTIO_MMIO_DRIVER_DESC_HIGH) = (uint64)disk.avail >> 32;
8137 *R(VIRTIO_MMIO_DEVICE_DESC_LOW) = (uint64)disk.used;
8138 *R(VIRTIO_MMIO_DEVICE_DESC_HIGH) = (uint64)disk.used >> 32;
8139
8140 // queue is ready.
8141 *R(VIRTIO_MMIO_QUEUE_READY) = 0x1;
8142
8143 // all NUM descriptors start out unused.
8144 for (int i = 0; i < NUM; i++)
8145     disk.free[i] = 1;
8146
8147 // tell device we're completely ready.
8148 status |= VIRTIO_CONFIG_S_DRIVER_OK;
8149 *R(VIRTIO_MMIO_STATUS) = status;

```

```
8150 // plic.c and trap.c arrange for interrupts from VIRTIO0_IRQ.
8151 }
8152
8153 // find a free descriptor, mark it non-free, return its index.
8154 static int
8155 alloc_desc()
8156 {
8157     for (int i = 0; i < NUM; i++) {
8158         if (disk.free[i]) {
8159             disk.free[i] = 0;
8160             return i;
8161         }
8162     }
8163     return -1;
8164 }
8165
8166 // mark a descriptor as free.
8167 static void
8168 free_desc(int i)
8169 {
8170     if (i >= NUM)
8171         panic("free_desc 1");
8172     if (disk.free[i])
8173         panic("free_desc 2");
8174     disk.desc[i].addr = 0;
8175     disk.desc[i].len = 0;
8176     disk.desc[i].flags = 0;
8177     disk.desc[i].next = 0;
8178     disk.free[i] = 1;
8179     wakeup(&disk.free[0]);
8180 }
8181
8182 // free a chain of descriptors.
8183 static void
8184 free_chain(int i)
8185 {
8186     while (1) {
8187         int flag = disk.desc[i].flags;
8188         int nxt = disk.desc[i].next;
8189         free_desc(i);
8190         if (flag & VRING_DESC_F_NEXT)
8191             i = nxt;
8192         else
8193             break;
8194     }
8195 }
8196
8197
8198
8199
```

```

8200 // allocate three descriptors (they need not be contiguous).
8201 // disk transfers always use three descriptors.
8202 static int
8203 alloc3_desc(int *idx)
8204 {
8205     for (int i = 0; i < 3; i++) {
8206         idx[i] = alloc_desc();
8207         if (idx[i] < 0) {
8208             for (int j = 0; j < i; j++)
8209                 free_desc(idx[j]);
8210             return -1;
8211         }
8212     }
8213     return 0;
8214 }
8215
8216 void
8217 virtio_disk_rw(struct buf *b, int write)
8218 {
8219     uint64 sector = b->blockno * (BSIZE / 512);
8220
8221     acquire(&disk.vdisk_lock);
8222
8223     // the spec's Section 5.2 says that legacy block operations use
8224     // three descriptors: one for type/reserved/sector, one for the
8225     // data, one for a 1-byte status result.
8226
8227     // allocate the three descriptors.
8228     int idx[3];
8229     while (1) {
8230         if (alloc3_desc(idx) == 0) {
8231             break;
8232         }
8233         sleep(&disk.free[0], &disk.vdisk_lock);
8234     }
8235
8236     // format the three descriptors.
8237     // qemu's virtio-blk.c reads them.
8238
8239     struct virtio_blk_req *buf0 = &disk.ops[idx[0]];
8240
8241     if (write)
8242         buf0->type = VIRTIO_BLK_T_OUT; // write the disk
8243     else
8244         buf0->type = VIRTIO_BLK_T_IN; // read the disk
8245     buf0->reserved = 0;
8246     buf0->sector = sector;
8247
8248
8249

```

```

8250 disk.desc[idx[0]].addr = (uint64)buf0;
8251 disk.desc[idx[0]].len = sizeof(struct virtio_blk_req);
8252 disk.desc[idx[0]].flags = VRING_DESC_F_NEXT;
8253 disk.desc[idx[0]].next = idx[1];
8254
8255 disk.desc[idx[1]].addr = (uint64)b->data;
8256 disk.desc[idx[1]].len = BSIZE;
8257 if (write)
8258     disk.desc[idx[1]].flags = 0; // device reads b->data
8259 else
8260     disk.desc[idx[1]].flags = VRING_DESC_F_WRITE; // device writes b->data
8261 disk.desc[idx[1]].flags |= VRING_DESC_F_NEXT;
8262 disk.desc[idx[1]].next = idx[2];
8263
8264 disk.info[idx[0]].status = 0xff; // device writes 0 on success
8265 disk.desc[idx[2]].addr = (uint64)&disk.info[idx[0]].status;
8266 disk.desc[idx[2]].len = 1;
8267 disk.desc[idx[2]].flags = VRING_DESC_F_WRITE; // device writes the
8268                                         // status
8269 disk.desc[idx[2]].next = 0;
8270
8271 // record struct buf for virtio_disk_intr().
8272 b->disk = 1;
8273 disk.info[idx[0]].b = b;
8274
8275 // tell the device the first index in our chain of descriptors.
8276 disk.avail->ring[disk.avail->idx % NUM] = idx[0];
8277
8278 __atomic_thread_fence(__ATOMIC_SEQ_CST);
8279
8280 // tell the device another avail ring entry is available.
8281 disk.avail->idx += 1; // not % NUM ...
8282
8283 __atomic_thread_fence(__ATOMIC_SEQ_CST);
8284
8285 *R(VIRTIO_MMIO_QUEUE_NOTIFY) = 0; // value is queue number
8286
8287 // Wait for virtio_disk_intr() to say request has finished.
8288 while (b->disk == 1) {
8289     sleep(b, &disk.vdisk_lock);
8290 }
8291
8292 disk.info[idx[0]].b = 0;
8293 free_chain(idx[0]);
8294
8295 release(&disk.vdisk_lock);
8296 }
8297
8298
8299

```

```

8300 void
8301 virtio_disk_intr()
8302 {
8303     acquire(&disk.vdisk_lock);
8304
8305     // the device won't raise another interrupt until we tell it
8306     // we've seen this interrupt, which the following line does.
8307     // this may race with the device writing new entries to
8308     // the "used" ring, in which case we may process the new
8309     // completion entries in this interrupt, and have nothing to do
8310     // in the next interrupt, which is harmless.
8311     *R(VIRTIO_MMIO_INTERRUPT_ACK) = *R(VIRTIO_MMIO_INTERRUPT_STATUS) & 0x3;
8312
8313     __atomic_thread_fence(__ATOMIC_SEQ_CST);
8314
8315     // the device increments disk.used->idx when it
8316     // adds an entry to the used ring.
8317
8318     while (disk.used_idx != disk.used->idx) {
8319         __atomic_thread_fence(__ATOMIC_SEQ_CST);
8320         int id = disk.used->ring[disk.used_idx % NUM].id;
8321
8322         if (disk.info[id].status != 0)
8323             panic("virtio_disk_intr status");
8324
8325         struct buf *b = disk.info[id].b;
8326         b->disk = 0; // disk is done with buf
8327         wakeup(b);
8328
8329         disk.used_idx += 1;
8330     }
8331
8332     release(&disk.vdisk_lock);
8333 }
8334
8335
8336
8337
8338
8339
8340
8341
8342
8343
8344
8345
8346
8347
8348
8349

```

```
8350 // init: The initial user-level program
8351
8352 #include "kernel/types.h"
8353 #include "kernel/stat.h"
8354 #include "kernel/spinlock.h"
8355 #include "kernel/sleeplock.h"
8356 #include "kernel/fs.h"
8357 #include "kernel/file.h"
8358 #include "user/user.h"
8359 #include "kernel/fcntl.h"
8360
8361 char *argv[] = {"sh", 0};
8362
8363 int
8364 main(void)
8365 {
8366     int pid, wpid;
8367
8368     if (open("console", O_RDWR) < 0) {
8369         mknod("console", CONSOLE, 0);
8370         open("console", O_RDWR);
8371     }
8372     dup(0); // stdout
8373     dup(0); // stderr
8374     // create the null node
8375     mknod("null", NULLDEV, 0);
8376
8377     for (;;) {
8378         printf("init: starting sh\n");
8379         pid = fork();
8380         if (pid < 0) {
8381             printf("init: fork failed\n");
8382             exit(1);
8383         }
8384         if (pid == 0) {
8385             exec("sh", argv);
8386             printf("init: exec sh failed\n");
8387             exit(1);
8388         }
8389
8390         for (;;) {
8391             // this call to wait() returns if the shell exits,
8392             // or if a parentless process exits.
8393             wpid = wait((int *)0);
8394             if (wpid == pid) {
8395                 // the shell exited; restart it.
8396                 break;
8397             } else if (wpid < 0) {
8398                 printf("init: wait returned an error\n");
8399                 exit(1);
8400             }
8401         }
8402     }
8403 }
```

```
8400         } else {
8401             // it was a parentless process; do nothing.
8402         }
8403     }
8404 }
8405 }
8406
8407
8408
8409
8410
8411
8412
8413
8414
8415
8416
8417
8418
8419
8420
8421
8422
8423
8424
8425
8426
8427
8428
8429
8430
8431
8432
8433
8434
8435
8436
8437
8438
8439
8440
8441
8442
8443
8444
8445
8446
8447
8448
8449
```

```
8450 // Shell.
8451
8452 #include "kernel/types.h"
8453 #include "user/user.h"
8454 #include "kernel/fcntl.h"
8455
8456 // Parsed command representation
8457 #define EXEC 1
8458 #define REDIR 2
8459 #define PIPE 3
8460 #define LIST 4
8461 #define BACK 5
8462
8463 #define MAXARGS 10
8464
8465 struct cmd {
8466     int type;
8467 };
8468
8469 struct execcmd {
8470     int type;
8471     char *argv[MAXARGS];
8472     char *eargv[MAXARGS];
8473 };
8474
8475 struct redircmd {
8476     int type;
8477     struct cmd *cmd;
8478     char *file;
8479     char *efile;
8480     int mode;
8481     int fd;
8482 };
8483
8484 struct pipecmd {
8485     int type;
8486     struct cmd *left;
8487     struct cmd *right;
8488 };
8489
8490 struct listcmd {
8491     int type;
8492     struct cmd *left;
8493     struct cmd *right;
8494 };
8495
8496 struct backcmd {
8497     int type;
8498     struct cmd *cmd;
8499 };
```

```

8500 // Define the default search paths
8501 char *path[] = {
8502     "",          // Current directory (empty string or ".")
8503     "/",         // Root directory
8504     "/bin",      // Standard binary directory
8505     "/usr/bin",  // user programs
8506     0           // Null terminator for the array loop
8507 };
8508
8509
8510
8511 int fork1(void); // Fork but panics on failure.
8512 void panic(char *);
8513 struct cmd *parsecmd(char *);
8514 void runcmd(struct cmd *) __attribute__((noreturn));
8515
8516 // Execute cmd. Never returns.
8517 void
8518 runcmd(struct cmd *cmd)
8519 {
8520     int p[2];
8521     struct backcmd *bcmd;
8522     struct execcmd *ecmd;
8523     struct listcmd *lcmd;
8524     struct pipecmd *pcmd;
8525     struct redircmd *rcmd;
8526
8527     if (cmd == 0)
8528         exit(1);
8529
8530     switch (cmd->type) {
8531     default:
8532         panic("runcmd");
8533
8534     case EXEC:
8535         ecmd = (struct execcmd*)cmd;
8536         if(ecmd->argv[0] == 0)
8537             exit(1);
8538
8539         // If the command already has an explicit path slash, execute
8540         //                                     it directly
8541         if(strchr(ecmd->argv[0], '/') != 0) {
8542             exec(ecmd->argv[0], ecmd->argv);
8543         } else {
8544             // Loop through all defined paths
8545             for(int i = 0; path[i] != 0; i++) {
8546                 char buf[64]; // Reallocated as a safe-sized array
8547                 int path_len = strlen(path[i]);
8548                 int cmd_len = strlen(ecmd->argv[0]);
8549

```

```

8550 // 1. Check if the constructed path will fit inside our
8551 // 64-byte buffer
8552 // (+2 accounts for a potential hidden extra "/" and the
8553 // null terminator)
8554 if(path_len + cmd_len + 2 > sizeof(buf)) {
8555     continue; // Skip paths that are too long to prevent crashes
8556 }
8557
8558 // 2. Handle current directory vs absolute paths
8559 if(path_len == 0) {
8560     strcpy(buf, ecmd->argv[0]);
8561 } else {
8562     strcpy(buf, path[i]);
8563
8564     // Append trailing slash if the array path doesn't end with one
8565     if(buf[path_len - 1] != '/') {
8566         buf[path_len] = '/';
8567         buf[path_len + 1] = '\0';
8568     }
8569
8570     // Append the command name safely using the offset pointer
8571     strcpy(buf + strlen(buf), ecmd->argv[0]);
8572 }
8573
8574 // Try executing the built path string
8575 exec(buf, ecmd->argv);
8576 }
8577 }
8578
8579 fprintf(2, "exec %s failed\n", ecmd->argv[0]);
8580 break;
8581
8582
8583
8584 case REDIR:
8585     rcmd = (struct redircmd *)cmd;
8586     close(rcmd->fd);
8587     if (open(rcmd->file, rcmd->mode) < 0) {
8588         fprintf(2, "open %s failed\n", rcmd->file);
8589         exit(1);
8590     }
8591 // If this is an append operation (identified because O_TRUNC is missing)
8592 if((rcmd->mode & O_TRUNC) == 0 && rcmd->mode != O_RDONLY){
8593     // Use seek system call to jump to the very end of the file
8594     lseek(rcmd->fd, 0, 2); // 2 usually represents SEEK_END or
8595                          // end-of-file in xv6
8596 }
8597 runcmd(rcmd->cmd);
8598 break;
8599

```

```

8600 case LIST:
8601     lcmd = (struct listcmd *)cmd;
8602     if (fork1() == 0)
8603         runcmd(lcmd->left);
8604     wait(0);
8605     runcmd(lcmd->right);
8606     break;
8607
8608 case PIPE:
8609     pcmd = (struct pipecmd *)cmd;
8610     if (pipe(p) < 0)
8611         panic("pipe");
8612     if (fork1() == 0) {
8613         close(1);
8614         dup(p[1]);
8615         close(p[0]);
8616         close(p[1]);
8617         runcmd(pcmd->left);
8618     }
8619     if (fork1() == 0) {
8620         close(0);
8621         dup(p[0]);
8622         close(p[0]);
8623         close(p[1]);
8624         runcmd(pcmd->right);
8625     }
8626     close(p[0]);
8627     close(p[1]);
8628     wait(0);
8629     wait(0);
8630     break;
8631
8632 case BACK:
8633     bcmd = (struct backcmd *)cmd;
8634     int bg_pid = fork1();
8635     if(bg_pid == 0){
8636         runcmd(bcmd->cmd);
8637     } else if(bg_pid > 0){
8638         // Print the PID to the console
8639         // fprintf(2, "[PID: %d]\n", bg_pid);
8640     }
8641     break;
8642 }
8643 exit(0);
8644 }
8645
8646
8647
8648
8649

```

```

8650 int
8651 getcmd(char *buf, int nbuf)
8652 {
8653     write(2, "$ ", 2);
8654     memset(buf, 0, nbuf);
8655     gets(buf, nbuf);
8656     if (buf[0] == 0) // EOF
8657         return -1;
8658     return 0;
8659 }
8660
8661 int
8662 main(void)
8663 {
8664     static char buf[100];
8665     int fd;
8666
8667     // Ensure that three file descriptors are open.
8668     while ((fd = open("console", O_RDWR)) >= 0) {
8669         if (fd >= 3) {
8670             close(fd);
8671             break;
8672         }
8673     }
8674
8675     // Read and run input commands.
8676     while (getcmd(buf, sizeof(buf)) >= 0) {
8677         char *cmd = buf;
8678         int print_time=0;
8679         while (*cmd == ' ' || *cmd == '\t')
8680             cmd++;
8681         if (*cmd == '\n') // is a blank command
8682             continue;
8683
8684         if(strcmp(buf, "exit\n") == 0 || strcmp(buf, "exit") == 0){
8685             exit(0);
8686         }
8687
8688         if (cmd[0] == 't' && cmd[1] == 'i' && cmd[2] == 'm' &&
8689             cmd[3] == 'e' && cmd[4] == ' ' ) {
8690             cmd += 5;
8691             print_time=1;
8692         }
8693
8694         if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
8695             // Chdir must be called by the parent, not the child.
8696             cmd[strlen(cmd) - 1] = 0; // chop \n
8697             if (chdir(cmd + 3) < 0)
8698                 fprintf(2, "cannot cd %s\n", cmd + 3);
8699         } else {

```

```
8700     uint64 start_time = gettime(1); // Get starting time in nanoseconds
8701     if (fork1() == 0)
8702         runcmd(parsecmd(cmd));
8703     wait(0);
8704     uint64 end_time = gettime(1); // Get ending time in nanoseconds
8705     uint64 elapsed_nsecs = end_time - start_time;
8706     if ( print_time ) {
8707         printf("\nExecution time: %ld msecs\n",
8708             elapsed_nsecs / 1000000);
8709     }
8710 }
8711 }
8712 exit(0);
8713 }
8714
8715 void
8716 panic(char *s)
8717 {
8718     fprintf(2, "%s\n", s);
8719     exit(1);
8720 }
8721
8722 int
8723 fork1(void)
8724 {
8725     int pid;
8726
8727     pid = fork();
8728     if (pid == -1)
8729         panic("fork");
8730     return pid;
8731 }
8732
8733
8734
8735
8736
8737
8738
8739
8740
8741
8742
8743
8744
8745
8746
8747
8748
8749
```

```
8750 // Constructors
8751
8752 struct cmd *
8753 execcmd(void)
8754 {
8755     struct execcmd *cmd;
8756
8757     cmd = malloc(sizeof(*cmd));
8758     memset(cmd, 0, sizeof(*cmd));
8759     cmd->type = EXEC;
8760     return (struct cmd *)cmd;
8761 }
8762
8763 struct cmd *
8764 redircmd(struct cmd *subcmd, char *file, char *efile, int mode, int fd)
8765 {
8766     struct redircmd *cmd;
8767
8768     cmd = malloc(sizeof(*cmd));
8769     memset(cmd, 0, sizeof(*cmd));
8770     cmd->type = REDIR;
8771     cmd->cmd = subcmd;
8772     cmd->file = file;
8773     cmd->efile = efile;
8774     cmd->mode = mode;
8775     cmd->fd = fd;
8776     return (struct cmd *)cmd;
8777 }
8778
8779 struct cmd *
8780 pipecmd(struct cmd *left, struct cmd *right)
8781 {
8782     struct pipecmd *cmd;
8783
8784     cmd = malloc(sizeof(*cmd));
8785     memset(cmd, 0, sizeof(*cmd));
8786     cmd->type = PIPE;
8787     cmd->left = left;
8788     cmd->right = right;
8789     return (struct cmd *)cmd;
8790 }
8791
8792
8793
8794
8795
8796
8797
8798
8799
```

```
8800 struct cmd *
8801 listcmd(struct cmd *left, struct cmd *right)
8802 {
8803     struct listcmd *cmd;
8804
8805     cmd = malloc(sizeof(*cmd));
8806     memset(cmd, 0, sizeof(*cmd));
8807     cmd->type = LIST;
8808     cmd->left = left;
8809     cmd->right = right;
8810     return (struct cmd *)cmd;
8811 }
8812
8813 struct cmd *
8814 backcmd(struct cmd *subcmd)
8815 {
8816     struct backcmd *cmd;
8817
8818     cmd = malloc(sizeof(*cmd));
8819     memset(cmd, 0, sizeof(*cmd));
8820     cmd->type = BACK;
8821     cmd->cmd = subcmd;
8822     return (struct cmd *)cmd;
8823 }
8824
8825
8826
8827
8828
8829
8830
8831
8832
8833
8834
8835
8836
8837
8838
8839
8840
8841
8842
8843
8844
8845
8846
8847
8848
8849
```

```
8850 // Parsing
8851
8852 char whitespace[] = " \t\r\n\v";
8853 char symbols[] = "<|>&()";
8854
8855 int
8856 gettoken(char **ps, char *es, char **q, char **eq)
8857 {
8858     char *s;
8859     int ret;
8860
8861     s = *ps;
8862     while (s < es && strchr(whitespace, *s))
8863         s++;
8864     if (q)
8865         *q = s;
8866     ret = *s;
8867     switch (*s) {
8868     case 0:
8869         break;
8870     case '|':
8871     case '(':
8872     case ')':
8873     case ';':
8874     case '&':
8875     case '<':
8876         s++;
8877         break;
8878     case '>':
8879         s++;
8880         if (*s == '>') {
8881             ret = '+';
8882             s++;
8883         }
8884         break;
8885     default:
8886         ret = 'a';
8887         while (s < es && !strchr(whitespace, *s) && !strchr(symbols, *s))
8888             s++;
8889         break;
8890     }
8891     if (eq)
8892         *eq = s;
8893
8894     while (s < es && strchr(whitespace, *s))
8895         s++;
8896     *ps = s;
8897     return ret;
8898 }
8899
```

```
8900 int
8901 peek(char **ps, char *es, char *toks)
8902 {
8903     char *s;
8904
8905     s = *ps;
8906     while (s < es && strchr(whitespace, *s))
8907         s++;
8908     *ps = s;
8909     return *s && strchr(toks, *s);
8910 }
8911
8912 struct cmd *parseline(char **, char *);
8913 struct cmd *parsepipe(char **, char *);
8914 struct cmd *parseexec(char **, char *);
8915 struct cmd *nulterminate(struct cmd *);
8916
8917 struct cmd *
8918 parsecmd(char *s)
8919 {
8920     char *es;
8921     struct cmd *cmd;
8922
8923     es = s + strlen(s);
8924     cmd = parseline(&s, es);
8925     peek(&s, es, "");
8926     if (s != es) {
8927         fprintf(2, "leftovers: %s\n", s);
8928         panic("syntax");
8929     }
8930     nulterminate(cmd);
8931     return cmd;
8932 }
8933
8934 struct cmd *
8935 parseline(char **ps, char *es)
8936 {
8937     struct cmd *cmd;
8938
8939     cmd = parsepipe(ps, es);
8940     while (peek(ps, es, "&")) {
8941         gettoken(ps, es, 0, 0);
8942         cmd = backcmd(cmd);
8943     }
8944     if (peek(ps, es, ";")) {
8945         gettoken(ps, es, 0, 0);
8946         cmd = listcmd(cmd, parseline(ps, es));
8947     }
8948     return cmd;
8949 }
```

```

8950 struct cmd *
8951 parsepipe(char **ps, char *es)
8952 {
8953     struct cmd *cmd;
8954
8955     cmd = parseexec(ps, es);
8956     if (peek(ps, es, "|")) {
8957         gettoken(ps, es, 0, 0);
8958         cmd = pipecmd(cmd, parsepipe(ps, es));
8959     }
8960     return cmd;
8961 }
8962
8963 struct cmd *
8964 parseredirs(struct cmd *cmd, char **ps, char *es)
8965 {
8966     int tok;
8967     char *q, *eq;
8968
8969     while (peek(ps, es, "<>")) {
8970         tok = gettoken(ps, es, 0, 0);
8971         if (gettoken(ps, es, &q, &eq) != 'a')
8972             panic("missing file for redirection");
8973         switch (tok) {
8974             case '<':
8975                 cmd = redircmd(cmd, q, eq, O_RDONLY, 0);
8976                 break;
8977             case '>':
8978                 cmd = redircmd(cmd, q, eq, O_WRONLY | O_CREATE | O_TRUNC, 1);
8979                 break;
8980             case '+': // >>
8981                 cmd = redircmd(cmd, q, eq, O_WRONLY | O_CREATE, 1);
8982                 break;
8983         }
8984     }
8985     return cmd;
8986 }
8987
8988
8989
8990
8991
8992
8993
8994
8995
8996
8997
8998
8999

```

```

9000 struct cmd *
9001 parseblock(char **ps, char *es)
9002 {
9003     struct cmd *cmd;
9004
9005     if (!peek(ps, es, "("))
9006         panic("parseblock");
9007     gettoken(ps, es, 0, 0);
9008     cmd = parseline(ps, es);
9009     if (!peek(ps, es, ")"))
9010         panic("syntax - missing )");
9011     gettoken(ps, es, 0, 0);
9012     cmd = parseredirs(cmd, ps, es);
9013     return cmd;
9014 }
9015
9016 struct cmd *
9017 parseexec(char **ps, char *es)
9018 {
9019     char *q, *eq;
9020     int tok, argc;
9021     struct execcmd *cmd;
9022     struct cmd *ret;
9023
9024     if (peek(ps, es, "("))
9025         return parseblock(ps, es);
9026
9027     ret = execcmd();
9028     cmd = (struct execcmd *)ret;
9029
9030     argc = 0;
9031     ret = parseredirs(ret, ps, es);
9032     while (!peek(ps, es, "|)&")) {
9033         if ((tok = gettoken(ps, es, &q, &eq)) == 0)
9034             break;
9035         if (tok != 'a')
9036             panic("syntax");
9037         cmd->argv[argc] = q;
9038         cmd->eargv[argc] = eq;
9039         argc++;
9040         if (argc >= MAXARGS)
9041             panic("too many args");
9042         ret = parseredirs(ret, ps, es);
9043     }
9044     cmd->argv[argc] = 0;
9045     cmd->eargv[argc] = 0;
9046     return ret;
9047 }
9048
9049

```

```
9050 // NUL-terminate all the counted strings.
9051 struct cmd *
9052 nulterminate(struct cmd *cmd)
9053 {
9054     int i;
9055     struct backcmd *bcmd;
9056     struct execcmd *ecmd;
9057     struct listcmd *lcmd;
9058     struct pipecmd *pcmd;
9059     struct redircmd *rcmd;
9060
9061     if (cmd == 0)
9062         return 0;
9063
9064     switch (cmd->type) {
9065     case EXEC:
9066         ecmd = (struct execcmd *)cmd;
9067         for (i = 0; ecmd->argv[i]; i++)
9068             *ecmd->eargv[i] = 0;
9069         break;
9070
9071     case REDIR:
9072         rcmd = (struct redircmd *)cmd;
9073         nulterminate(rcmd->cmd);
9074         *rcmd->efile = 0;
9075         break;
9076
9077     case PIPE:
9078         pcmd = (struct pipecmd *)cmd;
9079         nulterminate(pcmd->left);
9080         nulterminate(pcmd->right);
9081         break;
9082
9083     case LIST:
9084         lcmd = (struct listcmd *)cmd;
9085         nulterminate(lcmd->left);
9086         nulterminate(lcmd->right);
9087         break;
9088
9089     case BACK:
9090         bcmd = (struct backcmd *)cmd;
9091         nulterminate(bcmd->cmd);
9092         break;
9093     }
9094     return cmd;
9095 }
9096
9097
9098
9099
```

```

9100 OUTPUT_ARCH( "riscv" )
9101 ENTRY( _entry )
9102
9103 SECTIONS
9104 {
9105     /*
9106      * ensure that entry.S / _entry is at 0x80000000,
9107      * where qemu's -kernel jumps.
9108      */
9109     . = 0x80000000;
9110
9111     .text : {
9112         kernel/entry.o(_entry)
9113         *(.text .text.*)
9114         . = ALIGN(0x1000);
9115         _trampoline = .;
9116         *(trampsec)
9117         . = ALIGN(0x1000);
9118         ASSERT(. - _trampoline == 0x1000,
9119             "error: trampoline larger than one page");
9120         PROVIDE(etext = .);
9121     }
9122
9123     .rodata : {
9124         . = ALIGN(16);
9125         *(.srodata .srodata.*) /* do not need to distinguish this
9126                                     from .rodata */
9127         . = ALIGN(16);
9128         *(.rodata .rodata.*)
9129     }
9130
9131     .data : {
9132         . = ALIGN(16);
9133         *(.sdata .sdata.*) /* do not need to distinguish this from .data */
9134         . = ALIGN(16);
9135         *(.data .data.*)
9136     }
9137
9138     .bss : {
9139         . = ALIGN(16);
9140         *(.sbss .sbss.*) /* do not need to distinguish this from .bss */
9141         . = ALIGN(16);
9142         *(.bss .bss.*)
9143     }
9144
9145     PROVIDE(end = .);
9146 }
9147
9148
9149

```